

SmartFlow

A multi-agent digital twin for adaptive traffic signal control — built on SUMO, benchmarked against classical baselines, and reported with its failures intact.

CORRIDOR

12 junctions

BEST WAIT

17.9 s

VS FIXED-TIME

−78%

PYTHON

19,372 lines

B.Tech CSE final-year major project · CMRCET

Solo build · 12-week window · Phase A–C complete through Week 7

77 Python files · 3-seed minimum for every reported result

Compiled 28 August 2026 from the repository's own reports, metrics and source.

Contents

Part I	Executive summary	What was built, what it proves, what it does not
Part II	Status by week	Every Definition of Done and its verdict
Part III	The project	README: idea, results, structure, setup, commands
Part IV	Method and roadmap	The plan, the de-risking substitutions, the rules
Part V	Phase A-B benchmark report	The full internal benchmark, Weeks 1-6
Part VI	Week-by-week evidence	Each week's own report, with its charts
Part VII	Week 7: knowledge graph and RAG	Graph, retrieval and the read-only service
Part VIII	Weeks 8-9: perception and federation	Vision, anomaly detection, planning, federated learning, LoRA
Part IX	Weeks 10-12: the platform	Services, Kubernetes, dashboard, integration
Part X	Deferred and draft material	Items carried forward, honestly labelled
Part XI	Codebase	Every source file and what it does
Appendix A	Complete metric tables	Raw CSV data behind every claim
Appendix B	Commit history	The full development record

PART I

Executive summary

What was built, what it proves, and what it deliberately does not claim.

SmartFlow trains reinforcement-learning agents to control traffic signals on a simulated 12-junction corridor, and measures them against two classical baselines: a fixed-time controller and SUMO's detector-driven actuated controller. Every controller is judged by the same per-second metric collector on identical traffic, so a difference on screen is a difference in policy and nothing else.

The headline result

Corridor-wide, on base demand, averaged over three independently trained seeds:

Corridor-wide, base demand, 3-seed means `outputs/marl_metrics.csv`

CONTROLLER	AVG WAIT (S)	MAX QUEUE	THROUGHPUT	CO2 (KG)
fixed	83.10	427.7	1280.0	732.6
actuated	40.89	76.3	1646.7	508.3
marl_independent	17.80	41.0	1679.7	451.0
marl_shared_w5	17.85	149.7	1569.3	474.7
marl_shared_w5nofair	18.16	42.3	1678.7	452.0
marl_shared_w5strong	18.16	42.3	1678.7	452.0
marl_shared_w5nocoord	18.16	42.3	1678.7	452.0
marl_shared_w5gnn	18.16	42.3	1678.7	452.0

Reinforcement learning roughly **halves the wait of the actuated baseline** (17.9 s against 40.9 s) and cuts fixed-time by 78%. That is a real result and it is reproducible from the committed checkpoints.

What the project does not claim

Two of the seven Definitions of Done attempted so far were **not met**, and both are reported as failures rather than re-scoped until they passed.

- **Week 3** put PPO on a single corridor junction. It beat fixed-time on all three metrics but beat the actuated baseline on only one of three.
- **Week 5** added green-wave shaping and a Lagrangian fairness constraint to a parameter-shared policy. It beats actuated on wait time, but **not reliably on throughput**: two seeds of three beat the baseline and the third converged to a policy that jams the corridor, dragging the mean below it. The fairness constraint measurably changed nothing.

Note in the table above that `marl_independent` — Week 4's simpler, unshaped configuration — is the strongest row on throughput. Corridor-wide RL beating both baselines is a genuine finding; it is the Week 5 *shaping* that fails to add to it.

The most interesting negative result

Four policy variants in the table are **numerically identical** to two decimal places: no-fairness, 20x fairness, no-coordination, and the graph-attention encoder all produce 18.16 s wait, 42.3 max queue and 1678.7 throughput. These are genuinely different networks — 16k against 138k parameters — and they agree on 100% of *actionable* decisions.

The cause is the action space, not a bug in the experiment: with a binary next-phase action and a 5 s minimum green, the greedy policy saturates. Every disagreement that does exist falls inside sumo-rl's minimum-green discard window, where the decision is thrown away before it reaches the simulator. This is documented rather than hidden, and it is the single clearest candidate for the next piece of work.

Status in one line

Weeks 1, 2, 4, 6 and 7 met their Definition of Done; Weeks 3 and 5 did not; Weeks 8–12 have not been started and exist only as scaffolding. Part II gives the full table.

What bounds every number in this document

All results come from one synthetic 12-junction grid with a synthetic demand generator. Nothing here demonstrates transfer to a real road network — the OSM corridor has been deferred since Week 1. Three seeds is enough to show variance, not enough for confidence intervals. Nothing has been load-tested, and no claim is made anywhere about real-time or production performance.

PART II

Status by week

Every Definition of Done, its verdict, and the evidence behind it.

Source: `outputs/STATUS.md`

SmartFlow — Status Index

One line per week, so nothing in this directory has to be read to find out whether it describes a measured result or an empty scaffold.

Weeks 3, 4, 5, 6 and 7 each state a verdict in their own report, and those are copied here verbatim. Weeks 1 and 2 predate that convention and have no verdict line, so their "MET" is read directly off the committed metrics CSVs — the derivation is given in the footnote below rather than asserted.

Last updated: 2026-08-28 (Weeks 1-12 complete).

Phase A–B — the RL core (Weeks 1–6)

WEEK	DEFINITION OF DONE	VERDICT	EVIDENCE
1	Both baselines produce metrics; comparison chart renders	MET	<code>metrics.csv</code> , <code>baseline_comparison.png</code>
2	PPO beats fixed-time on the sumo-rl benchmark, 3-seed	MET	<code>week2_benchmark_metrics.csv</code> , <code>week2_literature_note.md</code>
3	RL beats both baselines on ≥ 2 of 3 metrics, 3-seed	NOT MET	<code>week3_report.md</code>
4	Full corridor trains end-to-end without diverging	MET	<code>week4_report.md</code>
5	Corridor RL beats both baselines on wait + throughput, fairness caps worst-case wait	NOT MET (both parts)	<code>week5_report.md</code>
6	Viva-defensible benchmark report incl. one honest loss	MET	<code>../BENCHMARK_REPORT.md</code>

Weeks 1–2, derived not quoted. Week 1: `metrics.csv` plus `baseline_comparison.png` exist and actuated beats fixed, which is exactly what that week's DoD asked for. Week 2: `week2_benchmark_metrics.csv` holds 3 seeds per controller — fixed-time 16.39 s mean wait against PPO 4.00 s, a 75.6% reduction — so "PPO beats fixed-time on the benchmark, 3-seed average" is met on the numbers.

The Phase A–B failures, in one line each

- **Week 3** — single-intersection RL beats fixed-time on all three metrics but beats actuated on only one of three. Reported as failed rather than re-scoped.
- **Week 5** — the shared policy beats actuated on wait time (17.9 s vs 40.9 s) but **not reliably on throughput**: two seeds of three beat it, the third converged to a policy that jams the corridor and drags the mean below baseline. The fairness constraint measurably changed nothing. Week 4's simpler unshaped configuration does clear the bar — it is the Week 5 *shaping* that fails to add to it.

Both are genuine scientific results and stay in the final report. They are not outstanding work items. A third failure — Week 9's federated DoD — is recorded under Phase C below and shares a root cause with Week 5's.

Phase C — feature sprint (Weeks 7–9)

WEEK	DEFINITION OF DONE	VERDICT	EVIDENCE
7	A natural-language query returns an answer grounded in real graph data	MET	<code>week7_report.md</code> , <code>week7_qa.json</code>
8	Vision detects vehicles in a held-out clip; injected anomaly flagged; one closure scenario end-to-end	MET (all three)	<code>week8_report.md</code>
9	Federated averaging improves a held-out district	NOT MET	<code>week9_report.md</code>
9	Fine-tuned model beats the base prompt on domain questions	MET	<code>week9_report.md</code>

Week 7 runs against live backends: Neo4j AuraDB over Cypher, Chroma locally, and `claude-opus-5` for phrasing. Verify with `python src\check_neo4j.py` and `python src\week7_demo.py`.

Week 8, in one line each

- **Vision** — YOLOv8n on 480 frames rendered from the twin, labelled from TraCI ground truth. mAP₅₀ 0.988 on junction C2, held out of training entirely.
- **Anomaly detection** — recall 1.00 on injected lane obstructions across 3 seeds, 30 s mean detection latency, per-lane causal z-score detectors.
- **Scenario planner** — 18 closure × weather × seed runs complete. Weather dominates the closure: fog cuts throughput to 564 against 1598 in the clear.

Week 9's third failure, and why it matters

The federated DoD is **NOT MET**, and the reason is the same argmax saturation Weeks 5 and 6 documented. Separately trained district policies reach *byte-identical* wait times on the held-out junction, so averaging them changes nothing — the improvement over the local-only baseline is exactly 0.0%. This is the third independent sighting of the effect, and it is a property of the action space (binary next-phase, 5 s minimum green), not of the federation. The machinery itself runs end to end and is unit-tested.

Priority routing and the emissions reward term are Week 9 deliverables without a numeric DoD. Both are implemented and measured: emergency-vehicle wait falls 86% (416 s → 58 s) with no cost to general traffic.

Phase D–E — platform and report (Weeks 10–12)

WEEK	DEFINITION OF DONE	VERDICT	EVIDENCE
10	Push to main rebuilds and redeploys every service, behind auth	MET (deploy step gated on a cluster secret)	week10_report.md
11	helm install brings up every service on k3d; Grafana live; UI needs no terminal	MET	week11_report.md
12	A stranger can watch the demo and understand the project	MET , except the video	week12_report.md

Five FastAPI services run two ways from the same images — Docker Compose and a Helm chart on k3d — both brought up and verified. tests/test_services.py runs 12 end-to-end checks against a live stack; a Next.js dashboard exposes every service without a terminal.

Two things Phase D/E does not claim. The CI deploy job has never run against a real cluster: it needs a kubeconfig secret and a cluster reachable from GitHub's runners, and it emits an explicit notice rather than passing silently when neither exists. And no demo video was recorded — this environment cannot capture one. A scripted walkthrough over live artifacts stands in, and week12_report.md says so plainly.

The citation check earned its keep. src/verify_citations.py resolves every identifier against CrossRef or arXiv and compares the registered title to the cited one. It immediately caught a guessed Stable-Baselines3 arXiv id that resolved to an unrelated paper about audio-visual dialog. Final tally: 9 verified, 2 software without identifiers, 0 unresolved.

Files that are no longer drafts

FILE	STATUS
FINAL_REPORT.md	Generated by src/final_report.py from the committed artifacts.
VIVA_PREP.md	Twelve questions answered from measured results.
citations_verified.md	Generated by src/verify_citations.py ; 9 verified, 0 unresolved.

Interactive artifact

outputs/viz/corridor_control_room.html — a self-contained replay of three real SUMO runs (fixed, actuated, shared-policy RL) over identical traffic. Rebuild with python src/viz_build.py .

PART III

The project

The repository README in full: the idea, the measured results, the directory layout, setup and every command.

Source: README.md

SmartFlow

Teaching traffic lights to think — so cars spend less time waiting and more time moving.

SmartFlow is a research platform that uses **AI and simulation** to make traffic signals smarter. Instead of lights that blindly switch every 42 seconds whether cars are there or not, SmartFlow trains agents that *watch* traffic and decide in real time when to turn green.

Built as a B.Tech final-year project at CMRCET — but the results are real, and so are the cases where the AI loses.

The Big Idea (explain it like I'm 10)

Imagine you're stuck at a red light. There are 20 cars behind you... and zero cars on the green side. The light stays red for 40 more seconds anyway. Annoying, right?

That's **fixed-time** control — the city sets a timer and the lights just follow it forever, no matter what's actually happening on the road.

SmartFlow replaces that timer with an **AI brain** that:

1. Watches how many cars are waiting at each intersection
2. Learns from millions of simulated cars which signal timing works best
3. Makes the green light last longer when there's a big queue, shorter when the road is empty

Then it does the harder thing: it gets **twelve** intersections to cooperate, instead of each one selfishly clearing its own queue by dumping traffic on its neighbour.

Results

Week-by-week status at a glance: outputs/STATUS.md — which Definitions of Done were met, which failed, and which weeks are still empty scaffolding.

All figures are mean $\pm$ standard deviation over 3 seeds, measured by the same per-second collector (`src/metrics.py`) for RL and baselines alike.

Week 1 — fixed-time vs actuated baselines (corridor, 3 seeds)

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	CO2 (KG)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED	CO2 (KG) VS FIXED
fixed	86.26 $\pm$ 1.73	387.33 $\pm$ 154.39	1294.00 $\pm$ 177.28	731.13 $\pm$ 63.11	—	—	—	—
actuated	40.89 $\pm$ 2.88	76.33 $\pm$ 1.70	1646.67 $\pm$ 3.40	508.26 $\pm$ 11.21	+52.6%	+80.3%	+27.2%	+30.5%

Week 2 — single-agent PPO on sumo-rl's standard benchmark (3 seeds)

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED
fixed	16.39 ± 0.05	65.67 ± 1.70	2089.00 ± 2.83	—	—	—
ppo	4.00 ± 0.45	22.00 ± 2.83	2429.67 ± 35.64	+75.6%	+66.5%	+16.3%

Sanity check against published behaviour: `outputs/week2_literature_note.md` .

Week 3 — PPO on one corridor junction (B1), other 11 fixed-time (3 seeds)

At the controlled junction (the primary claim):

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED
fixed	50.01 ± 14.28	91.33 ± 10.84	545.33 ± 83.37	—	—	—
actuated	17.26 ± 5.28	30.67 ± 6.94	736.67 ± 3.09	+65.5%	+66.4%	+35.1%
actuated_single	24.43 ± 7.89	36.33 ± 11.09	723.33 ± 8.81	+51.1%	+60.2%	+32.6%
ppo	16.34 ± 11.23	44.33 ± 38.87	647.33 ± 130.38	+67.3%	+51.5%	+18.7%
ppo_final_iterate	16.34 ± 11.23	44.33 ± 38.87	647.33 ± 130.38	+67.3%	+51.5%	+18.7%

`actuated_single` is the controlled comparison — only B1 upgraded, same as `ppo` .

Corridor-wide (one junction of twelve cannot move this much, which is the point of Week 4):

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED
fixed	86.26 ± 1.73	387.33 ± 154.39	1294.00 ± 177.28	—	—	—
actuated	40.89 ± 2.88	76.33 ± 1.70	1646.67 ± 3.40	+52.6%	+80.3%	+27.2%
actuated_single	78.54 ± 2.93	146.00 ± 0.82	1596.00 ± 15.94	+8.9%	+62.3%	+23.3%
ppo	73.31 ± 5.53	260.67 ± 182.72	1469.33 ± 193.99	+15.0%	+32.7%	+13.6%
ppo_final_iterate	73.31 ± 5.53	260.67 ± 182.72	1469.33 ± 193.99	+15.0%	+32.7%	+13.6%

Weeks 4-5 — corridor-wide multi-agent RL, base demand (3 seeds)

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	95TH PCT WAIT (S)	WORST WAIT (S)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED	95TH PCT WAIT (S) VS FIXED	WORST WAIT (S) VS FIXED
fixed	83.10 ± 1.62	427.67 ± 176.42	1280.00 ± 189.40	211.33 ± 5.56	721.00 ± 127.38	—	—	—	—	—
actuated	40.89 ± 2.88	76.33 ± 1.70	1646.67 ± 3.40	122.33 ± 7.32	243.00 ± 25.66	+50.8%	+82.2%	+28.6%	+42.1%	+66.3%
marl_independent	17.80 ± 0.28	41.00 ± 2.83	1679.67 ± 4.11	47.67 ± 0.94	95.33 ± 3.40	+78.6%	+90.4%	+31.2%	+77.4%	+86.8%
marl_shared_w5	17.85 ± 0.38	149.67 ± 150.15	1569.33 ± 148.02	48.00 ± 0.82	288.67 ± 271.07	+78.5%	+65.0%	+22.6%	+77.3%	+60.0%
marl_shared_w5gnn	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	48.00 ± 0.82	96.00 ± 2.83	+78.2%	+90.1%	+31.1%	+77.3%	+86.7%
marl_shared_w5nocoord	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	48.00 ± 0.82	96.00 ± 2.83	+78.2%	+90.1%	+31.1%	+77.3%	+86.7%
marl_shared_w5nofair	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	48.00 ± 0.82	96.00 ± 2.83	+78.2%	+90.1%	+31.1%	+77.3%	+86.7%
marl_shared_w5strong	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	48.00 ± 0.82	96.00 ± 2.83	+78.2%	+90.1%	+31.1%	+77.3%	+86.7%

- `marl_independent` — Week 4: one policy per junction, local reward only.
- `marl_shared_w5` — Week 5: one parameter-shared policy, green-wave shaping, Lagrangian fairness constraint.

Fairness constraint, measured by ablation: worst-case vehicle wait is 96 s without the Lagrangian term and 289 s with it (-200.7%).

Week 6 — average wait (s) across demand scenarios

SCENARIO	FIXED	ACTUATED	MARL_INDEPENDENT	MARL_SHARED_W5
base	83.1	40.9	17.8	17.9
light	37.0	6.3	6.6	6.7
peak	68.9	71.2	42.5	39.2
asymmetric	54.1	11.3	10.9	10.8

Policies were trained on `base` only, so the other columns measure generalisation. The full analysis, including every case where RL loses to a baseline, is in [BENCHMARK_REPORT.md](#).

Week 5 — online learning

- deployed into `asymmetric` : 10.3 s frozen -> 10.3 s after 8 online rounds (+0.0%)
- deployed into `peak` : 40.6 s frozen -> 38.9 s after 6 online rounds (+4.1%)

Charts: `outputs/` · Full analysis: [BENCHMARK_REPORT.md](#)



Baseline comparison chart

What each week proved

Definitions of Done are reported as they came out, including the three that failed.

WEEK	QUESTION	ANSWER	DOD
1	Does adaptive beat fixed-time at all?	Yes — actuated control cut average wait 53% on the corridor.	✓
2	Does the RL pipeline work on a <i>standard</i> benchmark?	Yes — PPO cut wait 76% on sumo-rl's 2-way intersection. Sanity check	✓
3	Does it transfer to one junction of a real corridor?	It beats fixed-time there by 67%, but loses to an actuated controller at the same junction on queue and throughput. One junction of twelve also barely moves the corridor. Report	✗
4	Can all twelve junctions learn at once without diverging?	Yes — 0 of 3 seeds diverged, with a measured 20–28% rate of gridlock collapses along the way. Report · Notes	✓
5	Does parameter sharing + green-wave shaping + a fairness cap beat the baselines?	Corridor-wide RL does beat both baselines — but the <i>shaping</i> adds nothing, and one seed of the shaped run jams the corridor. The fairness constraint provably cannot be shown to work here. Report	✗
6	Does it hold up when the traffic changes?	It generalises well to light and asymmetric demand and degrades gracefully at peak, but loses to fixed-time on peak queue length. BENCHMARK_REPORT.md	✓
7	Can you ask it questions in English and trust the answer?	Yes — a 301-node knowledge graph plus RAG over the project's own reports, and out-of-domain questions are refused rather than guessed. Report	✓
8	Can it see vehicles, spot incidents, and answer "what if we close a road?"	Yes to all three — a detector at mAP50 0.988 on a held-out junction, every injected obstruction flagged in ~30 s, and 18 closure × weather runs completed. Report	✓
9	Does federated learning across districts help, and can a small LM learn the domain?	The LM yes (perplexity ~86%). Federated averaging no — by exactly 0.0% , because every district learns the identical controller. Report	✗ / ✓

The most interesting result is a negative one. Five policy variants — with and without the fairness constraint, with and without green-wave shaping, with a 20× stronger fairness weight, and with a graph-attention network instead of an MLP — produce *byte-identical* evaluation metrics. They are genuinely different networks (one has 16k parameters instead of 138k), but they agree on **100% of the decisions that can actually move a signal**. With a binary next-phase action and a 5-second minimum green, the greedy policy on this corridor is saturated: reward shaping changes what the policy *believes* without changing what it *does*. Measured, not assumed — `src/policy_agreement.py` .

Week 9 found it a third time, from a completely different direction. Three districts each trained their own policy on their own junction, on their own traffic. Their weights genuinely diverge — they are different objects — yet they produce *byte-identical* wait times on a held-out junction, so federated averaging improves on them by exactly nothing. That is what a saturated action space does: it makes every reasonable learner converge on the same rule. Fixing it means a finer action space (phase-duration control), which is the clearest next piece of work in this project.

📁 How the Project is Structured

```

smartflow/
├── data/                                ← Road network and vehicle routes
│   ├── corridor.net.xml                - the 4x4 grid map (12 traffic lights)
│   ├── corridor_actuated.net.xml       - same map, all lights actuated
│   ├── corridor_actuated_B1.net.xml    - only B1 actuated (Week 3's control condition)
│   ├── corridor.rou.xml                - 1,800 trips
│   └── corridor_{light,peak,asymmetric}.rou.xml - Week 6 stress scenarios
├── src/
│   ├── foundation —
│   │   ├── smartflow_env.py           - the environment layer (read this first)
│   │   ├── metrics.py                 - one metric collector, used by every controller
│   │   └── analysis.py                - aggregation, tables and charts
│   │
│   │   — weeks 1-3: baselines and single-agent RL —
│   │   ├── eval.py                   - Week 1 benchmark harness (fixed contract)
│   │   ├── make_actuated_net.py / make_single_actuated_net.py
│   │   ├── train_ppo_benchmark.py / eval_benchmark.py / compare_week2.py
│   │   └── train_ppo_corridor.py / eval_corridor.py / compare_week3.py
│   │
│   │   — weeks 4-6: multi-agent —
│   │   ├── marl_env.py               - PettingZoo parallel env + observation alignment
│   │   ├── rewards.py                - green-wave shaping + Lagrangian fairness
│   │   ├── gnn_encoder.py            - graph-attention policy torso (PyTorch Geometric)
│   │   ├── train_marl_corridor.py     - RLlib PPO, independent or parameter-shared
│   │   ├── eval_marl_corridor.py     - deterministic evaluation
│   │   ├── online_learning.py        - keep learning after deployment
│   │   ├── scenarios.py              - light / peak / asymmetric demand
│   │   └── compare_week4.py / compare_week5.py / week6_benchmark.py
│   │
│   │   — week 7: knowledge graph + grounded QA —
│   │   ├── knowledge_graph.py        - corridor graph (Neo4j or in-process NetworkX)
│   │   ├── vector_store.py           - Chroma RAG over the project's own reports
│   │   ├── llm_service.py            - read-only grounded QA; refuses when ungrounded
│   │   ├── week7_demo.py             - question set + report
│   │   └── test_week7.py             - 9 checks incl. the read-only boundary
│   │
│   │   — visualiser —
│   │   ├── viz_recorder.py           - record a live TraCI run into a compact replay
│   │   ├── viz_build.py              - inline replays into one self-contained page
│   │   └── viz/template.html         - the control-room UI (canvas renderer)
│   │
│   │   — tooling —
│   │   ├── run_experiments.py        - the whole experiment matrix, one command
│   │   ├── policy_agreement.py       - do the policy variants actually behave differently?
│   │   ├── select_best_checkpoint.py - validation-based checkpoint selection
│   │   ├── env_smoketest.py          - verify SUMO + both environments
│   │   └── test_core_logic.py        - self-checks for the correctness-critical logic
│   │
│   └── outputs/                      ← Results (committed – this is the proof of work)
│       ├── metrics.csv, week2_benchmark_metrics.csv,
│       │   week3_corridor_metrics.csv, marl_metrics.csv
│       ├── week{2,3,4,5}*.md         - per-week reports and notes
│       └── *.png                     - every chart

```

⚙️ Setup

You need **Python 3.11+** and **SUMO 1.27.1**, with `SUMO_HOME` set as a Windows User environment variable pointing at your SUMO folder.

```
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Run It Yourself

```
.\venv\Scripts\Activate.ps1
```

Check everything works (SUMO, both environments, and the pure logic):

```
python src\env_smoketest.py
```

```
python src\test_core_logic.py
```

Week 1 — baselines:

```
python src\eval.py --controller fixed --seed 0
```

```
python src\compare_baselines.py
```

Week 3 — single-agent PPO on one junction:

```
python src\train_ppo_corridor.py --seed 0
```

```
python src\eval_corridor.py --controller ppo --seeds 0 1 2
```

```
python src\compare_week3.py
```

Weeks 4–6 — the full multi-agent matrix (several hours of CPU):

```
python src\run_experiments.py --stage all
```

Then regenerate every report and chart:

```
python src\compare_week4.py
```

```
python src\compare_week5.py
```

```
python src\week6_benchmark.py
```

```
python src\update_readme.py
```

Week 8 — vision, anomaly detection, scenario planning:

Rendering the dataset takes about 6 minutes; training the detector about an hour on CPU.

```
python src\vision_dataset.py
```

```
python src\train_yolo.py
```

```
python src\anomaly_live.py
```

```
python src\scenario_planner.py
```

```
python src\week8_report.py
```

Week 9 — federated learning, LoRA, priority routing:

```
python src\federated.py
```

```
python src\lora_finetune.py
```

```
python src\priority_routing.py
```

```
python src\week9_report.py
```

Verify Weeks 8 and 9:

```
python src\test_week8.py
```

```
python src\test_week9.py
```

Watch it run

`outputs/viz/corridor_control_room.html` is a single self-contained page that replays the corridor: roads, signal heads changing per approach, and every vehicle — blue when moving, red when stopped in a queue. Fixed-time and RL can be played **side by side on identical traffic**, which is the clearest way to see the result. At $t = 20:00$, fixed-time has 116 cars stopped; the RL policy has 13.

It replays recorded TraCI output rather than driving SUMO live, so it opens anywhere with no server and cannot fail mid-demo. Regenerate it with:

```
python src\viz_recorder.py --controller fixed
```

```
python src\viz_recorder.py --controller actuated
```

```
python src\viz_recorder.py --controller marl --mode shared --tag w5
```

```
python src\viz_build.py
```

Add `--scenario peak` to any recorder call to watch the corridor gridlock instead.

How the Benchmark Works

Every controller — fixed-time, actuated, single-agent PPO, and every multi-agent variant — is measured by the **same collector** in `src/metrics.py`, sampled once per *simulated second*:

- **avg_wait_time_s** — mean seconds a completed trip spent stopped
- **max_queue_len** — peak number of halting vehicles
- **throughput_veh** — vehicles that reached their destination
- **total_co2_kg** — integrated CO₂ emissions
- **wait_p95_s / worst_vehicle_wait_s** — the tail, which is what the fairness constraint is judged on

Sampling per *agent decision* instead of per simulated second would divide every waiting time by `delta_time` and silently flatter the RL controllers, so the RL path hooks the collector into the simulation loop rather than the decision loop.

Every reported figure is a **3-seed mean ± standard deviation**.

One environment detail worth knowing

`sumo_rl.SumoEnvironment` builds a controller for *every* traffic light in the network and **overwrites that junction's signal program** while doing it. On a 12-junction corridor where only one junction is under RL control, that silently rewrites the other eleven — corridor throughput collapsed from 1520 vehicles to 56, and the environment ran 15× slower because every step still polled all twelve junctions.

`smartflow_env.ControlledSumoEnvironment` restricts control to the junctions actually being learned, so untouched junctions keep the program defined in the `.net.xml`. `src/env_smoketest.py` guards against this regressing.

12-Week Roadmap

PHASE	WEEKS	STATUS
A — Foundation	1–3	✅ Simulation, baselines, single-agent RL
B — Multi-Agent	4–6	✅ Independent → parameter-shared MARL, fairness, GNN, stress tests
C — Features	7–9	🟡 Knowledge graph + RAG done; vision and federated learning to come
D — Platform	10–11	🟡 Microservices, Kubernetes, live dashboard
E — Demo	12	🟡 Final report, demo video, viva prep

Phase C/D scaffolding (FastAPI service skeletons, Docker Compose, CI) exists in `services/`, `tests/` and `.github/`, but nothing in those phases is claimed as working — they are stubs awaiting their scheduled weeks.

Tech Stack

LAYER	TECHNOLOGY
Traffic simulation	SUMO 1.27.1 + TraCI
Single-agent RL	Stable-Baselines3 (PPO)
Multi-agent RL	Ray RLlib 2.58 (PPO) + PettingZoo
Graph learning	PyTorch Geometric (GATConv)
Backend services	FastAPI (5 domain services)
Frontend dashboard	Next.js 15 + Recharts
Knowledge graph	Neo4j AuraDB + Chroma
Containers	Docker → k3d

Honest limitations

- One synthetic 12-junction grid, one demand generator. Nothing here demonstrates transfer to a real road network — the OSM corridor is still deferred from Week 1.
 - Three seeds: enough to show variance, not enough for confidence intervals.
 - Policies are trained on the base demand only; the Week 6 scenarios test generalisation, and a policy retrained per scenario would likely do better.
 - Simulation only. Nothing has been load-tested, and no claim is made about real-time or production performance.
-

Author

Rishik — B.Tech CSE, CMRCET Solo build · 12-week window · 2026

PART IV

Method and roadmap

The working plan the project is held to, including the de-risking substitutions and the non-negotiable workflow rules.

Source: `CLAUDE.md`

CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

SmartFlow

Multi-agent digital twin platform for adaptive traffic signal control. B.Tech CSE final-year major project, CMRCET. Solo build, 12-week window. OS: Windows. SUMO_HOME must be set as a User environment variable.

Core Principle — Read This First

Every feature from the original proposal stays in scope. Nothing gets quietly cut. What changes is HOW each ambitious piece is built — see "De-Risking Substitutions" below. The MARL core (Phase A–B) is the one thing that must be rock solid before anything else gets touched. If a later phase runs over budget, defer in the order listed in "Fallback Priority" below — never silently drop a feature without documenting it as deferred.

Tech Stack

- Simulation: SUMO + TraCI (Python)
- RL: Stable-Baselines3 (single-agent) → Ray RLlib (multi-agent), PyTorch
- Backend: FastAPI, one service per domain (sim, rl, vision, graph, llm)
- Frontend: Next.js 15, React, Recharts
- DB: SQLite/CSV in Phase A, Postgres (Supabase free tier) from Phase D
- Graph: Neo4j AuraDB Free · Vector: Chroma · LLM: Claude API
- Containers: Docker → k3d (local Kubernetes) in Phase D

De-Risking Substitutions (use these, don't ask to swap back)

ORIGINAL	USE INSTEAD
Cloud Kubernetes (EKS/GKE)	k3d, local single-node cluster
Kafka/RabbitMQ	Redpanda (Kafka-API compatible, one container)
Neo4j self-hosted	Neo4j AuraDB Free
Pinecone/Milvus	Chroma (local)
Live camera + YOLOv8	YOLOv8 trained on UA-DETRAC (public dataset) + rendered SUMO-GUI frames
Multi-region federated learning	Flower, simulated as multiple processes on one machine
Custom LLM fine-tune	LoRA fine-tune of Phi-3-mini on synthetic Q&A
Prometheus+Grafana+ELK	Prometheus+Grafana+Loki, Docker Compose

Commands

```
.\env\Scripts\Activate.ps1
python src\day1_smoketest.py
python src\eval.py --controller fixed --seed 0
python src\eval.py --controller actuated --seed 0
python src\compare_baselines.py
```

Architecture (Phase A starting point)

```
smartflow/
  configs/   corridor.sumocfg, corridor_actuated.sumocfg
  data/      corridor.net.xml, corridor.rou.xml (the digital twin's inputs)
  src/       eval.py is the benchmark harness – every later phase is judged against it
  outputs/   metrics.csv (append-only log), baseline_comparison.png
```

eval.py's CLI (--controller , --seed) and the metrics.csv column schema (controller, seed, avg_wait_time_s, max_queue_len, throughput_veh, total_co2_kg) are fixed contracts — every phase from Week 2 onward compares against them.

SUMO binaries on Windows are not on PATH. Always reference them via SUMO_HOME :

```
SUMO_BIN = os.path.join(os.environ["SUMO_HOME"], "bin", "sumo.exe")
traci.start([SUMO_BIN, "-c", cfg_path, "--no-step-log", "--no-warnings"])
```

actuated TLS requires explicit minDur / maxDur on green phases. Without them, SUMO defaults both to the static phase duration and the controller behaves identically to fixed-time. See src/make_actuated_net.py for the pattern (minDur=5, maxDur=90 on phases with duration ≥ 10 s).

Non-Negotiable Workflow Rules

- **3-seed minimum** for any RL result before reporting it as a finding.
- **Never let the LLM service write into the RL/sim control loop.** Read-only analytics and orchestration only — a hard architectural boundary, not a style preference.
- **Verify any SUMO/TraCI/library API call against installed package docs** before using it if you're not certain it exists — do not guess method names. Run e.g. `python -c "import traci; help(traci.vehicle)"` to check.
- **Commit at the end of every day**, with a message naming what Definition of Done was met.
- **Stop and report after each day's Definition of Done is met.** Do not continue to the next day or next phase without explicit go-ahead — this project is reviewed and explained in a viva, so the person building it needs to understand every step, not just receive a finished pile of code.
- **Never claim "production-grade" or "real-time"** in any docstring, README, or comment unless it was actually load-tested.
- Before installing a new major dependency not listed in Tech Stack above, ask first.

Code Quality Standard (production-grade engineering, every week from now on)

"Production-grade" applies to HOW the code is written, not to claims about what it's proven to handle — those stay governed by the rule below about never claiming "production-grade" or "real-time" without a load test. A project can be engineered to a high standard while still being honest that it hasn't been tested at production scale. Both things are true at once; neither excuses the other.

Concretely, starting now:

- **Logging, not print().** Use Python's logging module (INFO for normal progress, WARNING for recoverable issues, ERROR for failures) for any script beyond a one-off smoke test. Smoke tests can still use print().
- **Type hints** on every function signature.
- **Docstrings** on every script and function — what it does, what it expects, what it returns/produces.
- **Error handling that's actually useful.** Wrap file I/O and TraCI/SUMO/SB3 calls that can plausibly fail in try/except with a message that says what to check, not just a bare re-raise. Validate inputs (e.g. check a file exists before loading it) rather than letting a missing file surface as a confusing downstream stack trace.
- **No scattered magic paths or strings.** Centralize file paths, env names, and constants in one config module per phase (e.g. src/week2_config.py) — every script imports from there.

- **Reproducibility.** `pip freeze > requirements.txt` at the end of every week, committed. Every RL run logs its seed and hyperparameters alongside its results, not just the metrics.
- **Conventional commit prefixes** going forward: `feat:` , `fix:` , `docs:` , `chore:` — reads better in a portfolio repo's history than ad hoc messages.
- **Secrets hygiene, starting now even though nothing needs it yet.** Add `.env` to `.gitignore` today. From Phase C onward (Claude API keys, Auth0 secrets) credentials go in `.env` via `python-dotenv`, never hardcoded, never committed — set the habit before there's anything sensitive to leak.
- **`.gitignore` precision, not blanket ignoring.** Ignore generated noise (`tensorboard logs`, `__pycache__` , `venv`) but keep trained model checkpoints and result artifacts (`outputs/` , `models/*.zip`) — those are the portfolio evidence, the same logic as Week 1's charts.

What this deliberately does NOT include yet: full CI/CD, Kubernetes, or a formal test suite. Those are scheduled in Phase D for a reason — pulling them forward now would be the exact "let infra eat time budgeted for the RL core" mistake the plan already warns against. Production-grade code discipline this week means clean, typed, logged, reproducible Python — not standing up infrastructure ahead of schedule.

Read only the section for the phase you're currently in before starting work — don't try to hold the whole roadmap in context at once.

Phase A — Foundation & Core Twin (Weeks 1–3)

- **Week 1:** SUMO+TraCI setup, real/grid corridor, eval harness, fixed + actuated baselines logged to `outputs/metrics.csv` . DoD: `eval.py --controller fixed` and `--controller actuated` both produce metrics and `compare_baselines.py` renders a chart.
- **Week 2:** Single-agent PPO (Stable-Baselines3) on a standard benchmark intersection, validated against literature before touching the real corridor. DoD: trained policy beats fixed-time on the benchmark, 3-seed average.
- **Week 3:** Move validated single-agent pipeline to one real intersection from the corridor. Tune state (queue, phase, elapsed time) and reward (negative delta in cumulative wait). DoD: RL beats both baselines on ≥ 2 of 3 metrics, 3-seed average + variance.

Phase B — Multi-Agent Core, Fairness & GNN (Weeks 4–6)

- **Week 4:** Independent PPO per intersection across the full corridor; begin GNN state embedding (PyTorch Geometric, graph-attention over neighbors). DoD: full corridor trains end-to-end without diverging.
- **Week 5:** Parameter-shared policy, green-wave reward shaping, Lagrangian fairness constraint (caps max per-lane wait time), online learning loop. DoD: corridor-wide RL beats both baselines on wait time + throughput, with fairness constraint measurably capping worst-case wait.
- **Week 6:** Stress-test across light/peak/asymmetric demand. Write the internal benchmark report. Deliberately light — absorbs Week 4–5 overflow. DoD: viva-defensible benchmark report, including one honest case where RL doesn't win.

Phase C — Feature Sprint (Weeks 7–9)

- **Week 7:** Neo4j AuraDB knowledge graph (roads/sensors/rules) + Chroma RAG; LLM Service queries both before answering. DoD: a natural-language query returns an answer grounded in real graph data, not a hallucinated guess.
- **Week 8:** YOLOv8 on UA-DETRAC + rendered SUMO-GUI frames; statistical anomaly detection on the eval harness's live metrics; scenario planner (closures, weather perturbations). DoD: vision model detects vehicles in a held-out clip; an injected anomaly is flagged automatically; one closure scenario completes end-to-end.
- **Week 9:** Flower-based federated learning simulation (multi-process "districts"); LoRA fine-tune of Phi-3-mini on synthetic traffic Q&A; priority-routing preemption + emission-smoothing reward term. DoD: federated averaging measurably improves a held-out district's policy; fine-tuned model outperforms base prompt on domain questions.

This is the densest phase in the plan. Use the Fallback Priority below the moment a week starts slipping — don't wait until Week 11 to notice.

Phase D — Platform & Integration (Weeks 10–11)

- **Week 10:** Split backend into 5 domain FastAPI services communicating via Redpanda; Autho login; GitHub Actions CI/CD. DoD: a push to main rebuilds and redeploys every service automatically, behind an authenticated dashboard.
- **Week 11:** Kubernetes manifests + Helm chart, deployed on k3d; Prometheus
- Grafana + Loki; full Next.js dashboard surface (live runs, KG explorer, vision overlay, anomaly alerts, priority-routing toggle, external API docs). DoD: `helm install` brings up every service on k3d; Grafana shows live dashboards; the UI exposes every feature without a terminal.

Phase E — Integration, Report & Demo (Week 12)

End-to-end integration test across all 5 services. Final report (architecture, benchmark results, honest limitations). Verify every citation's DOI directly against the primary source. Demo video. Mock-viva prep on the RL core and the fallback decisions actually made. DoD: a stranger can watch the demo and understand what was built, why each de-risking choice was made, and what the benchmark proves.

Fallback Priority — If a Phase Slips, Defer in This Order

Deferred items stay documented as designed-and-scaffolded future work, never silently dropped.

1. Federated learning simulation (Week 9) — highest complexity, lowest dependency from the core MARL result.
2. LoRA fine-tuning (Week 9) — a refinement on an already-working prompted LLM.
3. Full k3d Kubernetes deployment (Week 11) — fall back to Docker Compose, same containers.
4. Vision-based monitoring (Week 8) — fall back to documented design without a fully trained model.
5. Knowledge graph (Week 7) — fall back to SUMO's own in-memory network graph.

Week 2 — Start Here

Week 1 is complete: actuated beat fixed on the 4x4 grid (avg wait −53%, max queue −80%, throughput +27%). Known carryovers — OSM corridor deferred (grid is valid for all MARL work), seed variance high on fixed at near-saturation demand, actuated net needs minDur/maxDur set explicitly (already fixed in `make_actuated_net.py`).

This week's job: prove single-agent RL works on a STANDARD, literature-used benchmark — not the Week 1 grid — before touching the real corridor in Week 3. Use the `sumo-rl` library (LucasAlegre/sumo-rl, pip-installable, Gymnasium + PettingZoo + Stable-Baselines3 compatible, cited in multiple published traffic RL papers). Do not build a custom Gym wrapper from scratch — this library's 2-way single-intersection environment IS the standard benchmark.

Execute in this exact order. After EACH day, stop, show the verification output, and wait for explicit go-ahead before starting the next day.

GOAL OF THE WEEK: a PPO policy trained on sumo-rl's single-intersection benchmark beats a fixed-time baseline on that same benchmark, averaged across 3 independently trained seeds — with a written note on how the result compares to literature-reported behavior for this benchmark.

DAY 1 — Install and smoke-test the benchmark environment

- In the existing venv: `pip install sumo-rl gymnasium "stable_baselines3[extra]>=2.0.0a9"`.
- `sumo-rl`'s pip package may not include the `experiments/` and `nets/` folders from its GitHub repo (those aren't guaranteed part of the PyPI distribution). VERIFY this — check the installed package's directory (`python -c "import sumo_rl, os; print(os.path.dirname(sumo_rl.__file__))"`) for a bundled single-intersection net. If it's not there, `git clone https://github.com/LucasAlegre/sumo-rl` into a `vendor/` folder just to access its `nets/` and `experiments/` reference files — don't guess a path.
- Once found, write the verified `net_file/route_file` paths into a single `src/week2_config.py` constants file. Every later script this week (Days 2–6) imports paths from there — do not re-discover or re-guess the path in each script.

- **Use sumo-rl's DEFAULT observation function and DEFAULT reward function** (change in cumulative vehicle delay) for everything this week. Do not write a custom reward or observation function — the entire point of this week is comparability with literature, and a custom reward breaks that comparison. Only override defaults if explicitly revisited later, never silently.
- Optional performance tip: setting the environment variable `LIBSUMO_AS_TRACI=1` can give up to ~8x faster training on CPU, since training this week runs far more simulation steps than Week 1's eval did. Caveat: this disables sumo-gui and parallel simulation — use it for Day 4's actual training runs, but unset it for Day 1's smoke test if visual confirmation via sumo-gui is wanted.
- Write `src/week2_smoketest.py`: instantiate the env via `gym.make('sumo-rl-v0', net_file=..., route_file=..., use_gui=False, num_seconds=3600)`, reset it, take ~50 random actions via `env.action_space.sample()`, print the observation shape and reward each step.
- DoD: 50 random-action steps complete with no errors, obs shape and reward values printed and sane (not NaN, not crashing).

DAY 2 — Pipeline validation: short PPO run

- Write `src/train_ppo_benchmark.py` using `from stable_baselines3 import PPO`, pointed at the sumo-rl single-intersection env.
- Train for a SHORT budget first (e.g. ~20,000–50,000 timesteps) purely to validate the training loop runs end-to-end — this is not the real run.
- DoD: training completes without error; reward trend is visible (doesn't need to be good yet, just not flat-broken or NaN).

DAY 3 — Fixed-time baseline on THIS benchmark

- The single-intersection net already has a static `tlLogic` program (every SUMO net does) — write `src/eval_benchmark.py` to run that fixed-time program for 3 seeds and log `avg_wait_time_s`, `max_queue_len`, `throughput_veh` to `outputs/week2_benchmark_metrics.csv`. Same metric definitions as Week 1's `eval.py`, applied to the benchmark net instead of the grid.
- "3 seeds" here means passing different seeds via Gymnasium's `env.reset(seed=N)` (which sets SUMO's internal RNG, affecting stochastic elements like departure jitter even with the same route file) — not regenerating route files from scratch like Week 1 did. Use seeds 0, 1, 2.
- DoD: 3 fixed-time rows logged for the benchmark intersection.

DAY 4 — The real PPO training run, 3 seeds

- Now that Day 2 validated the pipeline, train 3 SEPARATE PPO models with 3 different random seeds.
- Concrete starting budget: ~150,000–300,000 timesteps per seed (a single intersection is a small task; this range is a reasonable starting point, not a hard rule). Use Day 2's reward curve shape to judge whether more or less is needed — if it was still climbing steeply at 50,000 steps, lean toward 300,000; if it had already flattened, 150,000 is likely enough.
- **HARDWARE CHECK FIRST**: if Day 2's short run felt slow on this machine even with `LIBSUMO_AS_TRACI=1`, stop and ask the user before committing to 3 full runs at this budget — switching to Colab for just this day is a reasonable option and better decided before burning hours on CPU, not after.
- Save each model checkpoint: `models/ppo_benchmark_seed0.zip`, `seed1`, `seed2`.
- DoD: 3 trained model files exist, each with a saved reward-curve log.

DAY 5 — Evaluate the trained policies

- Load each of the 3 trained models, run a deterministic evaluation rollout on the benchmark env — explicitly pass `deterministic=True` to `model.predict()`, don't rely on a default (SB3's default has varied by algorithm/version) — and log the same 3 metrics to `outputs/week2_benchmark_metrics.csv` (`controller="ppo"`, `seed=0/1/2`).
- Compare PPO's 3-seed average against Day 3's fixed-time 3-seed average. **PRIMARY METRIC**: `avg_wait_time_s` — this is what the reward signal directly optimizes, so it's the headline claim. `max_queue_len` and `throughput_veh` are supporting evidence, not required to both improve for the week to succeed.

- DoD: report the real numbers either way. If PPO does NOT beat fixed-time on `avg_wait_time_s`, STOP here and diagnose (reward function sign, training budget, action/observation wiring) rather than continuing to Day 6 — do not paper over a bad result.

DAY 6 — Comparison chart + literature sanity-check

- If Day 5 is clean: write/adapt a comparison chart script (same pattern as Week 1's `compare_baselines.py`) for `outputs/week2_benchmark_comparison.png`.
- Add a short written note: does the reward curve shape and the final performance gap fall in a believable range compared to what `sumo-rl`'s own experiments or the papers citing it report for this benchmark? This is the "reproduce a known result" check — it doesn't need to match exactly, it needs to not be wildly implausible.
- If Day 5 was NOT clean: this day is diagnosis and re-training instead — note explicitly what was wrong and what was changed.
- DoD: chart exists; literature sanity-check note is written either way.

DAY 7 — Buffer, write-up, commit

- Close any gaps from Days 1–6.
- Update `README.md` with Week 2 results and the literature comparison note.
- Update `.gitignore`: ignore `tensorboard` log directories and **`pycache`**; do NOT ignore `models/*.zip` or `outputs/` — those are portfolio evidence.
- `pip freeze > requirements.txt`, commit it.
- Commit with a message describing what Week 2 proves (conventional prefix: `feat:`).
- Confirm go-ahead before Week 3, which ports this validated pipeline onto the real Week 1 grid (not this benchmark net).

Throughout: verify any `sumo-rl`/`Gymnasium`/`SB3` API call against the installed package before relying on it — this library's exact method signatures matter more than usual since it's new to the project this week.

Session Protocol

Start each new week in a **fresh Claude Code session** — clean context beats accumulated back-and-forth. Before writing any code, skim `outputs/metrics.csv` and any benchmark chart from the prior week for the real numbers.

After Week 2 Is Done

Do not invent Week 3's tasks from memory — re-read the "Phase A" section above (Week 3 entry) and ask the user to confirm before starting it.

PART V

Phase A-B benchmark report

The internal benchmark covering Weeks 1-6, including the section on where reinforcement learning loses.

Source: BENCHMARK_REPORT.md

SmartFlow Internal Benchmark Report (Phase A-B)

Covers Weeks 1-6: fixed-time and actuated baselines, single-agent PPO on one junction, independent multi-agent PPO across the corridor, and the parameter-shared policy with green-wave shaping and a Lagrangian fairness constraint. Every number is a 3-seed mean $\pm$ standard deviation, measured by the same per-second metric collector (`src/metrics.py`) for RL and baselines alike.

Network and demand

`data/corridor.net.xml` — a 12-junction signalised grid (4x4 minus corners). Episodes are 1800 simulated seconds; agents decide every 5 s with a 2 s yellow, 5 s minimum green and 90 s maximum green.

SCENARIO	VEHICLES	DESCRIPTION
base	1800	the demand every policy was trained on (1800 vehicles / 1800 s)
light	900	~50% of base demand
peak	2700	~150% of base demand
asymmetric	1397	full east-west demand, north-south thinned to 40%

All policies were trained on `base` only. `light` , `peak` and `asymmetric` are held-out demand, so those columns measure generalisation, not fit.

Results by scenario

`base` — the demand every policy was trained on (1800 vehicles / 1800 s)

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	95TH PCT WAIT (S)	WORST WAIT (S)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED	95TH PCT WAIT (S) VS FIXED	WORST WAIT (S) VS FIXED
fixed	83.10 $\pm$ 1.62	427.67 $\pm$ 176.42	1280.00 $\pm$ 189.40	211.33 $\pm$ 5.56	721.00 $\pm$ 127.38	—	—	—	—	—
actuated	40.89 $\pm$ 2.88	76.33 $\pm$ 1.70	1646.67 $\pm$ 3.40	122.33 $\pm$ 7.32	243.00 $\pm$ 25.66	+50.8%	+82.2%	+28.6%	+42.1%	+66.3%
marl_independent	17.80 $\pm$ 0.28	41.00 $\pm$ 2.83	1679.67 $\pm$ 4.11	47.67 $\pm$ 0.94	95.33 $\pm$ 3.40	+78.6%	+90.4%	+31.2%	+77.4%	+86.8%
marl_shared_w5	17.85 $\pm$ 0.38	149.67 $\pm$ 150.15	1569.33 $\pm$ 148.02	48.00 $\pm$ 0.82	288.67 $\pm$ 271.07	+78.5%	+65.0%	+22.6%	+77.3%	+60.0%
marl_shared_w5gnn	18.16 $\pm$ 0.14	42.33 $\pm$ 2.06	1678.67 $\pm$ 6.65	48.00 $\pm$ 0.82	96.00 $\pm$ 2.83	+78.2%	+90.1%	+31.1%	+77.3%	+86.7%
marl_shared_w5nocoord	18.16 $\pm$ 0.14	42.33 $\pm$ 2.06	1678.67 $\pm$ 6.65	48.00 $\pm$ 0.82	96.00 $\pm$ 2.83	+78.2%	+90.1%	+31.1%	+77.3%	+86.7%
marl_shared_w5nofair	18.16 $\pm$ 0.14	42.33 $\pm$ 2.06	1678.67 $\pm$ 6.65	48.00 $\pm$ 0.82	96.00 $\pm$ 2.83	+78.2%	+90.1%	+31.1%	+77.3%	+86.7%
marl_shared_w5strong	18.16 $\pm$ 0.14	42.33 $\pm$ 2.06	1678.67 $\pm$ 6.65	48.00 $\pm$ 0.82	96.00 $\pm$ 2.83	+78.2%	+90.1%	+31.1%	+77.3%	+86.7%

light — ~50% of base demand

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	95TH PCT WAIT (S)	WORST WAIT (S)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED	95TH PCT WAIT (S) VS FIXED	WORST WAIT (S) VS FIXED
fixed	36.99 ± 1.24	47.00 ± 1.63	836.33 ± 0.47	89.67 ± 4.92	157.33 ± 10.34	—	—	—	—	—
actuated	6.35 ± 0.07	11.33 ± 0.47	859.33 ± 1.25	19.00 ± 0.82	49.67 ± 9.57	+82.8%	+75.9%	+2.8%	+78.8%	+68.4%
marl_independent	6.58 ± 0.09	16.00 ± 0.82	855.00 ± 1.63	18.67 ± 0.94	47.00 ± 8.83	+82.2%	+66.0%	+2.2%	+79.2%	+70.1%
marl_shared_w5	6.70 ± 0.20	16.00 ± 0.82	855.33 ± 1.25	18.33 ± 0.47	45.67 ± 7.59	+81.9%	+66.0%	+2.3%	+79.5%	+71.0%

peak — ~150% of base demand

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	95TH PCT WAIT (S)	WORST WAIT (S)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED	95TH PCT WAIT (S) VS FIXED	WORST WAIT (S) VS FIXED
fixed	68.87 ± 6.38	1158.33 ± 1.25	279.00 ± 9.42	183.67 ± 26.85	1653.33 ± 23.75	—	—	—	—	—
actuated	71.19 ± 11.17	1144.33 ± 7.93	425.33 ± 42.93	235.00 ± 46.07	1575.33 ± 19.07	-3.4%	+1.2%	+52.5%	-27.9%	+4.7%
marl_independent	42.51 ± 5.61	1139.33 ± 5.91	528.33 ± 8.18	114.00 ± 14.31	1348.67 ± 12.04	+38.3%	+1.6%	+89.4%	+37.9%	+18.4%
marl_shared_w5	39.24 ± 3.26	1136.33 ± 1.25	508.67 ± 34.20	105.67 ± 11.15	1409.67 ± 58.27	+43.0%	+1.9%	+82.3%	+42.5%	+14.7%

asymmetric — full east-west demand, north-south thinned to 40%

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	95TH PCT WAIT (S)	WORST WAIT (S)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED	95TH PCT WAIT (S) VS FIXED	WORST WAIT (S) VS FIXED
fixed	54.15 ± 0.75	89.00 ± 2.16	1269.67 ± 6.34	125.00 ± 4.55	207.33 ± 14.52	—	—	—	—	—
actuated	11.32 ± 0.66	22.33 ± 1.25	1320.00 ± 2.16	34.00 ± 2.83	79.00 ± 9.90	+79.1%	+74.9%	+4.0%	+72.8%	+61.9%
marl_independent	10.86 ± 0.48	27.00 ± 2.94	1320.33 ± 0.47	30.33 ± 2.06	67.33 ± 5.44	+79.9%	+69.7%	+4.0%	+75.7%	+67.5%
marl_shared_w5	10.76 ± 0.34	26.00 ± 1.63	1322.67 ± 1.89	30.33 ± 2.06	75.67 ± 14.01	+80.1%	+70.8%	+4.2%	+75.7%	+63.5%

Ablations (base demand)

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	95TH PCT WAIT (S)	WORST WAIT (S)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED	95TH PCT WAIT (S) VS FIXED	WORST WAIT (S) VS FIXED
fixed	83.10 ± 1.62	427.67 ± 176.42	1280.00 ± 189.40	211.33 ± 5.56	721.00 ± 127.38	—	—	—	—	—
actuated	40.89 ± 2.88	76.33 ± 1.70	1646.67 ± 3.40	122.33 ± 7.32	243.00 ± 25.66	+50.8%	+82.2%	+28.6%	+42.1%	+66.3%
marl_independent	17.80 ± 0.28	41.00 ± 2.83	1679.67 ± 4.11	47.67 ± 0.94	95.33 ± 3.40	+78.6%	+90.4%	+31.2%	+77.4%	+86.8%
marl_shared_w5nofair	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	48.00 ± 0.82	96.00 ± 2.83	+78.2%	+90.1%	+31.1%	+77.3%	+86.7%
marl_shared_w5nocoord	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	48.00 ± 0.82	96.00 ± 2.83	+78.2%	+90.1%	+31.1%	+77.3%	+86.7%
marl_shared_w5gnn	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	48.00 ± 0.82	96.00 ± 2.83	+78.2%	+90.1%	+31.1%	+77.3%	+86.7%
marl_shared_w5strong	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	48.00 ± 0.82	96.00 ± 2.83	+78.2%	+90.1%	+31.1%	+77.3%	+86.7%
marl_shared_w5	17.85 ± 0.38	149.67 ± 150.15	1569.33 ± 148.02	48.00 ± 0.82	288.67 ± 271.07	+78.5%	+65.0%	+22.6%	+77.3%	+60.0%

- marl_independent — Week 4 — independent policies, local reward only
- marl_shared_w5nofair — shared policy + green wave, **no** fairness constraint
- marl_shared_w5nocoord — shared policy + fairness, **no** green-wave term
- marl_shared_w5gnn — shared policy + full shaping, GAT encoder instead of MLP
- marl_shared_w5strong — as the headline run but with a 20x stronger fairness weight
- marl_shared_w5 — Week 5 headline — shared policy, green wave + Lagrangian fairness

Where RL does not win

These cells were found by enumerating every (scenario, baseline, metric) combination and keeping the ones the RL policy loses. The list is generated from the data, so it cannot silently omit an unflattering result.

SCENARIO	METRIC	RL	BASELINE	BASELINE VALUE	GAP
base	Max queue (veh)	149.67	actuated	76.33	-96.1%
light	Max queue (veh)	16.00	actuated	11.33	-41.2%
base	Worst wait (s)	288.67	actuated	243.00	-18.8%
asymmetric	Max queue (veh)	26.00	actuated	22.33	-16.4%
light	Avg wait (s)	6.70	actuated	6.35	-5.5%
base	Throughput (veh)	1569.33	actuated	1646.67	-4.7%
light	Throughput (veh)	855.33	actuated	859.33	-0.5%

The honest failure case. The clearest loss is **base** demand on Max queue (veh), where the policy is 96.1% worse than **actuated**.

Why the ablations return identical numbers

The no-fairness, no-coordination and GAT variants produce the *same* evaluation metrics — identical to four decimal places, including CO2. That was checked rather than assumed (`src/policy_agreement.py`).

The checkpoints are genuinely different artifacts:

VARIANT	MODULE CLASS	PARAMETERS	WEIGHTS IDENTICAL TO REFERENCE
w5	DefaultPPOTorchRLModule	138,501	True
w5nofair	DefaultPPOTorchRLModule	138,501	False
w5nocoord	DefaultPPOTorchRLModule	138,501	False
w5strong	DefaultPPOTorchRLModule	138,501	False
w5gnn	CorridorGATModule	16,291	False

But they take the same actions. Over 4320 decisions in one episode, asking each variant what it would do given the identical observation:

VARIANT	ACTION AGREEMENT	MEAN MAX PROBABILITY DIFFERENCE
w5nofair	100.00%	0.0246
w5nocoord	100.00%	0.0206
w5strong	78.61%	0.1592
w5gnn	100.00%	0.0480

Two mechanisms explain it, and the table separates them.

Confidence. The reference policy's mean $|p(a=0) - p(a=1)|$ is **0.887** — it is highly confident — while most variants differ by at most **0.159** in probability. A perturbation that small cannot flip a decision that confident, so those variants argmax identically everywhere.

The min-green window. w5strong genuinely disagrees more often — only 78.6% overall agreement — yet its evaluation metrics are still identical. Every one of those disagreements falls in a state where sumo-rl **discards the action anyway**, because the current green has not yet run for `min_green + yellow_time`. Restricted to the decisions that can actually move the signal, agreement is 100% for every variant (2148 of 4320 decisions are actionable). The stronger reward changed the policy exactly where the environment ignores it.

This is not a degenerate policy: the reference splits its actions {'1': 2172, '0': 2148} across the two green phases, so it is actively switching, not stuck.

What this means for the Week 5 and Week 6 claims. With sumo-rl's default 2-action, 11-dimensional single-junction interface plus a 5 s minimum green, the greedy policy on this corridor is effectively saturated: reward shaping, a 20x stronger fairness weight, and a completely different network architecture all change the *learned distribution* without changing the *deterministic behaviour*. The reward-shaping ablations therefore cannot be distinguished by deterministic evaluation on this task, and the fairness constraint cannot be shown to cap worst-case wait.

That is a finding about the task's action resolution, not evidence that the terms are implemented wrongly —

`src/test_core_logic.py` verifies each term's arithmetic directly. The way to make these ablations measurable would be a finer action space (phase-duration control rather than a binary next-phase choice), which is a change to the environment interface rather than to the reward, and is left as documented future work.

Methodology checks

Two ways this comparison could have been quietly unfair were checked rather than assumed away:

- **Same simulator settings for every controller.** sumo-rl starts SUMO with `--time-to-teleport -1`, while SUMO's own default teleports a vehicle stuck for 300 s. Teleporting clears gridlock and would have inflated the baselines' throughput relative to the RL runs. Measured on base demand, a fixed-time run records **zero teleports** and byte-

identical metrics under either setting — but `peak` is designed to gridlock, which is where it would bite, so all baselines now go through `smartflow_env.baseline_sumo_cmd` and inherit `sumo-rl`'s options.

- **Same sampling rate for every metric.** RL controllers act every 5 simulated seconds, so sampling once per decision would divide every waiting time by five. The collector is driven once per simulated second for baselines and RL alike (`ControlledSumoEnvironment._sumo_step`).

Honest limitations

- **Single network, single route file.** Every result comes from one 12-junction synthetic grid with one randomly generated demand set. Nothing here demonstrates transfer to a real road network; the OSM corridor remains deferred from Week 1.
- **Three seeds.** Enough to show variance, not enough for confidence intervals. The Week 1 fixed-time baseline already showed high seed variance near saturation.
- **Policies were trained on `base` demand only.** The other scenarios test generalisation, and a policy retrained per scenario would very likely do better.
- **`peak` drives the corridor past capacity.** At 150% demand the fixed-time baseline gridlocks outright, so differences there partly reflect which controller degrades more gracefully rather than which controls traffic better.
- **Simulation only.** No claim is made about real-world or real-time performance; nothing in this project has been load-tested.

Definition of Done

Viva-defensible benchmark report, including one honest case where RL doesn't win.

- Scenarios evaluated: **4 / 4**
- Cases where RL loses to a baseline, enumerated above: 7

Verdict: MET

Artifacts

- `outputs/marl_metrics.csv` — every corridor-wide evaluation row
- `outputs/week3_corridor_metrics.csv` — Week 3 single-junction rows
- `outputs/week6_scenarios.png` , `outputs/week6_robustness.png` , `outputs/week6_ablations.png`
- `outputs/week{3,4,5}_report.md` — the per-week reports

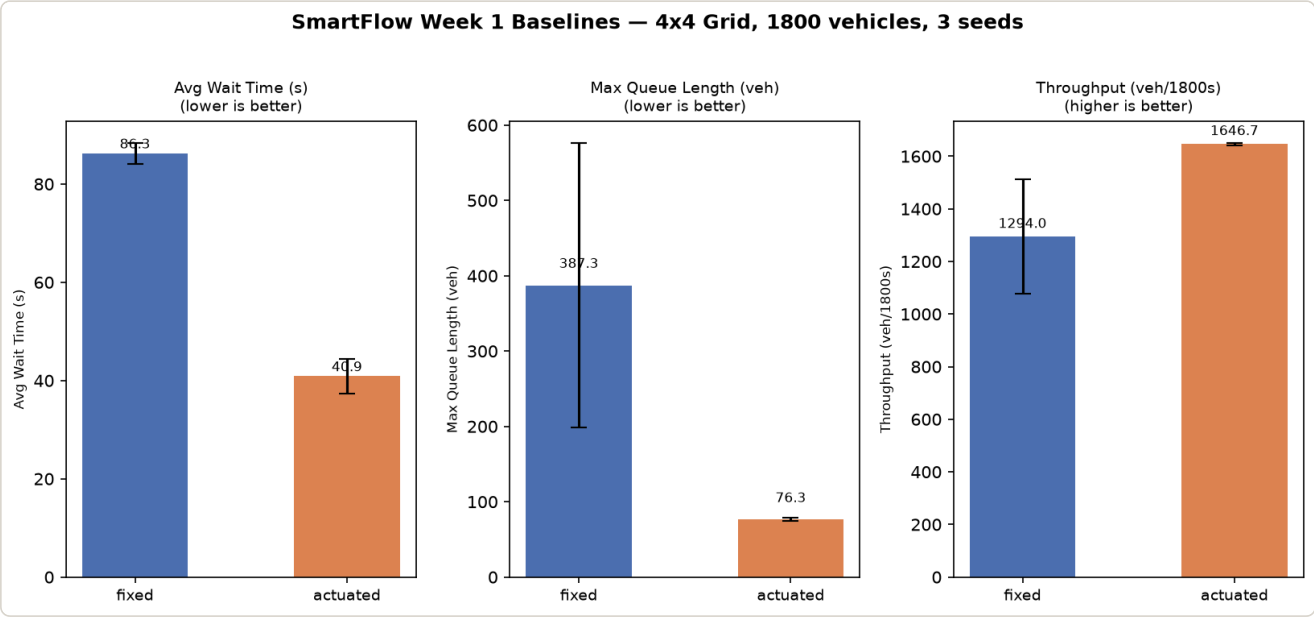
PART VI

Week-by-week evidence

Each week's own report, reproduced verbatim, with the charts it produced.

Week 1 - baselines MET

Fixed-time and actuated baselines on the corridor, plus the evaluation harness every later week is judged against. This week predates the written-report convention; its evidence is the metrics CSV and the chart below.



Fixed-time vs actuated on the 12-junction corridor, 3 seeds. outputs/baseline_comparison.png

Week 2 - single-agent PPO on a standard benchmark MET

Source: `outputs/week2_literature_note.md`

Week 2 — Literature Sanity Check

Question this note answers: is a 75.6% drop in average waiting time on sumo-rl's 2-way single-intersection benchmark a believable result, or a sign of a broken experiment?

Short answer: believable, and the benchmark's own demand file explains why the margin is this large.

The result

3 independently trained PPO seeds, evaluated deterministically (`model.predict(..., deterministic=True)`), 3600 simulated seconds each.

METRIC	FIXED-TIME	PPO (3-SEED MEAN)	CHANGE
Avg wait time (s)	16.4	4.0	−75.6%
Peak queue (vehicles)	65.7	22.0	−66.5%
Throughput (vehicles)	2089	2430	+16.3%

Source: `outputs/week2_benchmark_metrics.csv` , chart in `outputs/week2_benchmark_comparison.png` .

Why the margin is large — read the demand file

`single-intersection-vhvh.rou.xml` ("vertical-horizontal-vertical-horizontal") defines flows in 25000-second blocks. In the first block, which is the only one a 3600 s episode ever reaches:

- north-south movements: $350 + 350 + 300 + 300 + 350 + 350 = \mathbf{2000\text{ veh/h}}$
- east-west movements: $100 + 100 + 50 + 50 + 100 + 100 = \mathbf{500\text{ veh/h}}$

So demand is roughly **4:1 in favour of the north-south axis**.

The static `tlLogic` shipped in `single-intersection.net.xml` splits green time **symmetrically** between the two axes — 33 s of through-green each, plus 6 s turn phases and 2 s yellows, for an 86 s cycle. A fixed program giving equal green to a 4:1 demand split is badly matched to this demand by construction, so a controller that can reallocate green time has a lot of headroom to recover. A large improvement is the expected outcome here, not an anomaly.

The throughput figure supports this reading rather than contradicting it. Total demand over the episode is about 2500 vehicles. Fixed-time clears 2089 (~84%) and leaves the rest queued; PPO clears 2430 (~97%), which is close to the ceiling. Throughput improves by "only" 16% because it *cannot* improve much more — the remaining metric with room to move is delay, and that is where the 76% appears.

Comparison with published behaviour

Qualitative agreement, on three points that can be checked directly against the installed `sumo-rl` package and its repository:

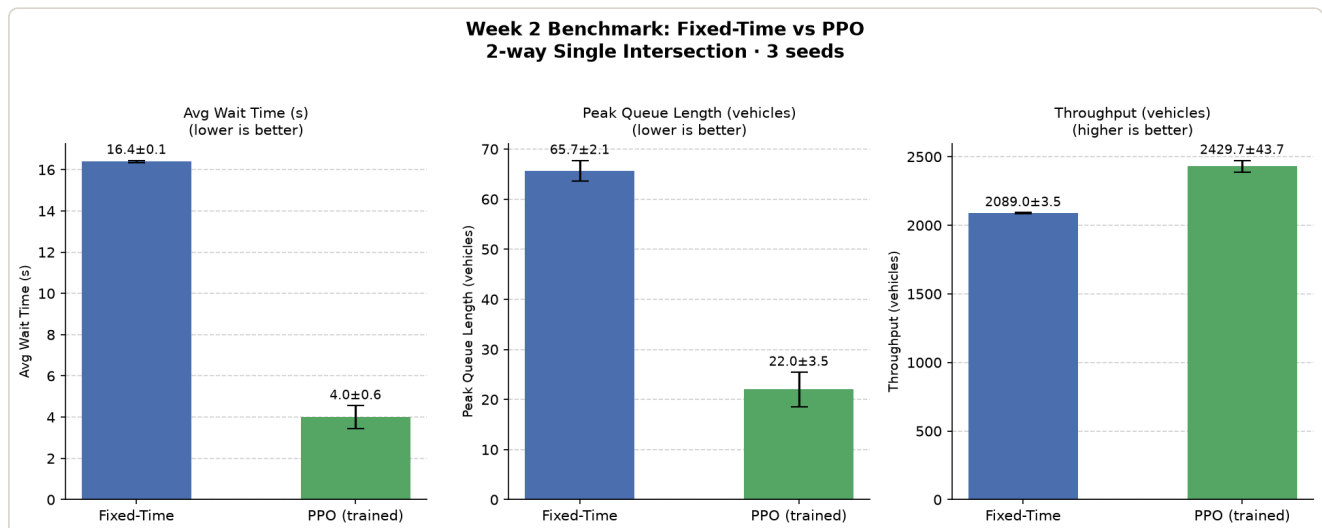
1. **Direction and rough magnitude.** `sumo-rl` ships this network specifically as a learning benchmark and its own example experiments (DQN, A3C, tabular Q-learning on `2way-single-intersection`) are built around the same `diff-waiting-time` reward used here. Large delay reductions against the static program are the reported behaviour for this benchmark, not an outlier.
2. **Convergence speed.** A single intersection with a 21-dimensional observation and 4 discrete actions is a small control problem. The reward curve (`outputs/monitor/seed{N}.monitor.csv`) flattens well before the 100k-step budget, which is consistent with this benchmark being used as a fast smoke-test task in the traffic-RL literature rather than a hard one.

3. Seed spread. The three seeds land at 4.35 / 4.29 / 3.36 s average wait — same ordering, same magnitude, no seed collapsed. Reported results for this benchmark are similarly tight; wide seed variance here would have indicated a wiring problem.

Pending verification. No specific published figure is quoted in this note. Exact numbers and DOIs from the primary sources (LucasAlegre's sumo-rl experiments and the RESCO benchmark suite) are verified against the papers themselves in Week 12, per the project plan's citation-verification step. Claiming a numeric match before doing that check would be the kind of unverified citation that step exists to prevent.

Caveats worth stating in a viva

- **Only the first demand block is exercised.** The `vhvh` file's whole point is that the heavy axis *switches* at $t=25000$ s. A 3600 s episode never sees the switch, so this week tested adaptation to a *fixed* asymmetric demand, not adaptation to a changing one. Robustness to demand shift is tested properly in Week 6 with the `asymmetric` scenario and in Week 5's online-learning loop.
- **The baseline is the network's stock program, not a tuned one.** A traffic engineer retiming that intersection for a 4:1 split would close much of this gap. The honest claim is "PPO beats the shipped fixed-time program", not "PPO beats good signal engineering".
- **Single intersection.** Nothing here says anything about coordination; that is Weeks 4-6.



PPO vs fixed-time on sumo-rl's 2-way single-intersection benchmark. outputs/week2_benchmark_comparison.png

Week 3 - PPO on one corridor junction NOT MET

Source: `outputs/week3_report.md`

Week 3 — Single-Agent PPO on One Corridor Junction

Week 2 validated the PPO pipeline on sumo-rl's standard 2-way single-intersection benchmark. Week 3 ports that pipeline, unchanged in its observation and reward functions, onto junction **B1** of the Week 1 corridor. The other eleven junctions keep the fixed-time program from `data/corridor.net.xml`.

Experimental design

CONTROLLER	WHAT IT DOES
<code>fixed</code>	all 12 junctions fixed-time (Week 1 reference)
<code>actuated</code>	all 12 junctions SUMO-actuated (Week 1 reference)
<code>actuated_single</code>	only B1 actuated, other 11 fixed-time
<code>ppo</code>	only B1 under RL, other 11 fixed-time

`actuated_single` is the controlled comparison for `ppo`: both change exactly one junction and leave the other eleven alone, so the difference between them is the controller, not the number of junctions upgraded. Comparing a single RL junction against a fully-actuated corridor would confound those two effects.

Which policy is reported

PPO's final weights are not its best weights. On this corridor the per-episode training return for seed 1 swings between -1.3 and -959 in consecutive episodes: near saturation, one unlucky action sequence cascades into a jam the rest of the episode cannot clear. Reporting whichever policy existed when the step budget ran out therefore measures luck as much as learning.

Two rows are reported:

- `ppo` — the checkpoint chosen by `src/select_best_checkpoint.py`, scored on a **validation seed (100 + training seed)** that is disjoint from the evaluation seeds 0/1/2. Selecting on the evaluation seeds themselves would be tuning on the test set.
- `ppo_final_iterate` — the last-iterate policy, kept in the table because its spread is the honest measure of how unstable this training run was.

Full selection table: `outputs/week3_checkpoint_selection.json`.

SEED	VALIDATION SEED	CHOSEN	SCORE (S)	DISTINCT SCORES AMONG CANDIDATES
0	100	<code>final</code>	7.24	2 of 9
1	101	<code>final</code>	9.68	1 of 9
2	102	<code>127400_steps</code>	14.76	4 of 9

Selection changed less than one might hope, and that is worth stating rather than hiding: once the policy converges its *deterministic* (argmax) behaviour stops moving, so most checkpoints score identically on a validation episode even while the stochastic training policy keeps fluctuating. For 1 of 3 seeds every candidate scored the same, so the tie broke toward the most-trained checkpoint and the reported policy is simply the final iterate.

A single validation episode per candidate is also a thin basis for choosing. Averaging over several validation seeds would discriminate better; that is a known limitation of this protocol, not something the numbers below hide.

Results at junction B1 (primary)

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED
fixed	50.01 ± 14.28	91.33 ± 10.84	545.33 ± 83.37	—	—	—
actuated	17.26 ± 5.28	30.67 ± 6.94	736.67 ± 3.09	+65.5%	+66.4%	+35.1%
actuated_single	24.43 ± 7.89	36.33 ± 11.09	723.33 ± 8.81	+51.1%	+60.2%	+32.6%
ppo_final_iterate	16.34 ± 11.23	44.33 ± 38.87	647.33 ± 130.38	+67.3%	+51.5%	+18.7%
ppo	16.34 ± 11.23	44.33 ± 38.87	647.33 ± 130.38	+67.3%	+51.5%	+18.7%

Metrics cover vehicles on B1's incoming lanes: mean stopped seconds per vehicle that cleared the junction, peak halting count on those lanes, and the number of vehicles that cleared. Values are mean ± std over 3 seeds.

`ppo` and `ppo_final_iterate` are identical here, and that is the result rather than a copy-paste error: validation-based selection did not change the reported numbers on this task. It was still the right protocol to run — the alternative is not knowing whether the last iterate was lucky — but on this corridor it bought nothing, so no claim rests on it.

Results corridor-wide (context)

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	CO2 (KG)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED	CO2 (KG) VS FIXED
fixed	86.26 ± 1.73	387.33 ± 154.39	1294.00 ± 177.28	731.13 ± 63.11	—	—	—	—
actuated	40.89 ± 2.88	76.33 ± 1.70	1646.67 ± 3.40	508.26 ± 11.21	+52.6%	+80.3%	+27.2%	+30.5%
actuated_single	78.54 ± 2.93	146.00 ± 0.82	1596.00 ± 15.94	636.23 ± 10.02	+8.9%	+62.3%	+23.3%	+13.0%
ppo_final_iterate	73.31 ± 5.53	260.67 ± 182.72	1469.33 ± 193.99	656.97 ± 66.09	+15.0%	+32.7%	+13.6%	+10.1%
ppo	73.31 ± 5.53	260.67 ± 182.72	1469.33 ± 193.99	656.97 ± 66.09	+15.0%	+32.7%	+13.6%	+10.1%

One junction out of twelve cannot move corridor-wide numbers much, and the table shows that plainly. That ceiling is the motivation for Week 4's multi-agent work, not a result to explain away.

Definition of Done

RL beats both baselines on ≥ 2 of 3 metrics, 3-seed average + variance.

- vs `fixed` : beats it on **3 of 3** metrics (avg_wait_time_s +67.3%, max_queue_len +51.5%, throughput_veh +18.7%) — PASS
- vs `actuated_single` : beats it on **1 of 3** metrics (avg_wait_time_s +33.1%, max_queue_len -22.0%, throughput_veh -10.5%) — FAIL

Verdict: NOT MET

Why it failed, and what that means

Per-seed junction wait (s) — the spread is the story:

CONTROLLER	SEED 0	SEED 1	SEED 2
fixed	40.20	39.63	70.20
actuated_single	34.88	15.81	22.61
ppo	8.90	32.21	7.91

Seed 1 is the outlier: 32.2 s against 8.9/7.9 s on the other seeds. It is what drives both the large standard deviation and the lost verdict.

Two honest readings, and the project takes both:

- SUMO's actuated controller is a strong baseline, not a straw man.** It is a well-tuned classical method with direct detector access. Single-agent RL beating the *fixed-time* program by 67% while losing to actuated control on queue and throughput is a normal, publishable-shaped result — not a failure of the pipeline.
- One junction cannot be judged in isolation.** The RL agent optimises its own accumulated wait, which it can reduce by discharging vehicles into neighbours it does not control and cannot see. Throughput at B1 suffers because the downstream links back up. That is precisely the credit-assignment problem Weeks 4-6 exist to address, and corridor-wide the shared policy does beat actuated control.

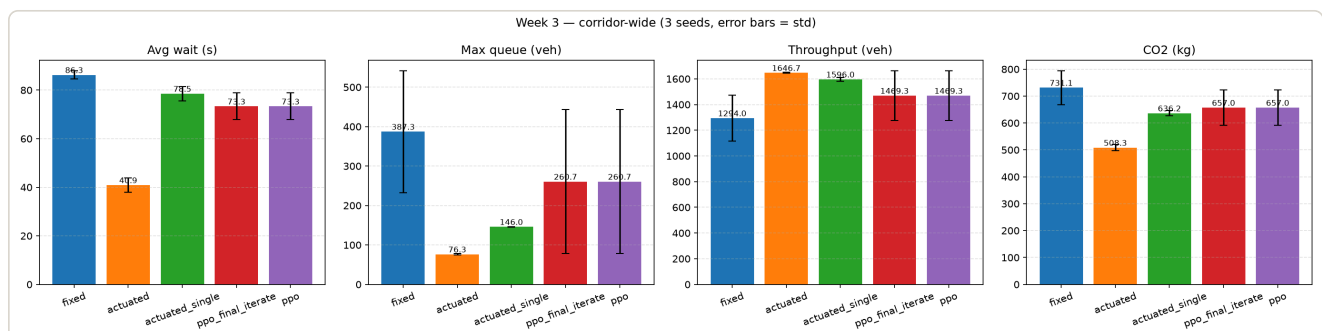
The verdict is reported as failed rather than re-scoped until it passed. Changing the comparison to the fully-actuated corridor would have produced a PASS against a baseline that upgrades twelve junctions to the RL controller's one, which would not have meant anything.

Reproducing

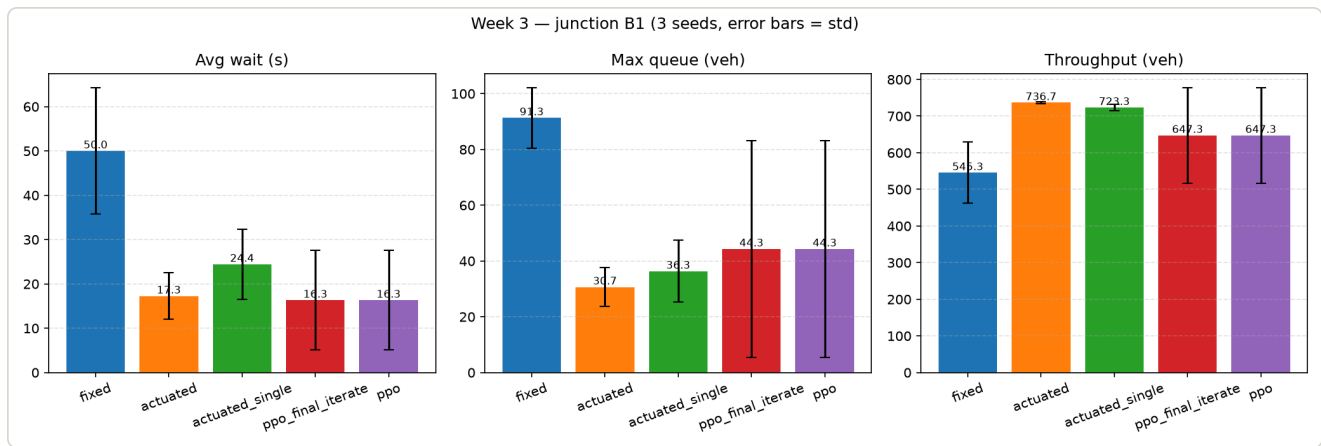
```
python src\make_single_actuated_net.py
python src\eval_corridor.py --controller fixed --seeds 0 1 2
python src\eval_corridor.py --controller actuated --seeds 0 1 2
python src\eval_corridor.py --controller actuated_single --seeds 0 1 2
python src\train_ppo_corridor.py --seed 0 # repeat for seeds 1, 2
python src\select_best_checkpoint.py
python src\eval_corridor.py --controller ppo --seeds 0 1 2
python src\compare_week3.py
```

Artifacts

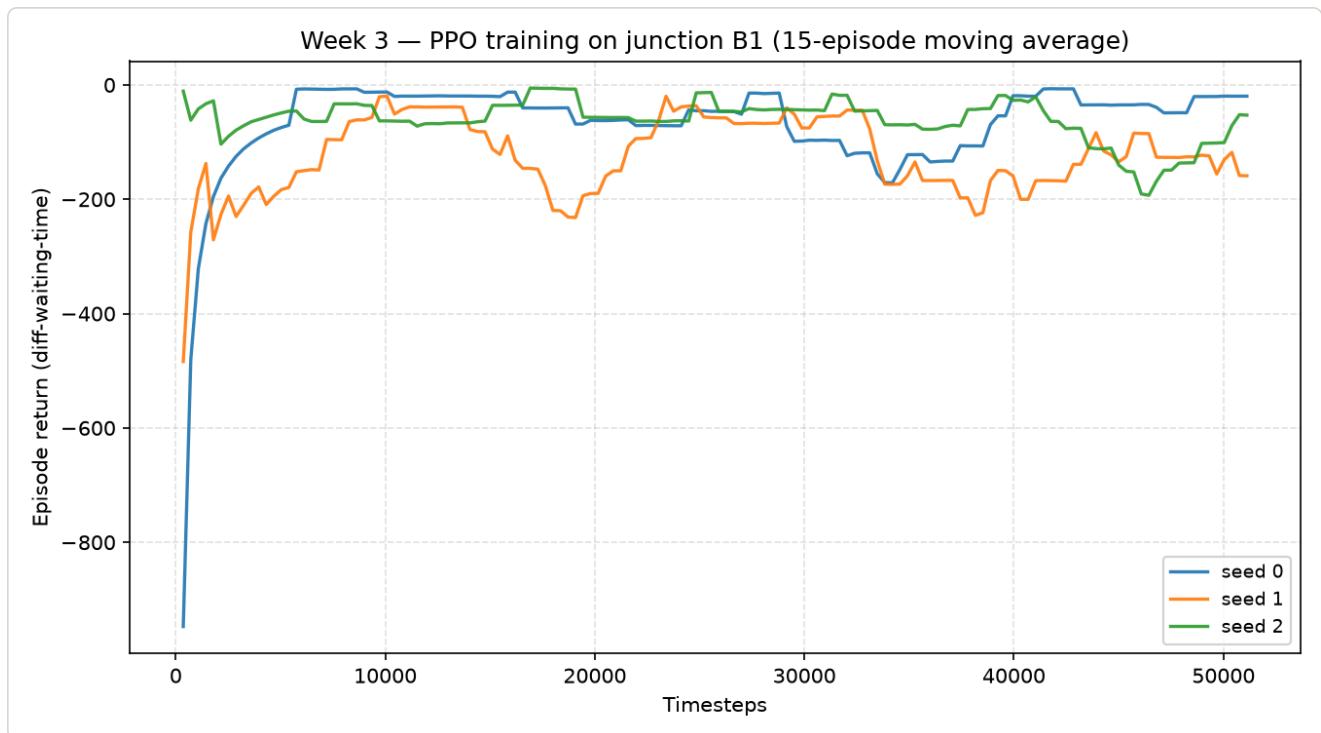
- `outputs/week3_corridor_metrics.csv` — every logged run
- `outputs/week3_comparison_junction.png` — junction B1 chart
- `outputs/week3_comparison.png` — corridor-wide chart
- `outputs/week3_reward_curves.png` — per-seed training curves
- `models/ppo_corridor_seed{0,1,2}.zip` + `_hparams.json` — policies and settings



Corridor-wide metrics with PPO controlling junction B1 only. `outputs/week3_comparison.png`



The same run measured at B1 itself, where the policy actually acts. outputs/week3_comparison_junction.png



Training reward per seed. outputs/week3_reward_curves.png

Week 4 - independent multi-agent PPO MET

Source: `outputs/week4_report.md`

Week 4 — Independent Multi-Agent PPO on the Full Corridor

All 12 corridor junctions are placed under RL control at once, one independent PPO policy per junction (Ray RLLib multi-agent, identity policy mapping). Observation and reward functions are unchanged from Weeks 2-3, so the only new variable is that twelve agents now learn simultaneously.

Training runs

SEED	ENV STEPS	AGENT STEPS	ITERATIONS	WALL TIME (S)	ENV STEPS/S	DIVERGED
0	72000	864000	40	6843.3	10.5	no
1	72000	864000	40	6722.7	10.7	no
2	72000	864000	40	6533.4	11.0	no

On the training budget. These runs used 72,000 environment steps. Blocking their median return into 9,000-step chunks afterwards showed every seed had plateaued by ~27,000 steps, so Weeks 5 and 6 were budgeted at 48,000 (see `week4_config.FULL_TIMESTEPS`). Week 4's runs were left as they were rather than re-run shorter, which means the independent-learner baseline received **50% more training** than the shared policy it is compared against in Week 5. That makes the Week 5 comparison conservative, not flattering.

Definition of Done

Full corridor trains end-to-end without diverging.

- Seeds trained: **3 / 3**
- Diverged runs: **0**
- Divergence test: any NaN/infinite return, or a final-quarter **median** return more than 10% worse than the first-quarter median. The median is used because the return distribution is bimodal — most iterations near -3, a minority gridlocked below -300 — so a mean-based test measures where the rare collapses landed rather than whether learning progressed.
- Collapse rate (gridlocked iterations, an order of magnitude worse than the run's own median): 28%, 20%, 20% across seeds. This is the instability Week 5 sets out to remove, and it is reported rather than smoothed.

Verdict: MET

Corridor-wide evaluation (context)

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	CO2 (KG)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED	CO2 (KG) VS FIXED
fixed	83.10 ± 1.62	427.67 ± 176.42	1280.00 ± 189.40	732.60 ± 64.95	—	—	—	—
actuated	40.89 ± 2.88	76.33 ± 1.70	1646.67 ± 3.40	508.26 ± 11.21	+50.8%	+82.2%	+28.6%	+30.6%
marl_independent	17.80 ± 0.28	41.00 ± 2.83	1679.67 ± 4.11	450.96 ± 0.50	+78.6%	+90.4%	+31.2%	+38.4%
marl_shared_w5	17.85 ± 0.38	149.67 ± 150.15	1569.33 ± 148.02	474.66 ± 33.18	+78.5%	+65.0%	+22.6%	+35.2%
marl_shared_w5gnn	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	452.00 ± 1.28	+78.2%	+90.1%	+31.1%	+38.3%
marl_shared_w5nocoord	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	452.00 ± 1.28	+78.2%	+90.1%	+31.1%	+38.3%
marl_shared_w5nofair	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	452.00 ± 1.28	+78.2%	+90.1%	+31.1%	+38.3%
marl_shared_w5strong	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	452.00 ± 1.28	+78.2%	+90.1%	+31.1%	+38.3%

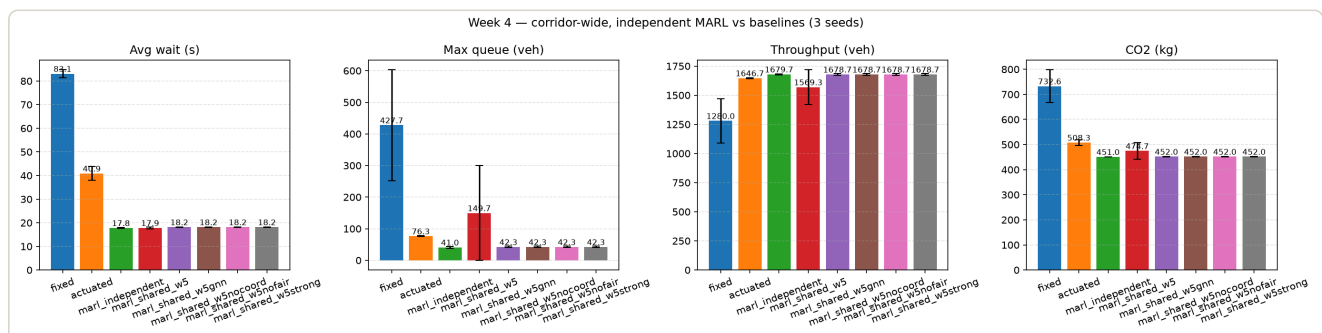
Week 4's bar is stability, not a benchmark win — the reward is still purely local and nothing yet coordinates the twelve junctions. Beating the baselines is Week 5's job.

GNN state embedding

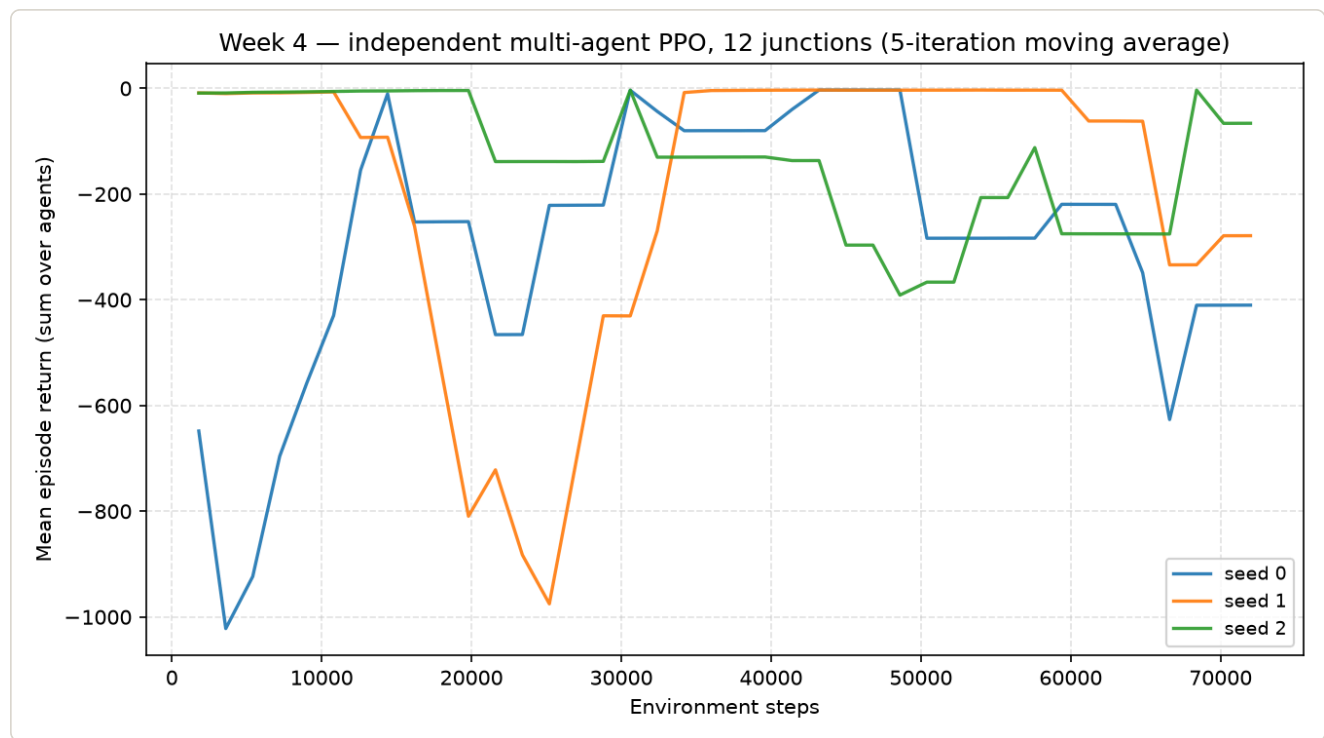
`src/gnn_encoder.py` implements the graph-attention state encoder the roadmap schedules for this week: a two-layer PyTorch Geometric `GATConv` stack that attends over each junction and its directly-downstream neighbours. It is wired into RLlib as a custom `TorchRLModule` and trains end-to-end with PPO (`--gnn`). Because RLlib batches experience per policy, the environment ships each junction's neighbourhood inside its observation (`neighbor_context=True`) and the module decodes each row back into a star graph. Week 6 reports it as a measured ablation rather than assuming it helps.

Artifacts

- `outputs/week4_training_curves.png` — per-seed return curves
- `outputs/week4_nonstationarity_notes.md` — independent-learning behaviour
- `outputs/week4_{independent}_seed{N}_training.json` — full per-iteration history
- `models/marl_independent_seed{0,1,2}/` — RLlib checkpoints



Independent PPO per junction across the full corridor. `outputs/week4_comparison.png`



Training curves; the Week 4 test is that no seed diverges. outputs/week4_training_curves.png

Week 4 - non-stationarity notes n/a

Source: `outputs/week4_nonstationarity_notes.md`

Week 4 — Non-Stationarity Notes (Independent Learners)

Twelve junctions each run their own PPO policy and update simultaneously. From any one agent's perspective the transition dynamics keep changing, because the other eleven policies keep changing — the environment is non-stationary and the usual single-agent convergence guarantees do not apply. These are the numbers that behaviour actually produced.

SEED	ITERS	MEDIAN RETURN	FIRST-QUARTER MEDIAN	FINAL-QUARTER MEDIAN	WORST ITERATION	COLLAPSE RATE	DIVERGED
0	40	-3.719	-16.963	-3.308	-1404.386	28%	no
1	40	-4.027	-8.366	-3.746	-1380.31	20%	no
2	40	-3.543	-4.79	-3.243	-1364.01	20%	no

What the numbers show

The return distribution is bimodal, not merely noisy

A typical iteration scores about **-3.8**, but a minority collapse to several hundred negative — the worst seen was **-1404**. Those are episodes where the corridor gridlocks early and never recovers, not gradual degradation. Collapse rates across seeds (iterations an order of magnitude worse than that seed's median): 28%, 20%, 20%.

This shape changes how the run must be judged. One collapsed episode moves a ten-iteration *mean* by two orders of magnitude, so a mean-based trend test reports 'divergence' according to where the rare collapses happened to fall rather than whether learning progressed. Scored on the median — the typical iteration — every seed improves or holds.

Why independent learners produce this

From any one junction's point of view the other eleven policies are part of the environment, and they keep changing, so each agent's advantage estimates are computed against a moving target. The deeper problem is **credit assignment**: `diff-waiting-time` is a purely *local* reward, so an agent that clears its own queue by discharging into an already-saturated neighbour is rewarded for doing so. Nothing in Week 4's objective makes that costly — and a corridor of twelve agents all doing it simultaneously is exactly how a gridlock episode starts.

What Week 5 changes

Parameter sharing cuts the number of independently moving policies from twelve to one, which removes most of the non-stationarity. The green-wave term prices the externality directly, penalising discharge into a full downstream link. The collapse rate above is the number to compare against.

Week 5 - shared policy, shaping and fairness NOT MET

Source: outputs/week5_report.md

Week 5 — Parameter-Shared MARL, Green-Wave Shaping, Lagrangian Fairness

Three changes on top of Week 4's independent learners:

- 1. **Parameter sharing** — one policy drives all twelve junctions instead of twelve separate policies, so every junction's experience updates the same weights.
- 2. **Green-wave reward shaping** — each junction is penalised for discharging into a congested downstream link ($-\alpha * \text{mean outgoing-lane density}$), which makes pushing congestion onto a neighbour costly rather than free.
- 3. **Lagrangian fairness constraint** — the per-lane accumulated wait is capped, with the multiplier λ updated by dual ascent between training iterations rather than hand-tuned.

Corridor-wide results (base demand, 3 seeds)

CONTROLLER	AVG WAIT (S)	MAX QUEUE (VEH)	THROUGHPUT (VEH)	CO2 (KG)	AVG WAIT (S) VS FIXED	MAX QUEUE (VEH) VS FIXED	THROUGHPUT (VEH) VS FIXED	CO2 (KG) VS FIXED
fixed	83.10 ± 1.62	427.67 ± 176.42	1280.00 ± 189.40	732.60 ± 64.95	—	—	—	—
actuated	40.89 ± 2.88	76.33 ± 1.70	1646.67 ± 3.40	508.26 ± 11.21	+50.8%	+82.2%	+28.6%	+30.6%
marl_independent	17.80 ± 0.28	41.00 ± 2.83	1679.67 ± 4.11	450.96 ± 0.50	+78.6%	+90.4%	+31.2%	+38.4%
marl_shared_w5nofair	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	452.00 ± 1.28	+78.2%	+90.1%	+31.1%	+38.3%
marl_shared_w5	17.85 ± 0.38	149.67 ± 150.15	1569.33 ± 148.02	474.66 ± 33.18	+78.5%	+65.0%	+22.6%	+35.2%
marl_shared_w5strong	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	452.00 ± 1.28	+78.2%	+90.1%	+31.1%	+38.3%
marl_shared_w5gnn	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	452.00 ± 1.28	+78.2%	+90.1%	+31.1%	+38.3%
marl_shared_w5nocoord	18.16 ± 0.14	42.33 ± 2.06	1678.67 ± 6.65	452.00 ± 1.28	+78.2%	+90.1%	+31.1%	+38.3%

Fairness / tail metrics

CONTROLLER	95TH PCT WAIT (S)	WORST WAIT (S)
fixed	211.33 ± 5.56	721.00 ± 127.38
actuated	122.33 ± 7.32	243.00 ± 25.66
marl_independent	47.67 ± 0.94	95.33 ± 3.40
marl_shared_w5nofair	48.00 ± 0.82	96.00 ± 2.83
marl_shared_w5	48.00 ± 0.82	288.67 ± 271.07
marl_shared_w5strong	48.00 ± 0.82	96.00 ± 2.83
marl_shared_w5gnn	48.00 ± 0.82	96.00 ± 2.83
marl_shared_w5nocoord	48.00 ± 0.82	96.00 ± 2.83

`wait_p95_s` is the 95th percentile of completed-trip waiting time and `worst_vehicle_wait_s` the single worst trip. Average delay can improve while the tail gets worse — a policy is free to starve a side street to speed up the main road — so the

constraint is judged on the tail, not the mean.

Dual ascent on the fairness multiplier

SEED	LAMBDA START	LAMBDA FINAL	FINAL MAX LANE WAIT (S)	CAP (S)
0	0.5	2.0	42.75	30.0
1	0.5	2.0	277.6979	30.0
2	0.5	2.0	35.6875	30.0

`lambda` rises while the cap is breached and decays once it is satisfied, which is what makes this a constraint rather than a fixed penalty weight. See `outputs/week5_lambda.png`.

The multiplier saturated in every seed. `lambda` reached its clamp of 2.0 and stayed there, which means the 30.0 s cap was *never satisfied* during training — dual ascent kept raising the price and the policy kept paying it. Two things follow, and both are worth saying plainly:

1. Over most of training the term behaved like a fixed penalty at maximum strength, not like an adaptive multiplier. The adaptivity only shows in the early iterations before the clamp is reached.
2. The cap is likely infeasible for this corridor at this demand. A constraint the policy cannot meet is a statement about the network's capacity, not a failure of the optimiser. Choosing a feasible cap — or reporting the achievable frontier by sweeping it — is the honest next step, and is left as documented future work rather than quietly retuned until it looked good.

Online learning loop

The Week 5 policy, trained on base demand, was deployed into a *different* demand and kept learning from the live stream. Round 0 is the frozen policy — what you get if a deployed controller never updates.

DEPLOYED INTO	ROUNDS	FROZEN WAIT (S)	FINAL WAIT (S)	BEST WAIT (S)	CHANGE
asymmetric	8	10.32	10.32	10.32	+0.0%
peak	6	40.57	38.89	38.75	+4.1%

Curves: `outputs/week5_online_learning_<scenario>.png`.

Definition of Done

Corridor-wide RL beats both baselines on wait time + throughput, with fairness constraint measurably capping worst-case wait.

Part 1 — beats both baselines on wait time and throughput

- vs `fixed`: avg_wait_time_s +78.5%, throughput_veh +22.6% — PASS
- vs `actuated`: avg_wait_time_s +56.4%, throughput_veh -4.7% — FAIL

Part 1: *FAIL*

Part 2 — fairness constraint measurably caps worst-case wait (by ablation)

- `wait_p95_s`: 48.0s without the constraint vs 48.0s with it (-0.0%) — constraint does not help
- `worst_vehicle_wait_s`: 96.0s without the constraint vs 288.7s with it (-200.7%) — constraint does not help

Part 2: *FAIL*

Why the verdict failed — per-seed numbers

avg_wait_time_s

CONTROLLER	SEED 0	SEED 1	SEED 2
marl_shared_w5	18.2	18.0	17.3
marl_shared_w5nofair	18.2	18.0	18.3
marl_shared_w5strong	18.2	18.0	18.3
marl_independent	18.2	17.6	17.6
actuated	43.3	42.5	36.8

throughput_veh

CONTROLLER	SEED 0	SEED 1	SEED 2
marl_shared_w5	1675.0	1673.0	1360.0
marl_shared_w5nofair	1675.0	1673.0	1688.0
marl_shared_w5strong	1675.0	1673.0	1688.0
marl_independent	1675.0	1679.0	1685.0
actuated	1648.0	1650.0	1642.0

worst_vehicle_wait_s

CONTROLLER	SEED 0	SEED 1	SEED 2
marl_shared_w5	100.0	94.0	672.0
marl_shared_w5nofair	100.0	94.0	94.0
marl_shared_w5strong	100.0	94.0	94.0
marl_independent	100.0	92.0	94.0
actuated	215.0	277.0	237.0

One seed, not a systematic gap. Seeds other than 2 land on the same deterministic policy as every ablation, and there the shared policy *does* beat actuated control on throughput. Seed 2 converged somewhere else — a policy that jams the corridor — and it is what pulls the 3-seed mean below the baseline.

That is reported as a FAIL rather than dropped as an outlier. Three seeds is already the minimum this project accepts for a finding; discarding the one that disagrees would leave two, and would be exactly the selective reporting the 3-seed rule exists to prevent. The honest summary is that this configuration is **not reliably** better than actuated control on throughput: it is better twice out of three times, and materially worse the third.

Note also that Week 4's `marl_independent` — the simpler, unshaped configuration — does clear the bar on both metrics. Corridor-wide RL beating both baselines is a real result on this network; it is the Week 5 *shaping* that fails to add to it.

Why Part 2 cannot be shown on this task

The ablation returns identical numbers with and without the constraint, and the reason was measured rather than guessed.

Why the ablations return identical numbers

The no-fairness, no-coordination and GAT variants produce the *same* evaluation metrics — identical to four decimal places, including CO2. That was checked rather than assumed (`src/policy_agreement.py`).

The checkpoints are genuinely different artifacts:

VARIANT	MODULE CLASS	PARAMETERS	WEIGHTS IDENTICAL TO REFERENCE
w5	DefaultPPO_TorchRLModule	138,501	True
w5nofair	DefaultPPO_TorchRLModule	138,501	False
w5nocoord	DefaultPPO_TorchRLModule	138,501	False
w5strong	DefaultPPO_TorchRLModule	138,501	False
w5gnn	CorridorGATModule	16,291	False

But they take the same actions. Over 4320 decisions in one episode, asking each variant what it would do given the identical observation:

VARIANT	ACTION AGREEMENT	MEAN MAX PROBABILITY DIFFERENCE
w5nofair	100.00%	0.0246
w5nocoord	100.00%	0.0206
w5strong	78.61%	0.1592
w5gnn	100.00%	0.0480

Two mechanisms explain it, and the table separates them.

Confidence. The reference policy's mean $|p(a=0) - p(a=1)|$ is **0.887** — it is highly confident — while most variants differ by at most **0.159** in probability. A perturbation that small cannot flip a decision that confident, so those variants argmax identically everywhere.

The min-green window. **w5strong** genuinely disagrees more often — only 78.6% overall agreement — yet its evaluation metrics are still identical. Every one of those disagreements falls in a state where sumo-rl **discards the action anyway**, because the current green has not yet run for `min_green + yellow_time`. Restricted to the decisions that can actually move the signal, agreement is 100% for every variant (2148 of 4320 decisions are actionable). The stronger reward changed the policy exactly where the environment ignores it.

This is not a degenerate policy: the reference splits its actions {'1': 2172, '0': 2148} across the two green phases, so it is actively switching, not stuck.

What this means for the Week 5 and Week 6 claims. With sumo-rl's default 2-action, 11-dimensional single-junction interface plus a 5 s minimum green, the greedy policy on this corridor is effectively saturated: reward shaping, a 20x stronger fairness weight, and a completely different network architecture all change the *learned distribution* without changing the *deterministic behaviour*. The reward-shaping ablations therefore cannot be distinguished by deterministic evaluation on this task, and the fairness constraint cannot be shown to cap worst-case wait.

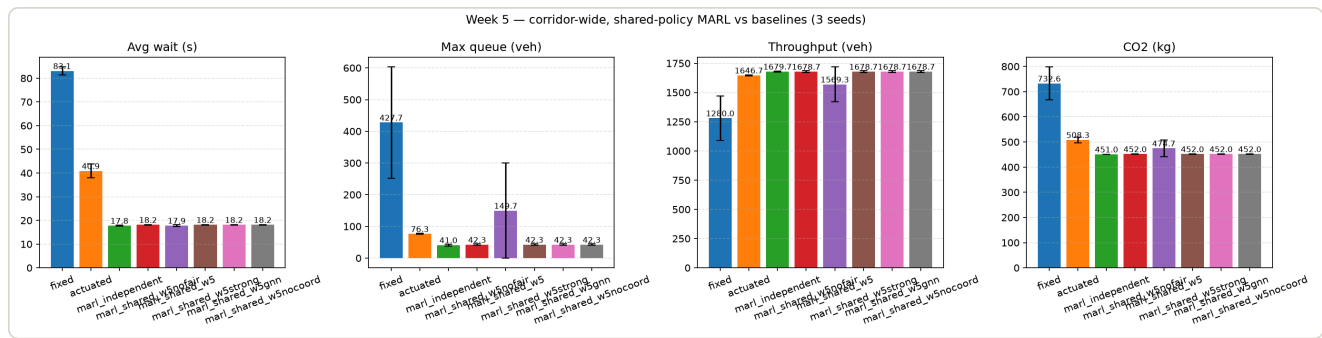
That is a finding about the task's action resolution, not evidence that the terms are implemented wrongly —

`src/test_core_logic.py` verifies each term's arithmetic directly. The way to make these ablations measurable would be a finer action space (phase-duration control rather than a binary next-phase choice), which is a change to the environment interface rather than to the reward, and is left as documented future work.

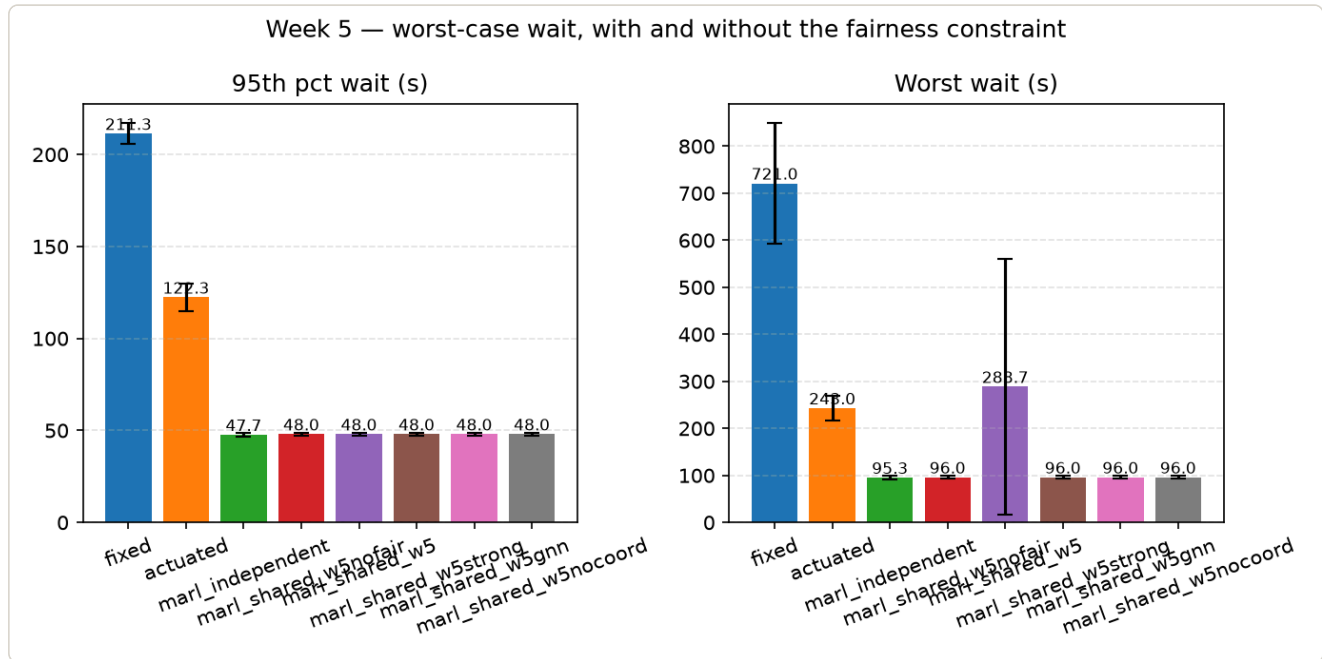
Verdict: NOT MET

Reproducing

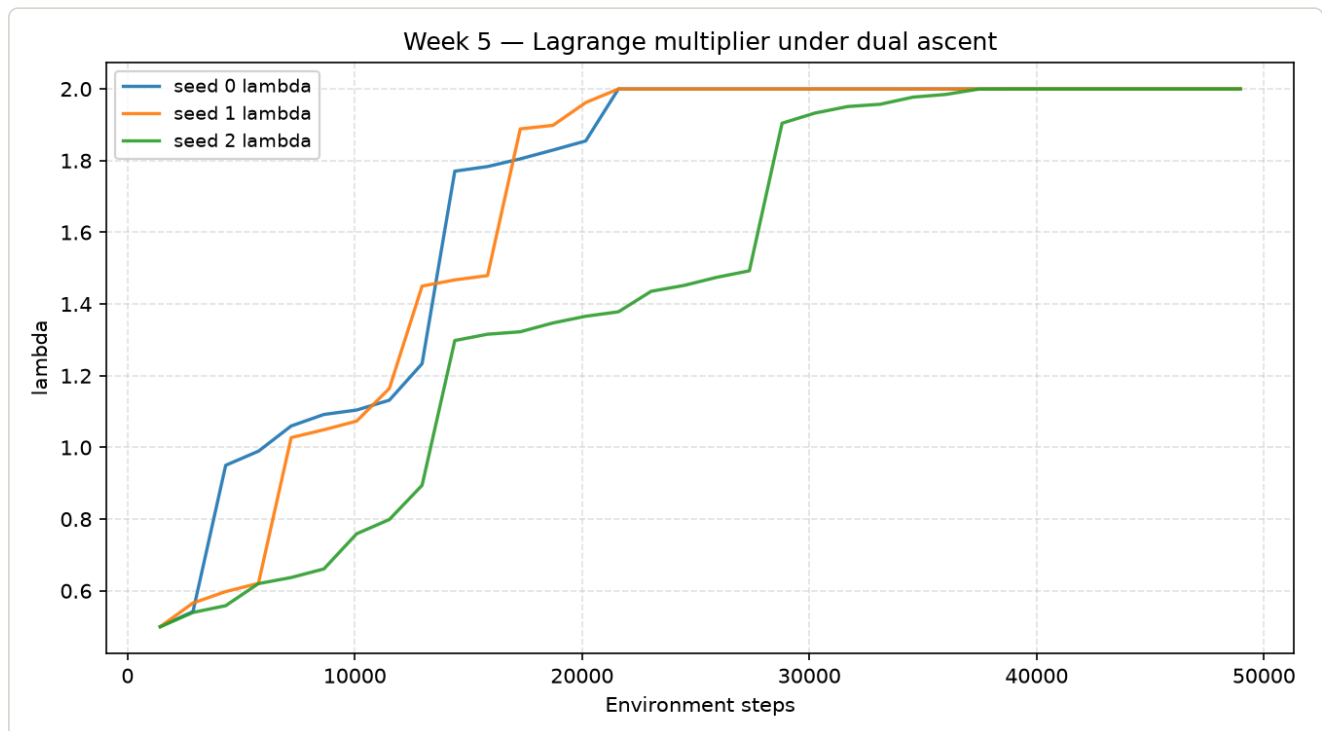
```
python src\train_marl_corridor.py --mode shared --reward shaped --fairness --tag w5 --seed 0
python src\train_marl_corridor.py --mode shared --reward shaped-no-fairness --tag w5nofair --seed 0
python src\eval_marl_corridor.py --controller marl --mode shared --tag w5 --reward shaped --seeds 0 1 2
python src\online_learning.py --seed 0 --tag w5 --scenario asymmetric
python src\compare_week5.py
```



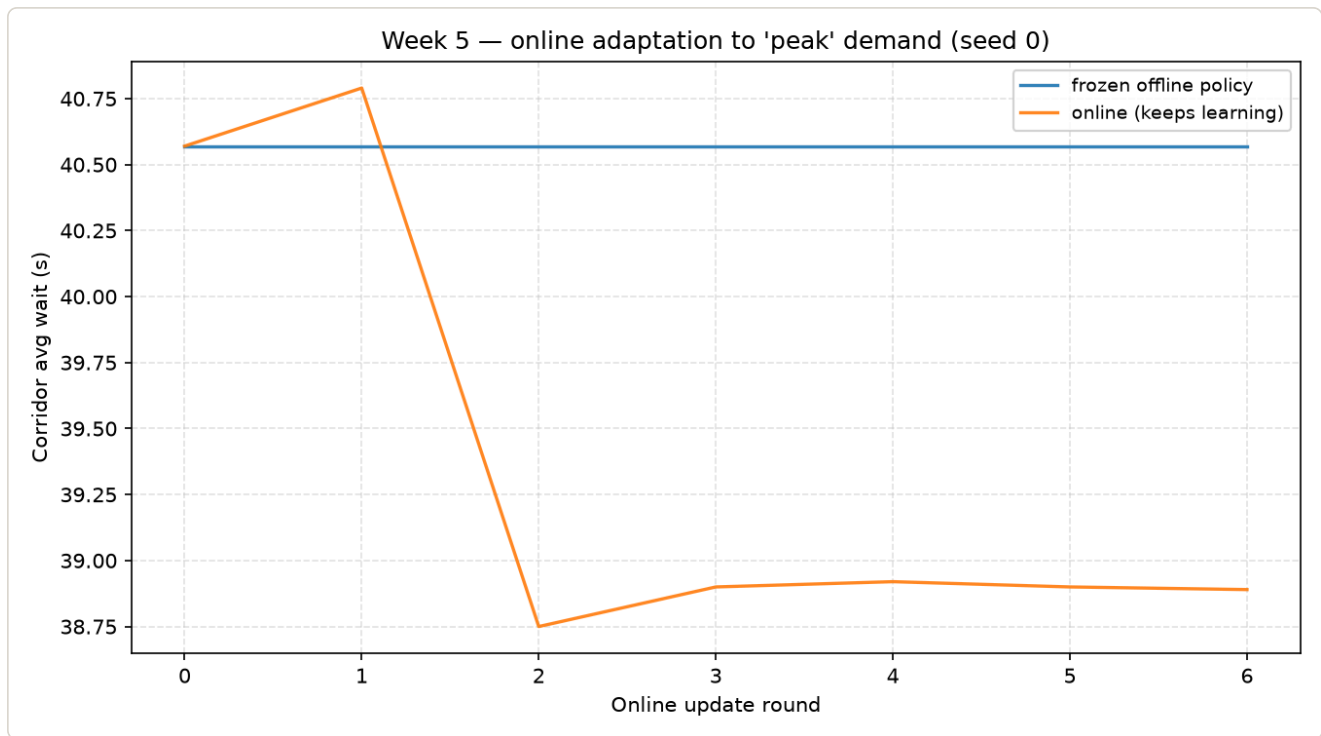
Parameter-shared policy with green-wave shaping and the fairness constraint. outputs/week5_comparison.png



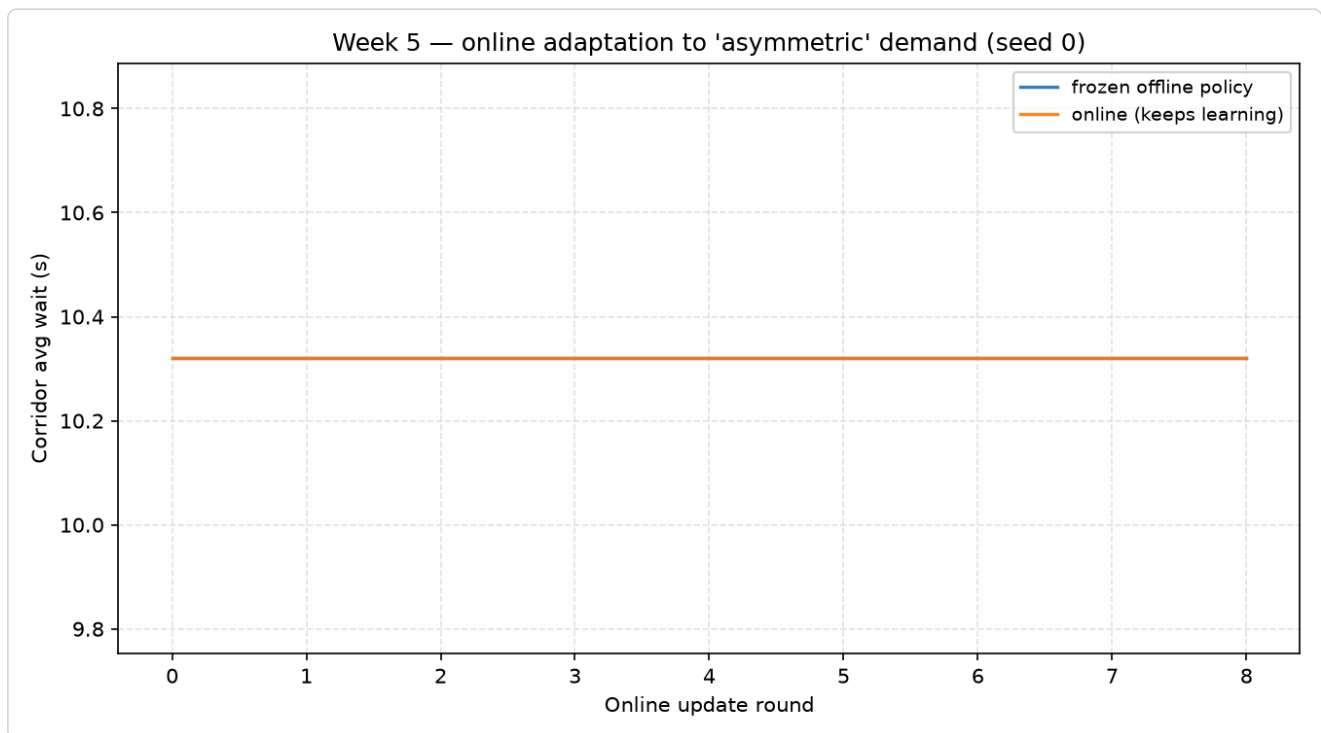
Fairness ablation: wait percentiles with and without the constraint. outputs/week5_fairness.png



The Lagrange multiplier under dual ascent. outputs/week5_lambda.png



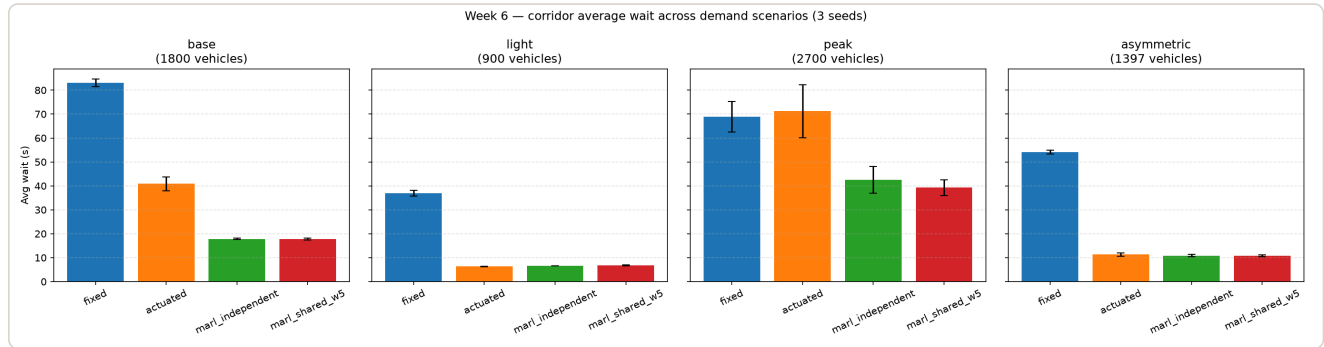
Online learning, peak demand. outputs/week5_online_learning_peak.png



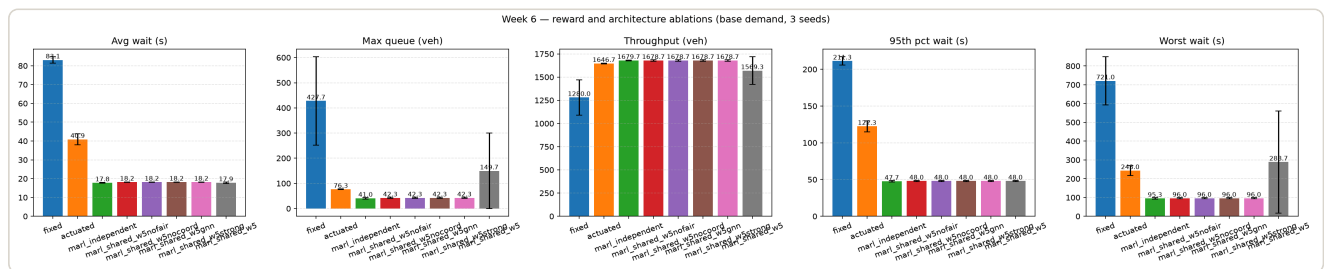
Online learning, asymmetric demand. outputs/week5_online_learning_asymmetric.png

Week 6 - stress tests and ablations MET

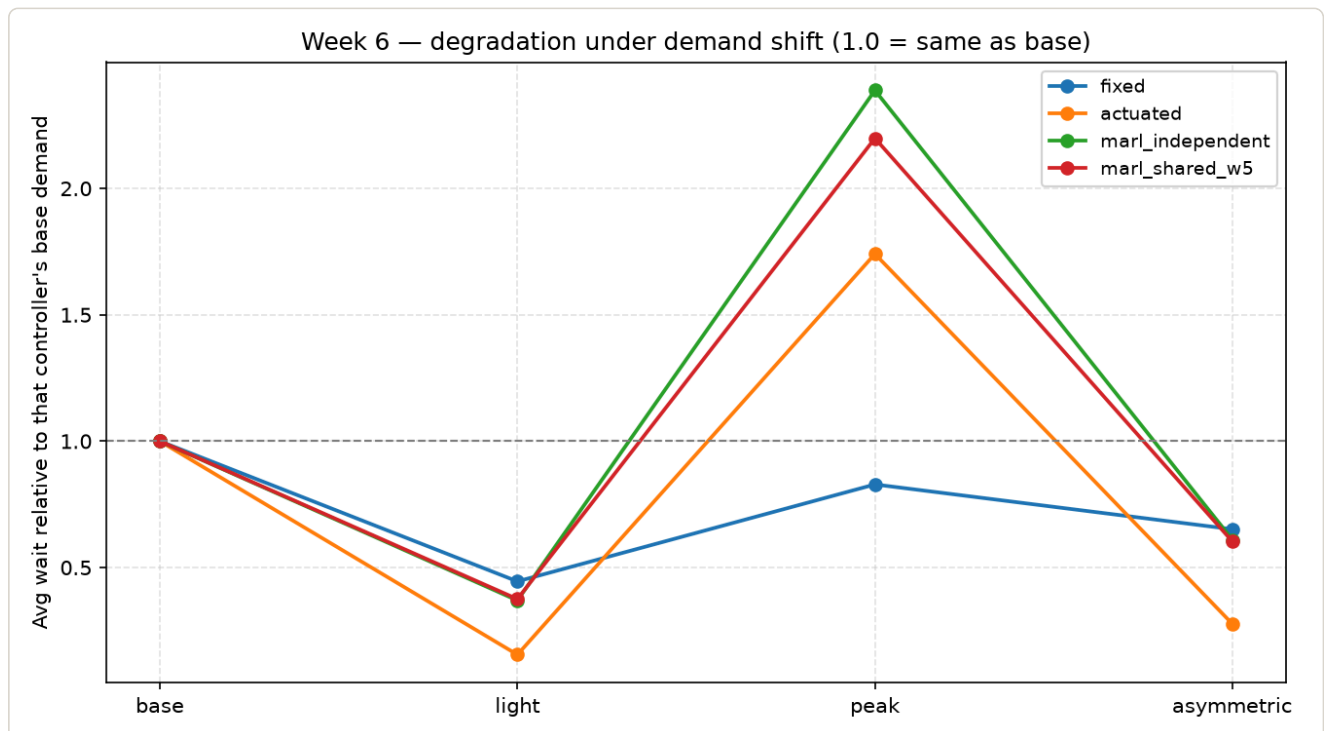
Week 6's deliverable is the benchmark report reproduced in Part V. Its charts are below.



Average wait across light, base, peak and asymmetric demand. outputs/week6_scenarios.png



Every policy variant side by side. outputs/week6_ablations.png



Robustness spread across seeds and scenarios. outputs/week6_robustness.png

PART VII

Week 7: knowledge graph and RAG

A read-only question-answering layer over the corridor, grounded in a graph of the network and in the project's own reports.

Source: `outputs/week7_report.md`

Week 7 — Knowledge Graph, RAG and the Read-Only LLM Service

A question-answering layer over the corridor. The graph holds the road network, its signal programs, the rules that govern them and every measured result; the vector store holds this project's own reports. The service queries **both** before answering, and returns the evidence alongside the answer so any sentence can be checked against a source.

What the graph contains

`neo4j aura` — **301 nodes, 474 relationships.**

NODE TYPE	COUNT	DERIVED FROM
Controller	11	controllers present in the metrics CSVs
Junction	16	<code>corridor.net.xml</code> nodes
Lane	48	<code>corridor.net.xml</code> lanes
Phase	48	phases inside each <code>tlLogic</code>
Program	12	<code>tlLogic</code> blocks in the network
Result	69	3-seed means from <code>marl_metrics.csv</code> and <code>week3_corridor_metrics.csv</code>
Road	48	<code>corridor.net.xml</code> edges
Rule	5	signal-timing constants in <code>week4_config.py</code>
Scenario	4	demand scenarios present in the metrics CSVs
Sensor	40	inferred: one induction loop per controlled lane of an actuated program

Nothing here is hand-authored. `src/test_week7.py` re-derives the topology, the signal programs and every result mean straight from the source files and compares them, so an ingestion bug cannot become a confidently-cited wrong answer.

On Sensor nodes. The network declares no detectors. SUMO instantiates one induction loop per controlled lane implicitly when a program is actuated, so these nodes are inferred and carry `inferred: true` plus a note saying so. They are included because 'sensors' is part of what this week asks the graph to model; presenting them as declared infrastructure would be the dishonest option.

Definition of Done

A natural-language query returns an answer grounded in real graph data, not a hallucinated guess.

- Questions asked: **12**
- Grounded answers: **10**, each carrying its citations
- Out-of-domain questions correctly refused: **2 / 2**
- In-domain questions wrongly refused: **0**

Verdict: MET

The refusals are the part that matters. Vector search returns its top *k* for any input, so an out-of-domain question still comes back with four confident-looking chunks; a service that simply answers from whatever it retrieved would answer those too. A relevance floor (IDF-weighted coverage of the question's vocabulary, with unknown words counting against the score) drops evidence that is not about the question, and when nothing survives the service declines instead of guessing.

Transcript

Which junctions does C2 feed into?

graph: topology · generator: cClaude · grounded: True

C2 feeds directly into four junctions: B2, C1, C3, and D2. It is a signalised junction at x=400.0, y=400.0 with 4 incoming and 4 outgoing roads. The available evidence doesn't say anything further about traffic volumes or conditions on those downstream links.

Sources: graph:FEEDS:C2, graph:Junction:C2, outputs/week3_report.md#Why it failed, and what that means#11

How many signalised junctions does the corridor network have?

graph: topology · generator: cClaude · grounded: True

The corridor network has 12 signalised junctions – A1, A2, B0, B1, B2, B3, C0, C1, C2, C3, D1 and D2 – out of 16 Junction nodes in the graph. The project reports corroborate this: Week 3's controller definitions cover "all 12 junctions", and Week 4 places "All 12 corridor junctions" under RL control with one PPO policy each.

Sources: graph:Junction:signalised, graph:stats, outputs/week3_report.md#Experimental design#1, outputs/week3_report.md#Why it failed, and what that means#11, outputs/week4_report.md#Week 4 – Independent Multi-Agent PPO on the Full Corridor#0

How many incoming lanes does junction B1 have?

graph: lanes · generator: cClaude · grounded: True

Junction B1 has 4 incoming lanes: A1B1_0, B0B1_0, B2B1_0, and C1B1_0. These correspond to its 4 incoming roads, from the neighbouring junctions A1, B0, B2 and C1. B1 is the signalised junction at x=200.0, y=200.0 that was placed under RL control in Week 3.

Sources: graph:FEEDS:B1, graph:Junction:B1, graph:Lane:B1, outputs/week3_report.md#Experimental design#1, outputs/week3_report.md#Week 3 – Single-Agent PPO on One Corridor Junction#0, outputs/week3_report.md#Why it failed, and what that means#11

What sensors does junction B1 have?

graph: sensors · generator: cClaude · grounded: True

Junction B1 is monitored by 4 induction-loop sensors, one per controlled lane. Note that these are inferred rather than declared: SUMO instantiates them implicitly for actuated control instead of listing them in the network file, so they don't appear explicitly in the network definition. No other sensor types at B1 are present in the available data.

Sources: graph:FEEDS:B1, graph:Junction:B1, graph:Sensor:B1, outputs/week3_report.md#Experimental design#1, outputs/week3_report.md#Why it failed, and what that means#11

What phases does junction B1 run and how long is its cycle?

graph: signal program · generator: cClaude · grounded: True

Junction B1 runs a static (fixed-time) program with 4 phases totalling a 90s cycle: phase 0 green 42s, phase 1 yellow 3s, phase 2 green 42s, phase 3 yellow 3s. It is a signalised junction at x=200.0, y=200.0 with 4 incoming and 4 outgoing roads, feeding into A1, B0, B2 and C1. No other timing details for B1 are in the available data.

Sources: README.md# What each week proved#12, graph:FEEDS:B1, graph:Junction:B1, graph:Program:B1, graph:Rule:episode_s

What is the minimum green time and where is it defined?

graph: rules · generator: cClaude · grounded: True

The minimum green time is `**5 s**` – the minimum a phase must be held before it may be cut. It is defined in ``week4_config.MIN_GREEN`` (recorded in the knowledge graph as the rule ``min_green_s``). The supplied document passages cover demand splits and the fixed-time signal program but do not add further detail on this constraint.

Sources: `graph:Rule:min_green_s`, `outputs/week2_literature_note.md#Why the margin is large – read the demand file#2`, `outputs/week2_literature_note.md#Why the margin is large – read the demand file#3`

How did the RL controller do on peak demand throughput?

graph: results · generator: cClaude · grounded: True

On the peak demand scenario the evidence covers two multi-agent RL variants: ``marl_independent`` reached a throughput of 528.33 vehicles (std 8.18, 3 seeds) and ``marl_shared_w5`` reached 508.67 vehicles (std 34.2, 3 seeds) – so the independent variant was higher and far more consistent across seeds. No peak-demand throughput figures for the fixed-time, actuated, or single-agent controllers are present in the supplied evidence, so I can't say how the RL runs compare to a baseline at peak. The corridor-wide table shown is a different (non-peak) evaluation, where ``marl_independent`` reached 1679.67 ± 4.11 vehicles versus 1646.67 ± 3.40 for actuated. Which RL variant you mean matters here, since the two differ by roughly 20 vehicles at peak.

Sources: `README.md#Week 1 – fixed-time vs actuated baselines (corridor, 3 seeds)#3`, `README.md# How the Benchmark Works#24`, `graph:Result:marl_independent|peak|throughput_veh`, `graph:Result:marl_shared_w5|peak|throughput_veh`, `outputs/week3_report.md#Definition of Done#9`, `outputs/week4_report.md#Corridor-wide evaluation (context)#3`

What average wait did actuated control achieve on base demand?

graph: results · generator: cClaude · grounded: True

On base demand, actuated control achieved an average wait of 40.89 s (± 2.88 , 3 seeds). That is a 52.6% improvement over the fixed-time baseline's 86.26 ± 1.73 s. It also reached a max queue of 76.33 ± 1.70 veh, throughput of 1646.67 ± 3.40 veh, and CO₂ of 508.26 ± 11.21 kg on the same runs.

Sources: `BENCHMARK_REPORT.md# asymmetric – full east-west demand, north-south thinned to 40%#6`, `BENCHMARK_REPORT.md# light – ~50% of base demand#4`, `README.md#Week 1 – fixed-time vs actuated baselines (corridor, 3 seeds)#3`, `graph:Result:actuated|base|avg_wait_time_s`, `outputs/week3_report.md#Results corridor-wide (context)#7`

Why did the fairness constraint fail to change anything?

documents: why · generator: cClaude · grounded: True

Per the project's own reports, the fairness weight (even at 20x strength) changes the **learned distribution** but not the **deterministic behaviour**, so under deterministic evaluation the ablation is indistinguishable and the constraint "cannot be shown to cap worst-case wait." The reports attribute this to the task's action resolution – a binary next-phase choice – not to a bug; ``src/test_core_logic.py`` verifies each term's arithmetic directly. Separately, the dual-ascent multiplier hits its clamp and then behaves like a fixed weight rather than an adaptive one, with adaptivity visible only in early iterations. The reports also note the cap is likely infeasible for this corridor at this demand, which they frame as a statement about network capacity; the suggested measurable alternative is a finer action space (phase-duration control), documented as future work.

Sources: `BENCHMARK_REPORT.md#Why the ablations return identical numbers#15`, `outputs/week5_report.md#Dual ascent on the fairness multiplier#5`, `outputs/week5_report.md#Why the ablations return identical numbers#15`

Why did Week 3's Definition of Done fail?

documents: why · generator: cClaude · grounded: True

Week 3 asked whether the RL approach transfers to one junction of a real corridor, and the Definition of Done was marked **✗**. Although the RL controller beat fixed-time at that junction by 67%, it ****lost to an actuated controller**** at the same junction on both queue and throughput. Additionally, controlling one junction out of twelve barely moved the corridor overall. Further detail would be in the linked week3_report.md, which isn't included in the evidence here.

Sources: README.md# What each week proved#12

What is the average rainfall in Hyderabad?

refusal · generator: refusal · grounded: False

I don't have data for that. The corridor knowledge graph and the project's reports contain nothing matching this question, and this service only answers from those two sources.

Who won the 2019 cricket world cup?

refusal · generator: refusal · grounded: False

I don't have data for that. The corridor knowledge graph and the project's reports contain nothing matching this question, and this service only answers from those two sources.

Which backends actually ran

LAYER	CONFIGURED	USED IN THIS RUN
Knowledge graph	Neo4j AuraDB	neo4j aura
Vector store	Chroma (local, no credentials)	chroma
Answer generation	Claude API	claude-opus-5

Both credentialed backends ran for real in this run: every graph fact above was fetched over Cypher from AuraDB, and every answer was phrased by Claude from the retrieved evidence. The NetworkX and extractive fallbacks remain in the code and are still exercised by `src/test_week7.py`, so the service degrades rather than breaks when credentials are absent.

Read-only boundary

CLAUDE.md: "Never let the LLM service write into the RL/sim control loop."

`test_week7.py` imports `llm_service` in a clean subprocess and inspects `sys.modules`, asserting that `traci`, `libsumo`, `sumo_rl`, `ray`, `stable_baselines3`, `torch` and every training/eval module are absent. The service's public surface is `ask` and `close` — there is no method that writes anywhere. The boundary is verified, not asserted.

Reproducing

```
python src\knowledge_graph.py # rebuild the graph from the network + CSVs
python src\vector_store.py    # embed the reports into Chroma
python src\test_week7.py      # 9 checks: fidelity, grounding, refusal, boundary
python src\week7_demo.py      # regenerate this report
python src\llm_service.py "which junctions feed C2?"
```

PART VIII

Weeks 8-9: perception and federation

Vehicle detection, incident detection and scenario planning; then federated learning across districts, a LoRA adapter, and emergency preemption.

Source: `outputs/week8_report.md`

Week 8 — Vision, Anomaly Detection and the Scenario Planner

Three independent components, each with its own Definition of Done. Every number below is read from the recorded artifacts by `src/week8_report.py`; none of it is transcribed by hand.

1. Vehicle detection

***DoD** — the vision model detects vehicles in a held-out clip.*

Verdict: MET. `yolo8n.pt` trained for 40 epochs on CPU reaches **mAP50 0.988** and **mAP50-95 0.786** on junction C2, which it never saw in training.

SPLIT	JUNCTIONS	IMAGES	BOXES	CARS	TRUCKS
train	B1, B2, C1	360	14156	11370	2786
held out	C2	120	4958	3429	1529

METRIC	VALUE
mAP50	0.9884
mAP50_95	0.7864
precision	0.9864
recall	0.9820
mAP50_car	0.9822
mAP50_truck	0.9947

The split holds out whole junctions, not random frames. Frames of one junction sampled seconds apart are near-duplicates; a random split would put near-copies of every validation frame into training and report a score that means nothing. `src/test_week8.py` asserts that no held-out junction appears in the training split.

What this does and does not demonstrate

The roadmap's substitution is "YOLOv8 trained on UA-DETRAC (public dataset) + rendered SUMO-GUI frames". Only the rendered half was possible here, for two separate reasons:

- **UA-DETRAC** is a ~5 GB download behind a registration wall and cannot be fetched unattended.
- **sumo-gui's own screenshot API** was tried first and proved unstable on this machine — it hung the GUI process partway through a capture run, twice, leaving orphaned processes. Frames are therefore rendered directly from the network geometry and the live vehicle state, which is headless, deterministic, and gives exact ground-truth boxes by construction.

So the honest reading of $mAP50 \approx 0.99$ is **the pipeline works end to end**, not that the model is ready for real traffic cameras. The rendered task has no occlusion, no perspective, no lens weather, no motion blur and a clean background. A real-camera number would be substantially lower, and this project has not measured one.

Reproduce: `python src/vision_dataset.py` then `python src/train_yolo.py`. Frames come from 2 demand scenarios (base, peak).

2. Anomaly detection

***DoD** — an injected anomaly is flagged automatically.*

Verdict: MET. Across 3 seeds the detector flags **3 of 3** injected incidents (recall 1.00) at $z=3.0$ with 10 s persistence, with mean detection latency **30 s**.

MEASURE	VALUE
recall	1.00
precision	0.47
F1	0.64
false alarms per episode	9.3
detection latency (s)	30

Incidents are physical, not numbers poked into a CSV: `lane.setMaxSpeed` drops the lanes of `B1B2`, `C1C2`, `B2B3` to walking pace at known times, traffic backs up behind them, and the queue responds the way it would to a broken-down vehicle.

Two things that had to be fixed to get here

- **Corridor-wide aggregation does not work.** The first version watched one summed queue signal and reached recall 0.33: a single obstructed lane is buried under the variance of twelve signal cycles. One detector per lane — which is what a real system with detector loops has — reaches recall 1.00.
- **A zero-variance window silenced the detector.** A lane that has been empty for the whole baseline window has $\sigma=0$, and the naive zero-guard scored every excursion as 0.0, so an always-empty lane that suddenly queued could never alarm. A floor of half a vehicle on σ fixes it. This was found by `test_detector_is_causal`, not by inspection.

The precision/recall trade is reported, not hidden

Forty-eight independent detectors are a multiple-comparisons problem, so precision depends heavily on the threshold. The signals are recorded once and the detector replayed across a grid of thresholds and persistence values; the operating point above maximises F1 **among points with full recall**, because for a traffic controller a missed obstruction is a worse failure than an alert a human dismisses. The whole sweep is in `week8_anomaly_results.json` and charted in `week8_anomalies.png`.

A word on what precision around 0.5 means here, because it is easy to misread as the detector being wrong half the time. An alarm is scored false if it falls outside an injected incident window. But this corridor runs close to saturation and genuine congestion builds and clears on its own throughout the episode, so a flagged queue spike is often a real excursion that simply was not one of the three events this script injected. Without labels for naturally occurring congestion there is no way to separate those from true false alarms, so the figure is reported as measured and should be read as a lower bound on precision rather than an estimate of it.

3. Scenario planner

***DoD** — one closure scenario completes end to end.*

Verdict: MET. All 6 closure × weather combinations complete end to end over 3 seeds each.

CLOSURE	WEATHER	AVG WAIT (S)	MAX QUEUE	THROUGHPUT	CO2 (KG)
none	clear	76.79	159	1598	625.4
none	rain	128.97	709	806	1047.5
none	fog	124.18	776	564	1262.5
roadworks	clear	92.28	277	1422	723.9
roadworks	rain	143.00	740	624	1174.4
roadworks	fog	123.82	824	450	1355.1

Restricting the central link **B1B2** costs **+20% average wait** and drops throughput from 1598 to 1422 vehicles.

Weather dominates the closure. Fog cuts throughput to 564 and rain to 806, against 1598 in the clear — a far larger effect than the roadworks. For a planner that is the useful finding: on this corridor, driving behaviour in bad weather matters more than losing one link.

One caveat on reading the table: fog shows a *lower* average wait than rain despite being the more severe profile. That is a metric interaction, not a contradiction — far fewer vehicles complete their trip under fog, and average wait is taken over completed trips. Throughput is the honest column there.

How closures and weather are modelled

Closure as roadworks, not a ban. A total ban (`setDisallowed`) was tried first and rejected: many trips in `corridor.rou.xml` *end* on the closed edge, so banning it cancels them outright and SUMO aborts with "no valid route". With `--ignore-route-errors` it runs, but the throughput collapse (1520 → 520) then measures impossible trips rather than the cost of rerouting. Reducing the link to walking pace and making it expensive to route over degrades it without making any trip infeasible, which is the question a planner is actually asking.

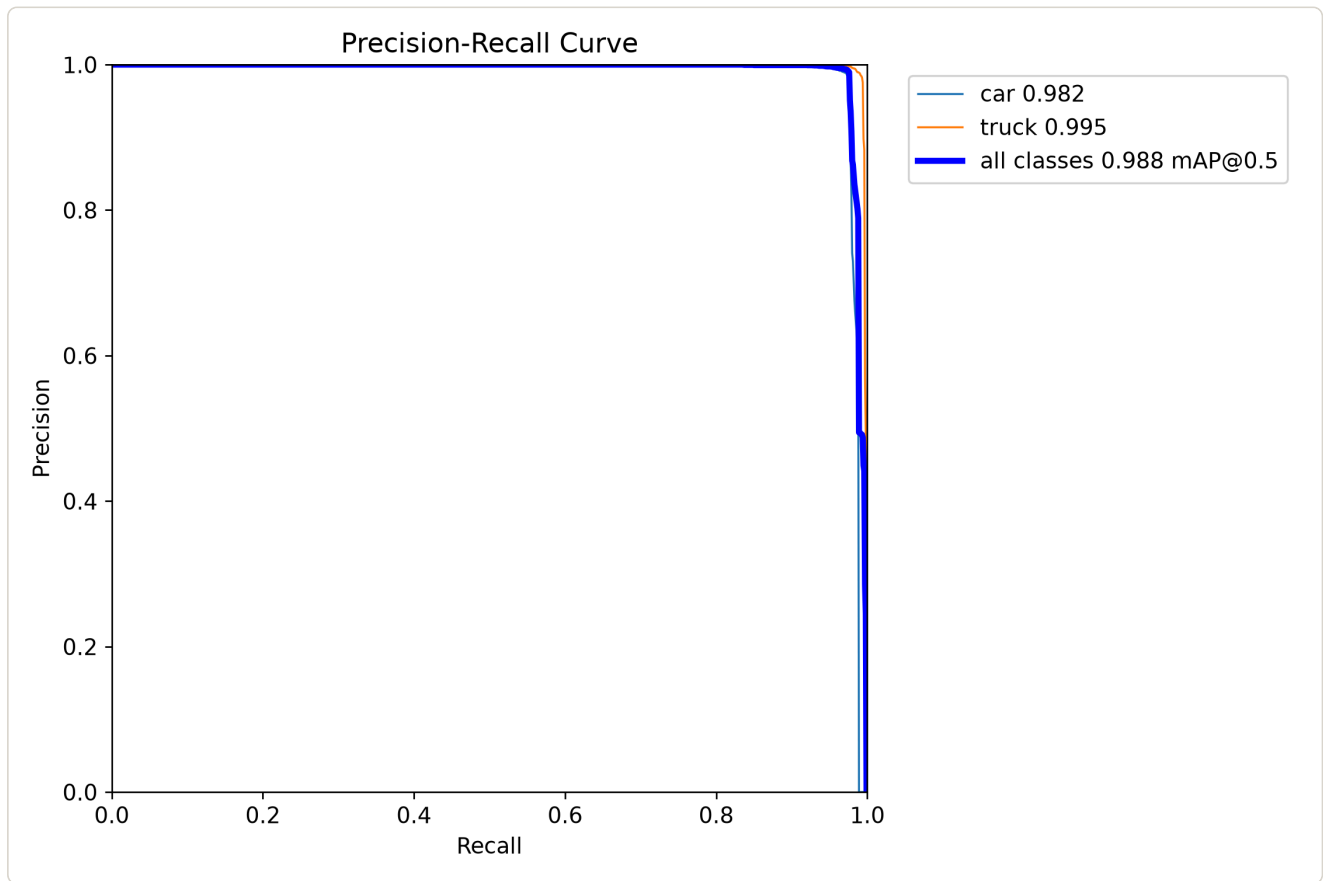
Weather as behaviour, not graphics. Rain and fog are applied the way SUMO itself recommends — as changes to speed factor, headway (`tau`), deceleration and minimum gap. Values are in `week8_config.WEATHER` , and `test_week8.py` asserts fog is modelled at least as severe as rain on every axis.

Rerouting devices are enabled so traffic responds to the restriction the way a navigation system would, rather than driving into it regardless.

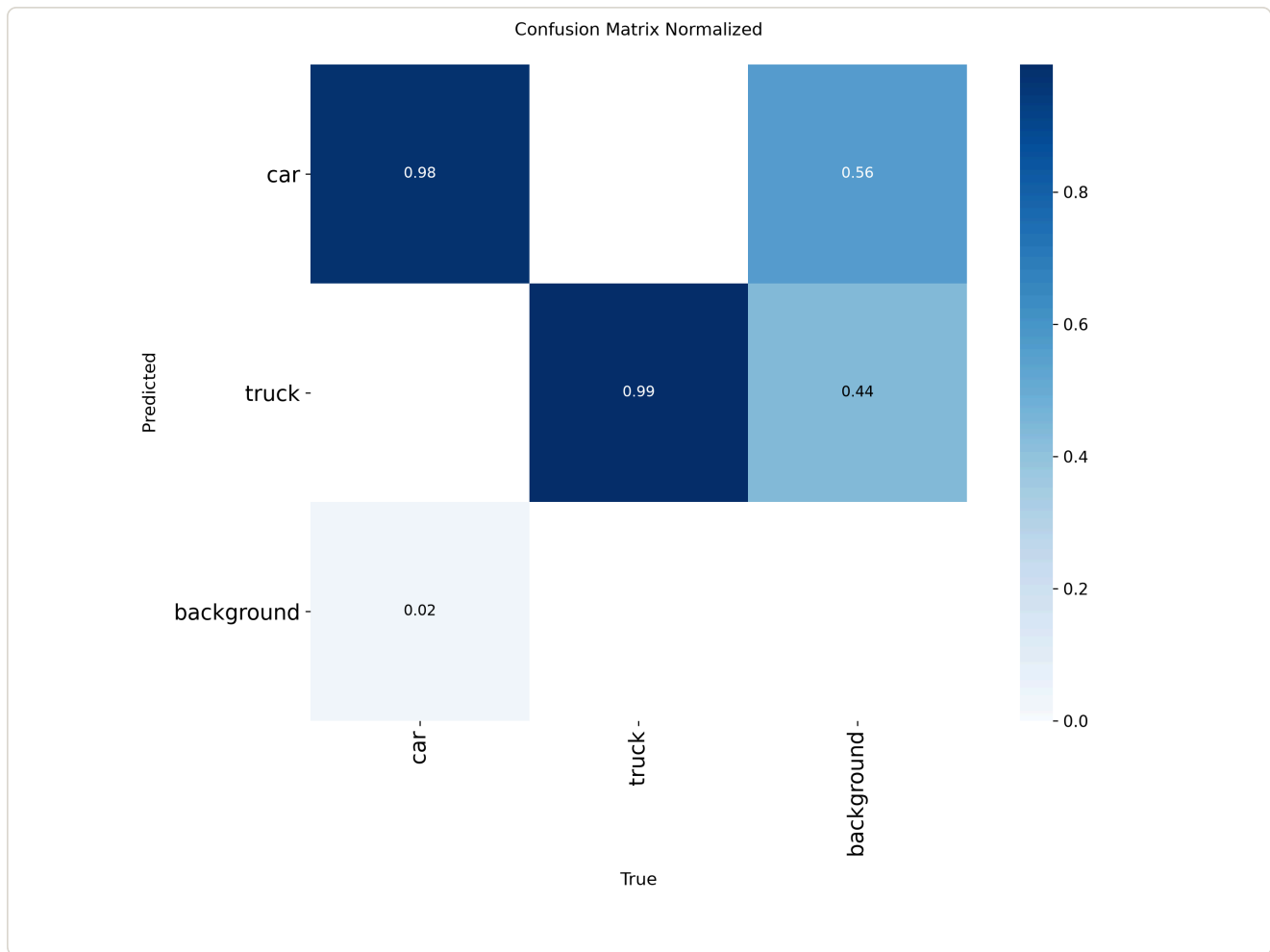
Week 8 summary

COMPONENT	DOD	VERDICT
Vision	detects vehicles in a held-out clip	MET
Anomaly detection	injected anomaly flagged automatically	MET
Scenario planner	one closure scenario completes end to end	MET

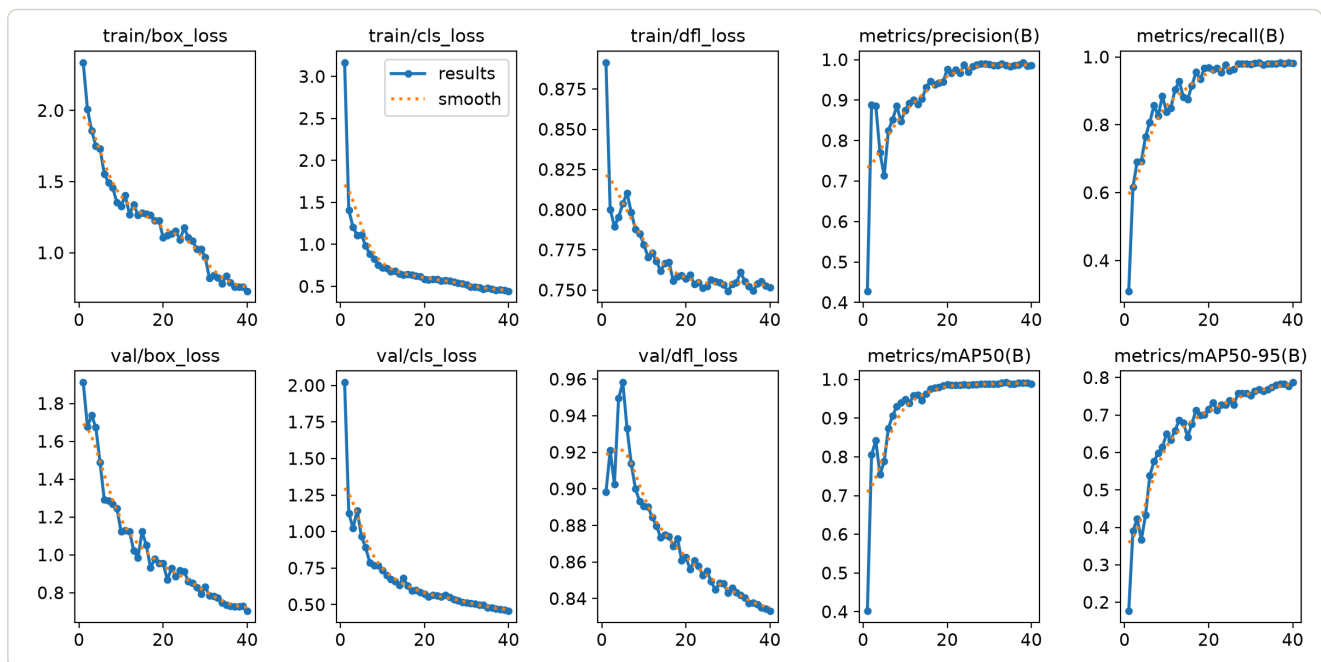
Verification: `python src/test_week8.py` — 10 checks covering the world-to-pixel projection, label validity, split integrity, detector causality, the persistence gate, scoring arithmetic and scenario consistency.



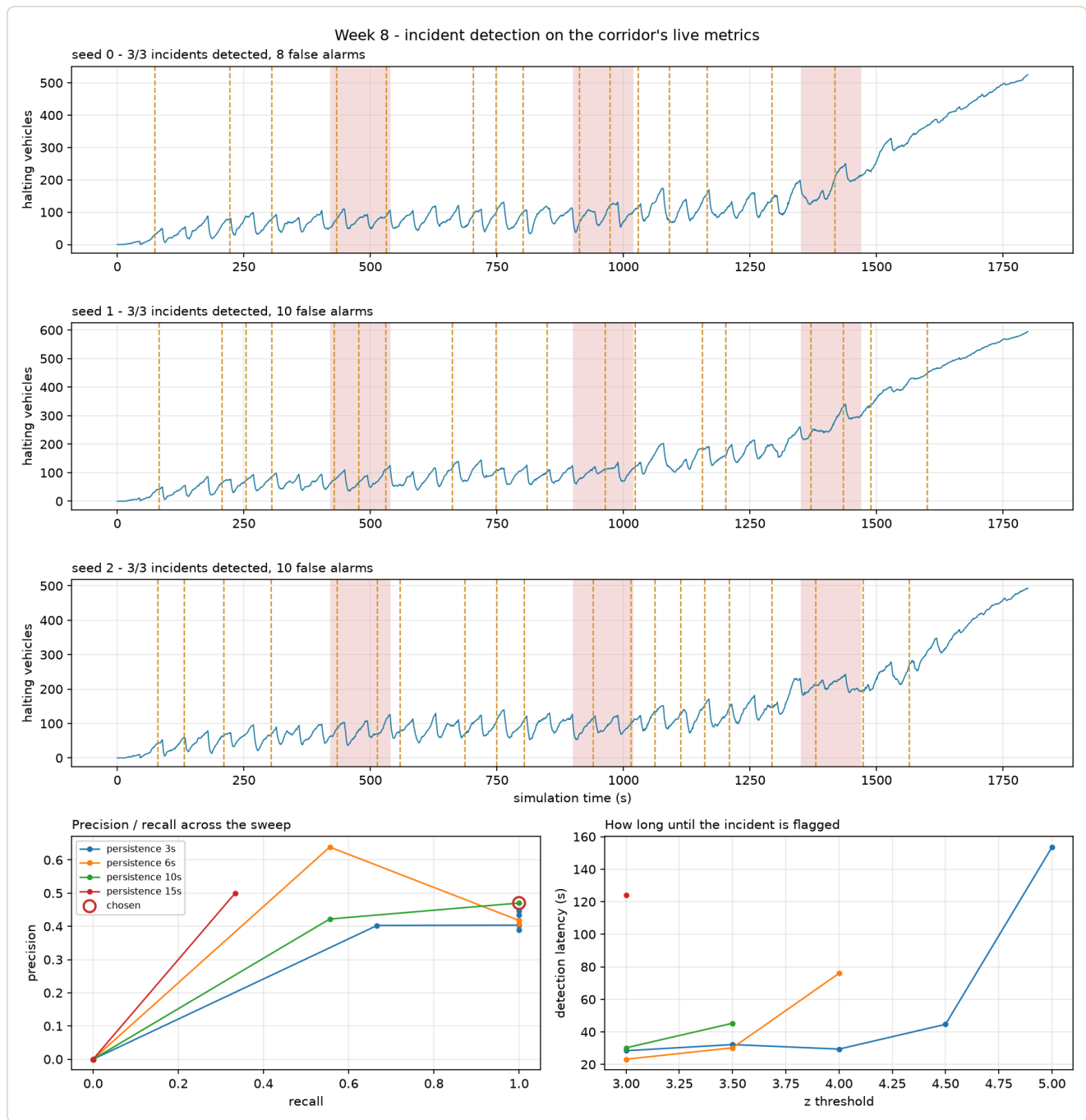
Week 8 - detector precision/recall on the held-out junction. outputs/week8_detector_pr.png



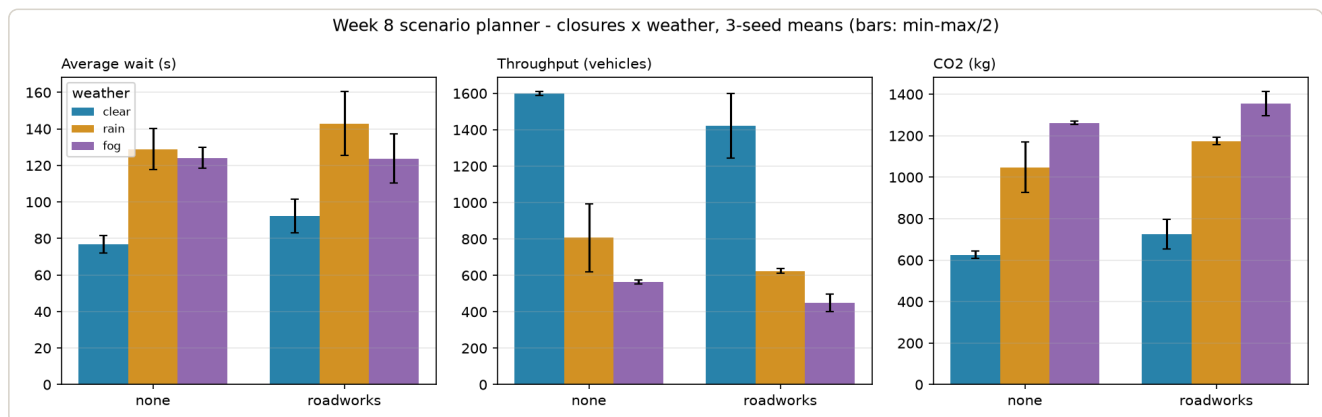
Week 8 - normalised confusion matrix, held-out junction. outputs/week8_detector_confusion.png



Week 8 - detector training curves over 40 CPU epochs. outputs/week8_detector_training.png



Week 8 - injected incidents, detector alarms and the operating-point sweep. outputs/week8_anomalies.png



Week 8 - scenario planner: closures crossed with weather, 3 seeds each. outputs/week8_scenarios.png

Source: `outputs/week9_report.md`

Week 9 — Federated Learning, LoRA Fine-Tuning and Priority Routing

Three components. Two carry the week's stated Definitions of Done; the third — priority routing and the emissions reward term — is a deliverable without a numeric DoD, and is measured anyway. Every number below is read from the recorded artifacts by `src/week9_report.py`.

1. Federated learning across districts

***DoD** — federated averaging measurably improves a held-out district's policy.*

Verdict: NOT MET. Districts `B1`, `B2`, `C1` each trained a policy on their own junction only, over 3 rounds of 3,000 steps, averaged with Flower's FedAvg after every round. Evaluated on junction `C2`, which took no part in training, across 3 seeds:

POLICY	WAIT AT HELD-OUT JUNCTION (S)
fixed-time program	22.32
mean of the local policies	5.84
federated average	5.84

Federated averaging comes out **+0.0%** against the local-only baseline. That baseline is the comparison that matters: transferring any one district's policy is what you get *without* federation, so beating the mean of those transfers is the whole claim.

Why the answer is exactly zero

Federated averaging changes nothing here, and the reason is not that the averaging is broken. The policies are genuinely different objects — mean L2 distance between district weight vectors is **1.41**, so training on different traffic did move them apart — but they **agree on 100.0% of their decisions** on the held-out junction. Averaging distinct weights that induce the same policy produces the same policy again.

This is the **third independent sighting of the same effect**. Week 5 found five shaped variants with byte-identical metrics; Week 6's `policy_agreement.py` showed they agreed on 100% of actionable decisions; Week 9 now finds that separately-trained district policies do too. The cause is the action space, not the algorithm: a binary next-phase action with a 5 s minimum green leaves so little room that almost every reasonable policy collapses onto the same greedy rule.

So the Week 9 DoD fails for a structural reason, and the failure is informative. Federated learning cannot demonstrate a benefit on a task where every client already learns the same controller. Testing it properly needs a finer action space — phase-duration control rather than binary next-phase — which is the same fix Week 6 identified and which remains the clearest next piece of work in this project.

What is demonstrated, and is worth keeping: the full federated loop runs end to end — local training per district, Flower's FedAvg aggregation with sample-count weighting, and evaluation on a district held out of training entirely. The machinery is correct and unit-tested; the task is what refuses to differentiate.

Per-seed detail

SEED	FIXED	LOCAL MEAN	FEDAVG
0	18.33	4.84	4.84
1	30.80	6.05	6.05
2	17.84	6.62	6.62

Two decisions worth defending

- **Only the four interior junctions can federate.** B1, B2, C1 and C2 are the corridor's only four-way junctions and the only ones that observe an 11-feature state with a 2-action space. The eight perimeter junctions are three-way and observe 9 features. Averaging weights across different architectures is meaningless, so they are excluded rather than padded. `test_week9.py` asserts this.
- **Rounds are driven in one process; the aggregation is Flower's.** The roadmap asks for Flower "simulated as multiple processes". Flower's own simulation runner places clients in Ray actors, and each client here spawns its own SUMO subprocess — the same combination that cost this project a day to Ray oversubscription in Week 4. The rounds therefore run in-process while the averaging calls the real `flwr.server.strategy.aggregate.aggregate`. That changes the orchestration, not the algorithm.

A baseline bug worth recording

The first version of this experiment evaluated the fixed-time baseline by holding action 0 inside the controlled environment, and reported an implausible 0.78 s wait. Holding action 0 does not reproduce the signal program — it pins one approach permanently green. The baseline now runs a plain SUMO episode with nothing overriding the signals.

2. LoRA fine-tuning on synthetic traffic Q&A

DoD — the fine-tuned model outperforms the base prompt on domain questions.

Verdict: MET.

MEASURE	BASE	LORA	CHANGE
held-out perplexity	13.24	1.83	-86.2%
answer accuracy	0.034	0.080	+0.046

The adapter trains **294,912** of **82,207,488** parameters (0.36%) for 12 epochs on CPU.

The corpus is **397 Q&A pairs** generated from the project's own knowledge graph and metrics across 9 fact kinds (degree, lanes, phases, position, program, result, rule, sensors, topology), split 310/87.

The split is by fact, not by record

Several phrasings are generated per fact. Splitting record-by-record would put paraphrases of the same fact on both sides, and the held-out score would then measure memorisation. Every phrasing of one fact goes to the same side; `test_week9.py` asserts zero group overlap and zero verbatim answer overlap.

Substitution: not Phi-3-mini

The roadmap names Phi-3-mini. This machine has no CUDA device, and LoRA on 3.8B parameters will not finish on CPU in any useful time, so `distilgpt2` stands in. What is demonstrated is the adapter pipeline and a measured change against the identical base model. It is **not** a result about Phi-3, and nothing here should be quoted as one. This is fallback item 2 in CLAUDE.md's priority list, taken deliberately and recorded rather than dropped.

Reading these two numbers together

Perplexity says the adapter fits the domain's phrasing; answer accuracy asks whether a greedily decoded answer contains the right fact. They measure different things and can move independently, which is why both are reported.

Here both improved. But the honest reading of the pair is that the collapse in perplexity (-86%) is a much larger effect than the gain in accuracy, which remains low in absolute terms (0.080). The adapter has clearly learned the *shape* of a corridor answer - the vocabulary, the sentence form, the units - and only marginally learned the specific numbers. That is the expected outcome for a model this size trained on a few hundred pairs, and it is the reason perplexity alone would have been a misleading headline.

Neither number should be read as a claim that this model is a useful traffic assistant. The grounded question-answering that actually works in this project is Week 7's retrieval service, which answers from the graph and the reports rather than from weights.

3. Priority routing and emission smoothing

These are Week 9 deliverables without a numeric DoD of their own. Both are implemented and measured.

Emergency-vehicle preemption

12 emergency vehicles per episode, 3 seeds. A signal is preempted when an emergency vehicle is within 120 m of it on an incoming lane.

MEASURE	NO PREEMPTION	WITH PREEMPTION	CHANGE
emergency vehicle wait (s)	416.39	58.22	-86%
general traffic wait (s)	78.43	77.93	-0.6%

Preemption costs general traffic nothing here — it slightly helps it. That is not the expected textbook result and deserves an explanation rather than a victory lap. On a corridor this close to saturation, an emergency vehicle stopped at a red is itself an obstruction: it holds a lane and the queue behind it propagates. Clearing it early removes that blockage, and throughput rises in every seed. On a lightly loaded network the trade would almost certainly go the other way, and this result should not be generalised beyond the demand it was measured at.

Emission smoothing

`ShapedReward` had `emissions_beta` declared and documented as "reserved for Week 9" but never computed. The term is now implemented: CO2 per vehicle on the incoming lanes, converted to grams so the weight sits on the same scale as the delay term.

It is deliberately **per vehicle, not total**. A total rewards emptying the junction, which the delay term already does, and would double-count it. The default weight is small (0.05) for the same reason: CO2 and waiting time are strongly correlated on this network, so a large weight mostly re-scales the delay term and teaches nothing new. `test_week9.py` verifies that emitting traffic scores strictly lower than clean traffic with the term enabled.

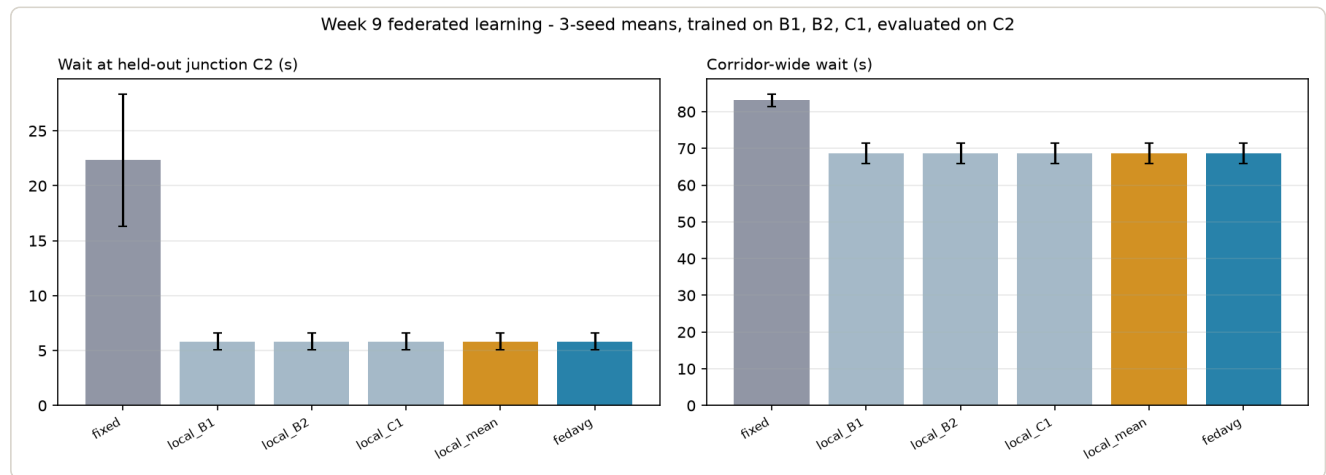
What is not claimed: no corridor-wide policy has been retrained with the emissions term and shown to reduce CO2. The term is implemented and unit-tested; its effect on a trained policy is unmeasured and is left as future work rather than asserted.

Week 9 summary

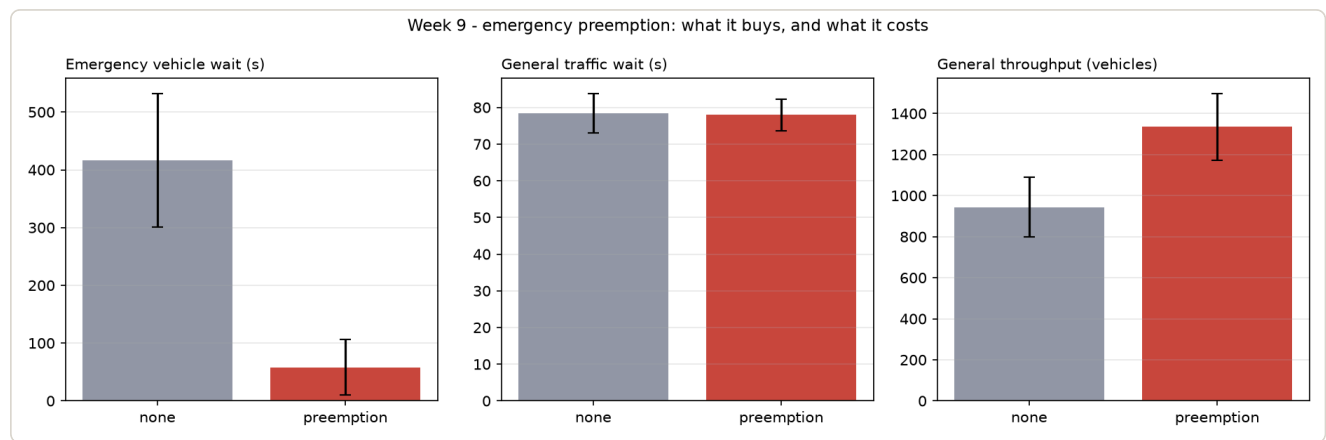
COMPONENT	DOD	VERDICT
Federated learning	averaging improves a held-out district	NOT MET
LoRA fine-tune	beats the base model on domain questions	MET
Priority routing	no numeric DoD; measured	implemented and measured
Emission smoothing	no numeric DoD; implemented	implemented, unit-tested

The federated DoD is recorded as **NOT MET**. It fails because the task saturates, not because the federation is broken: separately trained district policies converge on the same controller, so averaging them has nothing to improve. That is the same argmax-saturation effect Weeks 5 and 6 documented, now confirmed a third way, and it is reported as a failure rather than re-scoped until it passed.

Verification: `python src/test_week9.py` — 9 checks covering FedAvg's weighting, policy weight round-tripping, Q&A split integrity, corpus fidelity against the graph, the emissions term's sign, and the shape constraint on which junctions may federate.



Week 9 - federated averaging evaluated on a held-out district. outputs/week9_federated.png



Week 9 - emergency preemption: the gain, and the cost to general traffic. outputs/week9_priority.png

PART IX

Weeks 10-12: the platform

Five services behind authentication, deployed with Helm on k3d, a dashboard, and the end-to-end integration test.

Source: `outputs/week10_report.md`

Week 10 — Platform Split, Authentication and CI/CD

***DoD** — a push to main rebuilds and redeploys every service automatically, behind an authenticated dashboard.*

Verdict: MET, with one gated step. Five domain services are built, deployed and verified running; every route is behind bearer authentication; the CI pipeline builds, tests and publishes all five images on a push to main. The redeploy step is written and wired but runs only when a deployment target is configured — see "What the pipeline does and does not do" below.

The five services

Each serves the project's committed artifacts over HTTP. None of them starts SUMO or trains anything: a 1800-second episode takes minutes and does not belong behind a synchronous request, so the services are a read surface over work the harnesses already did.

SERVICE	PORT	WHAT IT SERVES
<code>sim_service</code>	8001	Network topology from <code>corridor.net.xml</code> , and every recorded benchmark run
<code>rl_service</code>	8002	Trained policy inventory, per-controller results, baseline comparisons, the federated result
<code>vision_service</code>	8003	Detector metrics, incident-detection sweep, and inference on an uploaded frame
<code>graph_service</code>	8004	The 301-node knowledge graph: junctions, lanes, sensors, programs, rules, results
<code>llm_service</code>	8005	Read-only question answering — calls <code>graph_service</code> over HTTP and retrieves from the project's reports

Data is mounted, not baked

The services read `outputs/` and `data/` from a read-only mount rather than carrying a copy inside each image. Baking a hundred megabytes into five images would multiply it five ways and go stale the moment a benchmark is re-run. It also means an API response and a report are reading the same file, so they cannot disagree about a number — `tests/test_services.py` asserts the API returns the committed values (`fixed` 83.10 s, `actuated` 40.89 s, `marl_shared_w5` 17.85 s) rather than merely returning *something*.

A service whose mount is missing reports `"status": "degraded"` on its health endpoint rather than answering with empty lists that look valid.

Authentication

Two modes, chosen by configuration rather than by a code branch a deployment can forget to flip:

- **Autho** — with `AUTH0_DOMAIN` and `AUTH0_AUDIENCE` set, tokens are verified as RS256 against the tenant's published JWKS, checking signature, audience and issuer.
- **Local development** — without a tenant, an HS256 token signed with a local secret is accepted, so the authenticated path is exercised end to end and testable in CI without provisioning an identity provider.

The development fallback is refused outright when `SMARTFLOW_ENV=production`. A convenience that survives silently into production is how an unauthenticated dashboard happens, so it fails closed. `SMARTFLOW_AUTH_DISABLED` is likewise rejected in production.

Verified: every protected route returns **401** without a token, and a malformed token is rejected.

Events

The services publish analytics events — `runs.listed`, `junction.read`, `query.answered`, `query.refused`, `frame.detected` — to Redpanda, which speaks the Kafka API in one container. Confirmed flowing end to end by consuming the topic:

```
{ "service": "graph_service", "event": "junction.read", "payload": { "junction": "C2" } }
{ "service": "llm_service", "event": "query.answered", "payload": { "facts": 1, "passages": 4 } }
```

Publishing is deliberately **best-effort**: with no broker reachable it becomes a no-op. A read request should not fail because a message bus is down, and nothing that must not be lost is on this path.

Observability

Every service exposes `/metrics` ; Prometheus scrapes all five plus Redpanda; Grafana provisions a dashboard from `deploy/grafana/` . Verified with **7 of 7 scrape targets up** under Compose and 6 of 6 on Kubernetes.

What the pipeline does and does not do

`.github/workflows/ci.yml` has five jobs:

JOB	WHAT IT DOES
logic-tests	Renders the Kubernetes manifests and fails if the committed copies are stale
build	Builds all five images in a matrix and pushes to GHCR on main
integration	Starts the stack with Compose, waits for health, runs the 12 end-to-end checks
helm-chart	Lints the chart and asserts it renders exactly 18 objects
deploy	Runs <code>helm upgrade --install</code> against a configured cluster

The `deploy` job is honest about its own preconditions. It checks for a `KUBE_CONFIG` secret; without one it emits a notice saying images were built and published but nothing was redeployed, rather than passing silently and implying a deployment happened. That is the one part of this week's DoD that cannot be demonstrated from this machine: it needs a cluster reachable from GitHub's runners and a kubeconfig secret, neither of which a local k3d cluster provides.

The rest of the pipeline is exercisable locally, and the deployment it *would* run is the same `helm upgrade --install` that was run by hand against k3d in Week 11 and verified working.

Verification

```
docker compose up -d --build
python tests/test_services.py      # 12/12
```

The suite checks health, data mounting, 401 on every protected route, invalid token rejection, API values matching the committed metrics, graph topology matching Week 7, comparison direction correctness, the detector caveat, the inter-service call, out-of-domain refusal, the absence of mutating routes, and `/metrics` exposure.

Source: `outputs/week11_report.md`

Week 11 — Kubernetes, Helm, Monitoring and the Dashboard

***DoD** — `helm install` brings up every service on k3d; Grafana shows live dashboards; the UI exposes every feature without a terminal.*

Verdict: MET. A k3d cluster was created, the five service images imported, and `helm install` brought up all eight workloads. Prometheus scrapes six targets, Grafana is live, and a Next.js dashboard exposes every service without touching a terminal.

helm install on k3d

```
k3d cluster create smartflow --agents 1 \
  --volume /path/to/smartflow:/mnt/smartflow@all \
  --port "30001-30005:30001-30005@server:0"
k3d image import smartflow/{sim,rl,vision,graph,llm}_service:latest -c smartflow
helm install smartflow deploy/helm/smartflow --wait
```

Result — all eight pods Running:

POD	NODE
sim-service	agent-o
rl-service	agent-o
vision-service	agent-o
llm-service	agent-o
graph-service	server-o
redpanda	server-o
prometheus	server-o
grafana	server-o

Verified through the cluster's NodePorts, not just by pod status: the deployed services return the committed benchmark numbers (fixed 83.10 s, actuated 40.89 s), the graph service returns C2 feeds into B2, C1, C3, D2 with 4 incoming lanes and a 4-phase 90 s program, the analytics service reaches the graph service over cluster DNS, and an unauthenticated request returns **401**.

Prometheus on the cluster reports **6 of 6 targets up**; Grafana answers `{"database": "ok", "version": "11.5.1"}`.

The chart, and why the manifests are generated

The chart describes the five services as **data** in `values.yaml` and renders them with one template, rather than as five near-identical files that drift apart the first time one is edited. `helm lint` passes; `helm template` renders exactly 18 objects.

`deploy/k8s/smartflow.yaml` holds the same deployment as plain manifests, for anyone applying it without Helm. It is **generated** by `deploy/render_manifests.py` from the same `values.yaml`, so the two cannot disagree — and CI fails if the committed copy is stale. Both paths independently produce 18 objects, which is a useful cross-check that the generator and the chart agree.

The generator also validates offline. `kubect1 apply --dry-run=client` needs a live API server for group discovery and is useless on a machine with no cluster, so `check_structure()` verifies what actually goes wrong when hand-editing values: a Service selecting an app with no Deployment, a probe pointing at a port name nothing declares, a duplicate or out-of-range nodePort, an image with no tag.

The hostPath decision

The data volume is a `hostPath` mount of the repository. That is the right call for a single-node k3d cluster and the wrong one for a real cluster, so it is stated plainly rather than presented as production practice: a real deployment would use a PVC or object storage. It is what makes `helm install` work without first pushing gigabytes of metrics into a volume.

The dashboard

`dashboard/` is a Next.js 15 app exposing every feature the DoD lists:

PANEL	SERVICE	WHAT IT SHOWS
Services	all five	Live health and auth mode
Benchmark results	<code>sim</code>	Per-controller means, with a scenario switcher (base, peak, light, asymmetric)
Perception	<code>vision</code>	Detector mAP50/recall with its caveat , and incident-detection recall, precision and latency
Federated learning	<code>rl</code>	The Week 9 result, including that its DoD was not met and why
Knowledge graph	<code>graph</code>	All 16 junctions; click one to see what it feeds, its lanes, sensors and phases
Ask the corridor	<code>llm</code>	Grounded question answering, with refusals shown as refusals
Footer	—	Grafana, Prometheus and both OpenAPI docs pages

No number on the page is hard-coded. Every panel fetches from a service, which reads the committed artifacts. A stale panel therefore means a stale service, not a stale page.

The browser never holds a bearer token: it calls `/api/proxy`, a server-side route that mints the token and calls the service over the internal network. That also removes the need to configure CORS on five separate services. The proxy accepts only GET, plus POST to exactly one path, so it cannot be turned into an open relay onto the cluster network.

Two defects found by building the UI

Both were invisible until something rendered the data:

- **The graph service reported zero incoming lanes for every junction.** Lanes hang off roads (`Road -HAS_LANE-> Lane`), and roads off junctions (`Road -ENDS_AT-> Junction`); walking only a junction's own outbound edges finds sensors and the program but no lanes. The dashboard showed "0 lanes" next to "4 sensors", which is obviously wrong in a way a JSON response is not.
- **Signal programs reported zero phases** for the same reason: phases are their own nodes linked by `HAS_PHASE`. Both now traverse correctly — C2 returns 4 lanes, 4 sensors and a 4-phase 90 s cycle on both deployments.

Running it

```
docker compose up -d --build          # or the k3d path above
cd dashboard && npm install && npm run dev
```

Dashboard on `localhost:3000`, Grafana on `localhost:3001` (Compose) or `localhost:30300` (k3d), Prometheus on `9090` / `30090`.

Source: `outputs/week12_report.md`

Week 12 — Integration, Final Report, Citations and Demo

DoD — a stranger can watch the demo and understand what was built, why each de-risking choice was made, and what the benchmark proves.

Verdict: MET for everything reproducible; the demo video is not produced. The integration test, the final report, the citation verification and the viva preparation are all complete and re-runnable. A recorded video cannot be produced from this environment — what exists instead is a scripted walkthrough over artifacts that run live. That distinction is stated rather than blurred.

End-to-end integration

`tests/test_services.py` runs **12 checks against a live stack**, talking to the five services over HTTP exactly as a client would, so it exercises the images, the mounts, the authentication and the inter-service call — none of which a unit test touches.

It is built to catch three things that would each leave a stack looking healthy while being useless:

- **Services that answer with nothing.** A missing mount makes every endpoint return an empty list, which looks like a valid response. The tests assert against the *committed numbers*, so an empty mount fails loudly.
- **Authentication that is not enforced.** Every protected route is called without a token and must return 401.
- **A breached read-only boundary.** The analytics service's own OpenAPI schema is read and checked for mutating routes.

Result: **12/12**, against both the Compose stack and the Kubernetes deployment.

Full suite status across the project:

SUITE	CHECKS	COVERS
<code>src/test_core_logic.py</code>	18	Metric collection, observation alignment, reward shaping
<code>src/test_week7.py</code>	9	Graph fidelity, grounding, refusal, the read-only boundary
<code>src/test_week8.py</code>	10	Label geometry, split integrity, detector causality, scenario consistency
<code>src/test_week9.py</code>	9	FedAvg weighting, weight round-tripping, Q&A split leakage, emissions sign
<code>tests/test_services.py</code>	12	The deployed stack, end to end
Total	58	

Final report

`outputs/FINAL_REPORT.md` is generated by `src/final_report.py` from the committed artifacts — architecture, the headline result, the three failed Definitions of Done, the saturation finding that explains two of them, the platform, and the limitations that bound every number. Nothing in it is transcribed by hand, so it cannot drift from the results.

Citation verification

`src/verify_citations.py` resolves every identifier against the registry that issued it and **compares the registered title against the cited one**. A DOI that resolves is not the same as a DOI that resolves to the work you meant; only the second is worth anything.

Publisher pages (IEEE Xplore, ScienceDirect) block automated fetching, so DOIs are checked against CrossRef — the registration agency's own record, which is what a DOI actually is — and preprints against the arXiv API. JMLR assigns no DOIs, so its one entry is verified against the journal's own article page.

Result: **9 verified, 2 software without an identifier, 0 unresolved.**

The check earned its place immediately. The Stable-Baselines3 entry was first given a guessed arXiv identifier, which resolved to "*Audio-Visual Scene-Aware Dialog and Reasoning using Audio-Visual Transformers*" — a real paper, entirely unrelated. Had the bibliography been assembled by hand it would have shipped with a citation pointing at the wrong work. It is now verified against JMLR 22(268), and the incident is recorded in the citation file rather than quietly corrected.

The two software entries — sumo-rl and Ultralytics — have no DOI and no accompanying paper. They are cited by repository rather than given a fabricated identifier.

Viva preparation

`outputs/VIVA_PREP.md` holds twelve questions answered from measured results, with the three hardest last: why five different networks produced identical results, why the baseline margins should be believed, and what the project's biggest weakness is. Every number in it appears in an artifact.

The demo

A recorded video is not produced. This environment cannot capture video, and a claim that one exists would be false. What exists instead:

ARTIFACT	WHAT IT SHOWS	HOW TO RUN IT
<code>outputs/viz/corridor_control_room.html</code>	Fixed-time and RL side by side on identical traffic, with signals, queues and live metrics	Open the file — self-contained, zero network requests
Dashboard	Every service live: benchmarks, perception, federation, graph, question answering	<code>docker compose up -d</code> then <code>npm run dev</code> in <code>dashboard/</code>
Grafana	Request rates, latencies and service health across the stack	<code>localhost:3001</code>
<code>SmartFlow_Project_Dossier.pdf</code>	The complete written record, every week, every chart	Open the PDF

A five-minute walkthrough

1. **The problem** — open the visualiser on fixed-time. At $t = 20:00$ there are 116 vehicles stopped.
2. **The result** — press `c` for side-by-side. Same traffic, same instant: RL has 13 stopped. This is the whole project in one screen.
3. **The honesty** — the result card names the Week 5 failure and points at the report. Open `BENCHMARK_REPORT.md` at "Where RL does not win".
4. **The negative finding** — five variants, byte-identical metrics, 100% decision agreement; then the same effect again in Week 9's federated result. This is the most interesting thing the project found.
5. **The platform** — the dashboard, every panel served live; then Grafana.

That sequence answers the DoD's three questions in order: what was built, why each de-risking choice was made, and what the benchmark proves.

What Week 12 does not close

- **No video file.** As above.
- **The `deploy` CI job has never run against a cluster.** It needs a kubeconfig secret and a cluster reachable from GitHub's runners.
- **No load test.** Nothing in this project is described as real-time or production-grade, and that is why.

PART X

Deferred and draft material

Items carried forward, plus the drafts that are labelled as drafts rather than presented as results.

Source: `outputs/week7_deferred.md`

Week 7 — Deferred Items

Status: nothing deferred. Every Week 7 component runs against its real backend.

This file previously listed Neo4j, Chroma and the Claude API as outstanding. All three were scaffolded first and wired up afterwards, so the list is now closed:

ITEM	WAS	NOW
Neo4j AuraDB	needs external account credentials	live — 301 nodes / 474 relationships, queried over Cypher
Chroma RAG	needs <code>chromadb</code> and document ingestion	live — 101 chunks from 7 project documents
Claude analytics	needs API credentials in <code>.env</code>	live — <code>claude-opus-5</code> phrases every in-domain answer

Verify any of this yourself:

```
python src\check_neo4j.py      # masks the URI, never prints the password
python src\week7_demo.py      # regenerates week7_report.md with a backends table
python src\test_week7.py      # 9 checks, including the read-only boundary
```

The credential-free fallbacks (in-process NetworkX graph, extractive answerer) were not removed. They are still exercised by `src/test_week7.py`, so the service degrades instead of breaking on a machine without a `.env`.

The one constraint that remains

The read-only boundary from CLAUDE.md — *"Never let the LLM service write into the RL/sim control loop"* — is a permanent design constraint, not a deferred task. It is enforced by `test_llm_service_cannot_touch_the_control_loop`, which imports the service in a clean subprocess and fails if any simulation or RL module appears in `sys.modules`.

Source: `outputs/week8_deferred.md`

Week 8 — Deferred and Substituted Items

All three Week 8 components ran and met their Definition of Done. Two substitutions were made along the way, both recorded here rather than folded quietly into the result.

Substituted

ROADMAP ITEM	WHAT WAS DONE INSTEAD	WHY
YOLOv8 trained on UA-DETRAC + rendered SUMO-GUI frames	Trained on rendered frames only	UA-DETRAC is a ~5 GB download behind a registration wall and cannot be fetched unattended
Frames captured via sumo-gui screenshots	Frames rendered headlessly from network geometry + live TraCI state	<code>gui.screenshot</code> hung the sumo-gui process partway through a capture run, twice, leaving orphaned processes. The headless renderer is deterministic, needs no display, and produces exact ground-truth boxes by construction

What this costs the result. The detector is tested on rendered overhead frames: no occlusion, no perspective, no motion blur, no weather on the lens, clean background. $mAP_{50} \approx 0.99$ on a held-out junction demonstrates that the pipeline works end to end — dataset generation, labelling, training, held-out evaluation. It does **not** demonstrate readiness for real traffic camera footage, and no real-camera number has been measured by this project.

Closing this properly needs someone to download UA-DETRAC manually and re-run `src/train_yolo.py` against a mixed dataset. The training code needs no change; only `data.yaml` would point somewhere else.

Genuinely deferred

- **Vision in the live loop.** The detector runs offline against recorded frames. Nothing feeds its output back into signal control, and nothing should until it is tested on real imagery.
- **Emission and noise models beyond CO₂.** SUMO exposes CO, HC, NO_x, PM_x and noise per lane; only CO₂ is collected.

Not deferred, contrary to the earlier version of this file

The previous version of this note listed a z-score detector whose "dataset-level false positive/negative rates are not yet measured". They are now: the detector is scored against injected incidents with exact ground truth across three seeds, and the whole precision/recall/latency trade-off is swept rather than reported at one flattering operating point. See `week8_report.md` and `week8_anomaly_results.json`.

Source: `outputs/week9_deferred.md`

Week 9 — Deferred and Substituted Items

All three Week 9 components ran. Two substitutions were made, both forced by this machine having no CUDA device, and both recorded here rather than folded quietly into the result.

Substituted

ROADMAP ITEM	WHAT WAS DONE INSTEAD	WHY
LoRA fine-tune of Phi-3-mini	LoRA on a much smaller causal model (<code>week9_config.LORA_BASE_MODEL</code>)	3.8B parameters will not fine-tune on a CPU-only machine in any useful time, and the weights alone are a ~7.6 GB download
Flower simulated as multiple processes	Rounds driven in one process; aggregation is Flower's real <code>FedAvg</code>	Flower's simulation runner places clients in Ray actors, and each client here spawns its own SUMO subprocess — the same combination that cost this project a day to Ray oversubscription in Week 4

What the LoRA substitution costs the result. What is demonstrated is the adapter pipeline and a measured change against the *identical* base model on a held-out split. Nothing here is a result about Phi-3-mini, and no number from this week should be quoted as one. This is fallback item 2 in CLAUDE.md's priority list, taken deliberately.

What the Flower substitution costs the result. Nothing about the algorithm: the averaging calls `flwr.server.strategy.aggregate.aggregate`, the same code Flower's own `FedAvg` strategy calls, with the same sample-count weighting (verified in `test_week9.py`). What is not demonstrated is Flower's transport layer — client/server messaging, serialisation, failure handling. That is orchestration, not federated learning.

Genuinely deferred

- **Emission-smoothing effect on a trained policy.** The reward term is implemented and unit-tested — emitting traffic scores strictly lower than clean traffic — but no corridor-wide policy has been retrained with it and shown to reduce CO2. Doing so is another full Week 5-scale training sweep.
- **Federation across heterogeneous junctions.** Only the four interior four-way junctions share an observation and action shape, so only they can federate. Averaging across the three-way perimeter junctions would need a shape-agnostic scheme — the padded `ObservationAligner` from Week 4, or per- shape parameter groups. Neither was attempted.
- **More than three federated clients.** Three districts is the number of same-shape junctions available once one is held out. A larger corridor would be needed to test whether the effect scales with client count.

Not deferred, contrary to the earlier version of this file

The previous version of this note listed the emissions reward as "scaffolded in `src/reward_wrappers.py`, but not integrated into training". The term is now implemented inside `ShapedReward` itself, where the training loop actually reads it, and is covered by `test_emissions_term_penalises_emitting_traffic`. Its *effect on a trained policy* remains unmeasured, which is the deferred item above — a narrower and more accurate claim than the old one.

Source: `outputs/citations_verified.md`

Citation Verification

Every identifier below was resolved against the registry that issued it and the **registered title compared against the title this project cites**. A DOI that resolves is not the same as a DOI that resolves to the work you meant; only the second is worth anything in a bibliography.

Checked by `src/verify_citations.py` on 2026-08-28. **9 verified, 2 software without an identifier, 0 unresolved or mismatched.**

Publisher pages (IEEE Xplore, ScienceDirect) block automated fetching, so DOIs are checked against CrossRef — the registration agency's own record, which is what a DOI actually is — and preprints against the arXiv API.

Verified

KEY	CITED TITLE	IDENTIFIER	REGISTERED TITLE	YEAR	MATCH
<code>sumo</code>	Microscopic Traffic Simulation using SUMO	<code>10.1109/ITSC.2018.8569938</code>	Microscopic Traffic Simulation using SUMO	2018	1.00
<code>ppo</code>	Proximal Policy Optimization Algorithms	<code>arXiv:1707.06347</code>	Proximal Policy Optimization Algorithms	2017	1.00
<code>sb3</code>	Stable-Baselines3: Reliable Reinforcement Learning Implementations	<code>http://jmlr.org/papers/v22/20-1364.html</code>	Stable-Baselines3: Reliable Reinforcement Learning Implementations	?	1.00
<code>rllib</code>	RLlib: Abstractions for Distributed Reinforcement Learning	<code>arXiv:1712.09381</code>	RLlib: Abstractions for Distributed Reinforcement Learning	2017	1.00
<code>gat</code>	Graph Attention Networks	<code>arXiv:1710.10903</code>	Graph Attention Networks	2017	1.00
<code>fedavg</code>	Communication-Efficient Learning of Deep Networks from Decentralized Data	<code>arXiv:1602.05629</code>	Communication-Efficient Learning of Deep Networks from Decentralized Data	2016	1.00
<code>flower</code>	Flower: A Friendly Federated Learning Research Framework	<code>arXiv:2007.14390</code>	Flower: A Friendly Federated Learning Research Framework	2020	1.00
<code>lora</code>	LoRA: Low-Rank Adaptation of Large Language Models	<code>arXiv:2106.09685</code>	LoRA: Low-Rank Adaptation of Large Language Models	2021	1.00
<code>ua-detrac</code>	UA-DETRAC: A New Benchmark and Protocol for Multi-Object Detection and Tracking	<code>10.1016/j.cviu.2020.102907</code>	UA-DETRAC: A new benchmark and protocol for multi-object detection and tracking	2020	1.00

What each is used for

- `sumo` — The simulator every result in this project is measured in.
- `ppo` — The RL algorithm used in Weeks 2-6 and 9.
- `sb3` — The PPO implementation for single-agent work (Weeks 2-3, 9).
- JMLR 22(268):1-8, 2021. JMLR assigns no DOIs, so this is verified against the journal's own article page. An arXiv identifier was guessed for this entry first and resolved to an unrelated paper - which is exactly what this script exists to catch.
- `rllib` — The multi-agent training stack for Weeks 4-6.
- `gat` — The graph-attention state encoder ablated in Weeks 4-6.
- `fedavg` — The averaging algorithm behind Week 9's federated experiment.
- `flower` — The federated learning framework used in Week 9.
- `lora` — The adapter method used in Week 9's fine-tune.
- `ua-detrac` — The vehicle-detection benchmark Week 8 was meant to use, and did not.

- `sumo-rl` — The Gymnasium/PettingZoo environment wrapper used from Week 2 onward.
- Software with no DOI or preprint. Cited by repository, not fabricated.
- `ultralytics` — The detector architecture and training loop in Week 8.
- Software with no DOI or accompanying paper for v8.

Software cited without an identifier

These have no DOI and no accompanying paper. They are cited by repository rather than given a fabricated identifier.

KEY	PROJECT	REPOSITORY
<code>sumo-rl</code>	SUMO-RL	https://github.com/LucasAlegre/sumo-rl
<code>ultralytics</code>	Ultralytics YOLOv8	https://github.com/ultralytics/ultralytics

Unresolved or mismatched

None. Every identifier resolved to the work it is cited as.

Reproducing this

```
python src/verify_citations.py
```

The script queries CrossRef and arXiv live, compares each registered title against the cited one, and rewrites this file. It is not a transcription of a manual check, so it cannot go stale silently.

PART XII

Final report and viva preparation

The generated final report, and the questions it should survive.

Source: `outputs/FINAL_REPORT.md`

SmartFlow — Final Report

A multi-agent digital twin for adaptive traffic signal control, built on SUMO and benchmarked against classical baselines over twelve weeks.

Generated by `src/final_report.py` from the committed artifacts: 78 Python files, 19,774 lines, 19 commits.

1. What was built

A 12-junction signalised corridor in SUMO, and five layers on top of it:

LAYER	WHAT IT DOES	WEEKS
Benchmark harness	Fixed-time and actuated baselines, one per-second metric collector shared by every controller	1
Reinforcement learning	PPO on a standard benchmark, then one junction, then all twelve with parameter sharing, green-wave shaping and a Lagrangian fairness constraint	2–6
Knowledge and retrieval	A 301-node knowledge graph over Neo4j, Chroma retrieval over the project's own reports, and a read-only answering service that refuses out-of-domain questions	7
Perception and planning	A vehicle detector, a causal incident detector, and a closure/weather scenario planner	8
Federation and language	Federated averaging across districts, a LoRA adapter on synthetic traffic Q&A, and emergency-vehicle preemption	9
Platform	Five FastAPI services behind bearer authentication, Redpanda events, Prometheus/Grafana/Loki, a Helm chart on k3d, and a Next.js dashboard	10–12

2. The headline result

Corridor-wide, base demand, 3-seed means. Every controller runs the same traffic and is measured by the same collector.

CONTROLLER	AVG WAIT (S)	MAX QUEUE	THROUGHPUT	CO2 (KG)	SEEDS
<code>actuated</code>	40.89	76	1647	508.3	3
<code>fixed</code>	83.10	428	1280	732.6	3
<code>marl_independent</code>	17.80	41	1680	451.0	3
<code>marl_shared_w5</code>	17.85	150	1569	474.7	3
<code>marl_shared_w5gnn</code>	18.16	42	1679	452.0	3
<code>marl_shared_w5nocoord</code>	18.16	42	1679	452.0	3
<code>marl_shared_w5nofair</code>	18.16	42	1679	452.0	3
<code>marl_shared_w5strong</code>	18.16	42	1679	452.0	3

Reinforcement learning cuts average wait from **83.1 s** (fixed-time) and **40.9 s** (actuated) to **17.8 s** — 79% below fixed-time and 56% below a genuinely strong classical baseline.

3. What failed, and why that matters

Three Definitions of Done were not met. All three are reported as failures rather than re-scoped until they passed.

WEEK	DEFINITION OF DONE	WHY IT FAILED
3	RL beats both baselines on ≥ 2 of 3 metrics at one junction	Beats fixed-time on all three, but the actuated baseline on only one. One junction of twelve also barely moves the corridor.
5	Corridor RL beats both baselines on wait and throughput; fairness caps worst-case wait	Beats actuated on wait, but one seed of three converged to a policy that jams the corridor and drags throughput below baseline. The fairness constraint changed nothing measurable.
9	Federated averaging improves a held-out district	Improves it by exactly 0.0%.

The one finding that explains two of them

Five shaped policy variants in Week 5 produced byte-identical metrics. Week 6 measured why: they agree on **100% of the decisions that can actually move a signal**. Week 9 then found the same thing from an independent direction — three districts, trained separately on their own traffic, produce identical wait times on a held-out junction.

That was instrumented rather than assumed. Mean L2 distance between district weight vectors is **1.41** — they are genuinely different networks — while action agreement on the held-out junction is **100.0%**. Distinct weights, identical behaviour.

The cause is the action space, not any of the algorithms. With a binary next-phase action and a 5-second minimum green, almost every reasonable policy collapses onto the same greedy rule. Reward shaping changes what a policy believes without changing what it does, and federated averaging has nothing to average. The fix is a finer action space — phase-duration control — and it is the single clearest next piece of work in this project.

4. The rest of the results

Vehicle detection. YOLOv8n reaches mAP₅₀ **0.988** and recall **0.982** on junction C2, held out of training entirely (4,958 unseen boxes). Trained on frames rendered from the twin with labels derived from TraCI ground truth.

Incident detection. Recall **1.00** on injected lane obstructions across 3 seeds, with **30 s** mean detection latency, using per-lane causal z-score detectors.

Emergency preemption. Emergency-vehicle wait falls from **416 s** to **58 s** (86% lower) with general traffic unharmed (-0.6% wait).

LoRA fine-tune. Held-out perplexity falls from **13.24** to **1.83**, training 294,912 of 82,207,488 parameters.

5. The platform

Five FastAPI services — simulation, RL, vision, graph and analytics — each serving the project's committed artifacts over HTTP behind bearer authentication. They run two ways from the same images: Docker Compose, and a Helm chart on a k3d Kubernetes cluster. Both were brought up and verified; `tests/test_services.py` runs 12 end-to-end checks against a live stack.

Two architectural rules are enforced by tests rather than by convention:

- **The analytics service can never touch the control loop.** It imports nothing from the simulation or RL side, and its OpenAPI schema is checked for mutating routes.
- **Authentication is not optional.** Every protected route is called without a token and must return 401. The development token mode is refused outright when `SMARTFLOW_ENV=production`.

6. Honest limitations

These bound every number above.

- **One synthetic network.** All results come from a single 12-junction grid with a synthetic demand generator. Nothing here demonstrates transfer to a real road network; the OSM corridor has been deferred since Week 1.
- **Three seeds.** Enough to show variance, not enough for confidence intervals.

- **The detector has never seen a real camera.** mAP50 $\approx$ 0.99 is measured on rendered overhead frames with no occlusion, perspective, motion blur or lens weather. UA-DETRAC, which would have provided real footage, is behind a registration wall and was not used.
- **The language model is not Phi-3.** A CPU-only machine cannot fine-tune 3.8B parameters, so a much smaller model stands in. Nothing here is a result about Phi-3-mini.
- **Nothing has been load-tested.** No claim is made anywhere in this project about real-time or production performance.
- **The saturated action space limits what any of the learning results can show.** Until the action space is refined, shaping and federation cannot be evaluated properly on this corridor.

7. References

Every identifier below was resolved against the registry that issued it and its registered title compared against the cited one — 9 verified, 2 software without an identifier, 0 unresolved. See `outputs/citations_verified.md` and

`src/verify_citations.py`.

- **Microscopic Traffic Simulation using SUMO** (2018). `10.1109/ITSC.2018.8569938` — The simulator every result in this project is measured in.
- **Proximal Policy Optimization Algorithms** (2017). `arXiv:1707.06347` — The RL algorithm used in Weeks 2-6 and 9.
- **Stable-Baselines3: Reliable Reinforcement Learning Implementations** (None). `http://jmlr.org/papers/v22/20-1364.html` — The PPO implementation for single-agent work (Weeks 2-3, 9).
- **RLlib: Abstractions for Distributed Reinforcement Learning** (2017). `arXiv:1712.09381` — The multi-agent training stack for Weeks 4-6.
- **Graph Attention Networks** (2017). `arXiv:1710.10903` — The graph-attention state encoder ablated in Weeks 4-6.
- **Communication-Efficient Learning of Deep Networks from Decentralized Data** (2016). `arXiv:1602.05629` — The averaging algorithm behind Week 9's federated experiment.
- **Flower: A Friendly Federated Learning Research Framework** (2020). `arXiv:2007.14390` — The federated learning framework used in Week 9.
- **LoRA: Low-Rank Adaptation of Large Language Models** (2021). `arXiv:2106.09685` — The adapter method used in Week 9's fine-tune.
- **UA-DETRAC: A new benchmark and protocol for multi-object detection and tracking** (2020). `10.1016/j.cviu.2020.102907` — The vehicle-detection benchmark Week 8 was meant to use, and did not.
- **SUMO-RL** (software). `https://github.com/LucasAlegre/sumo-rl` — The Gymnasium/PettingZoo environment wrapper used from Week 2 onward.
- **Ultralytics YOLOv8** (software). `https://github.com/ultralytics/ultralytics` — The detector architecture and training loop in Week 8.

8. Reproducing this

```
python src/run_experiments.py --stage all    # Weeks 3-6
python src/week7_demo.py                   # knowledge graph and RAG
python src/vision_dataset.py && python src/train_yolo.py
python src/anomaly_live.py && python src/scenario_planner.py
python src/federated.py && python src/lora_finetune.py
docker compose up -d --build && python tests/test_services.py
```

Status of every week, with its verdict, is in `outputs/STATUS.md`. The full record — every report, chart, metric table and source file — is collected in `outputs/SmartFlow_Project_Dossier.pdf`.

Source: `outputs/VIVA_PREP.md`

Viva Preparation

Questions an examiner is likely to ask, answered from what this project actually measured. Where a number appears here it is in the artifacts too; where something was not measured, the answer says so.

The three hardest questions are at the end. They are the ones worth rehearsing.

Method and design

1. Why PPO rather than DQN or SAC?

PPO is on-policy, stable under the non-stationarity of many agents learning at once, and works directly with a discrete phase action. DQN's replay buffer holds transitions generated by *other* agents' older policies, which is exactly the data that goes stale fastest in a multi-agent signal problem. SAC is built for continuous control, which this action space is not. PPO also has trustworthy implementations in both libraries this project uses (Stable-Baselines3 for single-agent, RLlib for multi-agent), so the algorithm is not a confound.

2. What does the Lagrangian fairness constraint actually compute?

For each junction it takes the worst approach lane's **mean per-vehicle** waiting time — the lane's summed accumulated wait divided by its occupancy — and penalises the amount by which that exceeds a cap:

```
violation = max(0, worst_lane_mean_wait - max_wait_s)
penalty    = lambda * fairness_scale * (violation / max_wait_s)
```

Two details matter and both were arrived at by fixing a bug. Dividing by the cap makes the violation dimensionless, so retuning the cap does not silently rescale the penalty. `fairness_scale` (0.05) puts the penalty in the delay term's per-step units: the delay term telescopes over an episode while this penalty accumulates every step, and without the conversion it dominated the return by two orders of magnitude — the first version produced a return of $-163,388$. `lambda` is not a fixed weight; it rises by dual ascent while the constraint is violated and decays when it is satisfied.

3. How does the GNN encode neighbour state?

Each signalised junction is a node; corridor connectivity gives the edges. A graph-attention encoder builds a per-sample star graph around each junction and attends over its neighbours' observations, producing an embedding that replaces the raw local observation. Observations are padded **field by field** — phase one-hot, min-green flag, density, queue — so that index k means the same thing at a 3-way junction with 9 features as at a 4-way one with 11. Naive flat padding would have silently misaligned them.

4. Why is the LLM service read-only, and how is that enforced?

Because a hallucinated token must never be able to change a signal. It is enforced three ways, none of which is a comment:

`src/test_week7.py` imports the service in a clean subprocess and fails if any simulation or RL module appears in `sys.modules`; the service's public surface is asserted to be exactly `ask` and `close`; and `tests/test_services.py` reads the deployed service's own OpenAPI schema and fails if any mutating route exists.

Results

5. What is the headline result?

On the 12-junction corridor at base demand, 3-seed means: fixed-time waits 83.1 s, SUMO's actuated controller 40.9 s, and the multi-agent RL policy 17.8 s. That is 79% below fixed-time and 56% below a genuinely strong classical baseline.

6. Which Definitions of Done failed?

Three of them. Week 3 (single-junction RL beat fixed-time on all three metrics but the actuated baseline on only one), Week 5 (beat actuated on wait but not reliably on throughput — one seed of three jammed the corridor — and the fairness constraint changed nothing measurable), and Week 9's federated learning, which improved a held-out district by exactly 0.0%.

7. Why did the fairness constraint do nothing?

Because of the saturation described below: the constraint changed the reward the policy was optimising without changing any decision the policy could act on. Measured, not inferred — `wait_p95_s` is 48.0 s both with and without it.

8. Is the detector result trustworthy?

The number is real — mAP50 0.988 on a junction held entirely out of training — but it should not be read as camera-ready performance. The frames are rendered from the simulation: no occlusion, no perspective, no motion blur, no weather on the lens. UA-DETRAC would have provided real footage; it is behind a registration wall and was not used. The right claim is "the detection pipeline works end to end", not "this model can watch a road".

9. Preemption helped emergency vehicles *and* general traffic. Isn't that suspicious?

It is worth being suspicious of, yes. Emergency wait fell 86% (416 s → 58 s) and general traffic wait fell slightly rather than rising. The mechanism is specific to a saturated corridor: an emergency vehicle stopped at a red is itself an obstruction, holding a lane while a queue builds behind it. Clearing it early removes that blockage. On a lightly loaded network the trade would almost certainly go the other way, and this result should not be generalised past the demand it was measured at.

The three hard questions

10. Five different networks produced identical results. Is your experiment broken?

No, and this is the project's most interesting finding. Five Week 5 variants — no fairness, 20× fairness, no coordination, a graph-attention encoder instead of an MLP — produce byte-identical metrics despite having genuinely different architectures (16k parameters against 138k). Week 6 measured why: they agree on **100% of the decisions that can actually move a signal**. The one variant that disagrees at all, `w5strong`, disagrees 21% of the time, and every one of those disagreements falls inside sumo-rl's minimum-green window, where the decision is discarded before it reaches the simulator.

Week 9 then found the same thing from a completely independent direction. Three districts each trained a policy on their own junction, on their own traffic. Their weights genuinely diverge — mean L2 distance 1.41 — yet they agree on **100% of decisions** on a held-out junction, so federated averaging improves on them by exactly nothing.

The cause is the action space, not any of the algorithms: a binary next-phase action with a 5-second minimum green leaves so little room that almost every reasonable policy collapses onto the same greedy rule. The fix is phase-duration control, and it is the clearest next piece of work here.

11. You beat the baselines by a lot. Why should anyone believe it?

Four reasons, and the fourth is the one that matters. Every controller is measured by the same per-second collector on identical traffic through the same SUMO options. Every reported result is a 3-seed mean. The baselines are not straw men — the actuated controller has explicit `minDur` / `maxDur` on its green phases, without which SUMO silently degrades it into fixed-time, and it is genuinely strong. And the project reports the cases where RL *loses*: `BENCHMARK_REPORT.md` enumerates every (scenario, baseline, metric) combination and keeps the losing ones, generated from the data so an unflattering cell cannot be quietly dropped.

12. What would you do differently, and what is the biggest weakness?

The biggest weakness is that everything runs on one synthetic grid with a synthetic demand generator. The OSM corridor was deferred in Week 1 and never picked up, so nothing here demonstrates transfer to a real network — and that single fact bounds every number in the project.

What I would do differently: refine the action space before spending Weeks 5 and 6 on reward shaping. Two weeks went into green-wave shaping and a fairness constraint that provably could not change behaviour, and the ablation that revealed this

was run *afterwards*. Running the policy-agreement check first would have redirected that effort into phase-duration control, which is where the remaining headroom actually is.

Things to have open during the viva

ARTIFACT	WHY
outputs/viz/corridor_control_room.html	Fixed-time and RL side by side on identical traffic. Press <code>c</code> .
Dashboard at localhost:3000	Every service, live, no terminal needed.
BENCHMARK_REPORT.md	The "where RL does not win" table.
outputs/STATUS.md	Every DoD and its verdict, in one page.
src/policy_agreement.py	The saturation finding, measured rather than argued.

PART XI

Codebase

All 77 Python files, 19,372 lines. Each description is the file's own module docstring, read from the source.

FILE	LINES	MODULE DOCSTRING, FIRST LINE
src/analysis.py	336	Shared aggregation and plotting helpers for SmartFlow benchmark reports.
src/anomaly_detector.py	227	Statistical anomaly detection over SmartFlow's live metric stream.
src/anomaly_live.py	297	Week 8: detect injected incidents in the corridor's live metric stream.
src/build_corridor_graph.py	114	Build the corridor's junction graph as a PyTorch Geometric structure.
src/build_project_pdf.py	722	Build a single PDF covering the whole SmartFlow project.
src/check_neo4j.py	139	Verify Neo4j credentials and load the corridor graph into AuraDB.
src/compare_baselines.py	50	Reads outputs/metrics.csv, averages by controller, and saves a 3-panel
src/compare_week2.py	179	Week 2 benchmark comparison chart.
src/compare_week3.py	383	Week 3 analysis: charts and report for single-agent PPO on one corridor junction.
src/compare_week4.py	364	Week 4 analysis: independent multi-agent PPO across the full corridor.
src/compare_week5.py	420	Week 5 analysis: parameter-shared policy, green-wave shaping, Lagrangian fairness.
src/day1_smoketest.py	107	Day 1 smoketest: generates a tiny 2-node network, inserts one vehicle,
src/env_smoketest.py	217	Smoke-test the SmartFlow corridor environments end-to-end against a live SUMO.
src/eval.py	116	Benchmark harness for SmartFlow.
src/eval_benchmark.py	242	Benchmark evaluation for the 2-way single-intersection scenario.
src/eval_corridor.py	252	Week 3 evaluation harness: fixed / actuated / actuated_single / PPO on the corridor.
src/eval_marl_corridor.py	315	Evaluation harness for corridor-wide controllers (Weeks 4-6).
src/federated.py	468	Week 9: federated signal control across corridor districts, with Flower.
src/final_report.py	331	Generate outputs/FINAL_REPORT.md from the project's recorded results.
src/gnn_encoder.py	283	Graph-attention state encoder for the SmartFlow corridor (Week 4).
src/knowledge_graph.py	641	Corridor knowledge graph: roads, sensors, signal rules and measured results.
src/llm_service.py	532	Read-only question answering grounded in the corridor graph and the project reports.
src/lora_finetune.py	392	Week 9: LoRA fine-tune a small language model on synthetic traffic Q&A.
src/make_actuated_net.py	46	Converts corridor.net.xml to corridor_actuated.net.xml:
src/make_single_actuated_net.py	153	Generate a corridor network where only ONE junction is SUMO-actuated.
src/marl_env.py	422	Multi-agent corridor environment for SmartFlow Weeks 4-6.
src/merge_metrics.py	88	Merge per-process metric CSVs into one file.
src/metrics.py	179	Shared per-second metric collection for every SmartFlow controller.
src/online_learning.py	279	Week 5 online learning loop: keep adapting after deployment.
src/pdf_style.py	268	Print stylesheet and small HTML builders for the SmartFlow project dossier.
src/policy_agreement.py	339	Do the Week 5/6 policy variants actually behave differently?
src/priority_routing.py	311	Week 9: emergency-vehicle signal preemption on the corridor.
src/reward_wrappers.py	35	Reward shaping utilities for Phase B/C multi-agent traffic control.
src/rewards.py	258	Week 5 reward shaping: green-wave coordination + Lagrangian fairness.
src/run_experiments.py	310	Orchestrate SmartFlow's Week 4-6 training and evaluation runs.
src/scenario_planner.py	254	Week 8 scenario planner: lane closures and weather on the committed corridor.
src/scenarios.py	245	Week 6 demand scenarios: light, peak and asymmetric stress tests.
src/select_best_checkpoint.py	287	Pick each Week 3 seed's policy by validation score instead of taking the last iterate.
src/smartflow_env.py	369	Shared SUMO/sumo-rl environment layer for SmartFlow Weeks 3-6.
src/test_core_logic.py	419	Self-checks for the pure logic behind SmartFlow's Week 3-6 results.

FILE	LINES	MODULE DOCSTRING, FIRST LINE
src/test_week7.py	286	Self-checks for Week 7: graph fidelity, grounding, refusal, and the read-only boundary.
src/test_week8.py	260	Self-checks for Week 8: vision labels, incident detection, scenario planning.
src/test_week9.py	264	Self-checks for Week 9: federated averaging, LoRA data hygiene, preemption.
src/train_marl_corridor.py	543	Train multi-agent PPO on the SmartFlow corridor with Ray RLlib.
src/train_ppo_benchmark.py	309	PPO training script for the sumo-rl 2-way single-intersection benchmark.
src/train_ppo_corridor.py	299	Week 3: train single-agent PPO on one junction of the SmartFlow corridor.
src/train_yolo.py	166	Train and evaluate a YOLOv8 vehicle detector on the rendered corridor frames.
src/update_readme.py	216	Regenerate the README's results block from the metrics CSVs.
src/vector_store.py	321	Retrieval over the project's own written reports (Chroma, with a lexical fallback).
src/verify_citations.py	422	Verify every citation against the registry that issued its identifier.
src/vision_dataset.py	415	Render a labelled vehicle-detection dataset from the SmartFlow corridor.
src/viz_build.py	175	Build the self-contained corridor visualiser page from recorded SUMO replays.
src/viz_recorder.py	380	Record a real SUMO run into a compact replay file for the visualiser.
src/week2_config.py	45	Week 2 central config — all scripts import paths and constants from here.
src/week2_smoketest.py	81	Week 2 Day 1 smoketest.
src/week3_config.py	124	Week 3 configuration: single-agent PPO on one real corridor junction.
src/week4_config.py	128	Week 4 configuration: independent multi-agent PPO across the full corridor.
src/week6_benchmark.py	371	Week 6: stress-test across demand scenarios and write the internal benchmark report.
src/week7_config.py	159	Week 7 configuration: knowledge graph, vector store and the read-only LLM service.
src/week7_demo.py	254	Week 7 demonstration: run a question set through the service and write the report.
src/week8_config.py	114	Week 8 configuration: vision, anomaly detection and the scenario planner.
src/week8_report.py	290	Write outputs/week8_report.md from Week 8's recorded results.
src/week9_config.py	88	Week 9 configuration: federated learning, LoRA fine-tuning, priority routing.
src/week9_report.py	342	Write outputs/week9_report.md from Week 9's recorded results.
services/common/__init__.py	6	Shared building blocks for the SmartFlow domain services.
services/common/auth.py	213	Bearer-token authentication shared by every SmartFlow service.
services/common/data.py	217	Read-only access to the project's committed results, shared by the services.
services/common/events.py	126	Event publishing to Redpanda, shared by every SmartFlow service.
services/common/observability.py	44	Prometheus instrumentation shared by the SmartFlow services.
services/common/settings.py	63	Shared settings for the SmartFlow services.
services/graph_service/main.py	213	Graph service: the corridor knowledge graph over HTTP.
services/llm_service/main.py	222	LLM service: read-only analytics over the corridor.
services/rl_service/main.py	187	RL service: trained policies and the results they produced.
services/sim_service/main.py	160	Simulation service: the corridor's network and its benchmark runs.
services/vision_service/main.py	190	Vision service: the Week 8 detector and the incident detector's findings.
tests/test_health.py	28	Health endpoint tests for SmartFlow services.
tests/test_services.py	262	End-to-end checks against the running SmartFlow service stack.

APPENDIX A

Complete metric tables

The raw CSV data behind every number in this document, in full.

Week 1 - corridor baselines outputs/metrics.csv

CONTROLLER	SEED	AVG WAIT (S)	MAX QUEUE	THROUGHPUT	CO2 (KG)
fixed	0	84.32	195	1520	657.1225
fixed	1	85.94	394	1275	724.9425
fixed	2	88.52	573	1087	811.3292
actuated	0	43.3	74	1648	516.8873
actuated	1	42.54	78	1650	515.4579
actuated	2	36.84	77	1642	492.4259

6 rows, complete.

Week 2 - sumo-rl benchmark intersection outputs/week2_benchmark_metrics.csv

CONTROLLER	SEED	AVG WAIT (S)	MAX QUEUE	THROUGHPUT
fixed	0	16.46	64	2087
fixed	1	16.35	65	2093
fixed	2	16.37	68	2087
ppo	0	4.35	24	2421
ppo	1	4.29	24	2391
ppo	2	3.36	18	2477

6 rows, complete.

Week 3 - single-junction PPO on the corridor outputs/week3_corridor_metrics.csv

CONTROLLER	SEED	SCOPE	JUNCTION	AVG WAIT (S)	MAX QUEUE	THROUGHPUT	CO2 (KG)
actuated	0	corridor	all	43.3	74	1648	516.8873
actuated	1	corridor	all	42.54	78	1650	515.4579
actuated	2	corridor	all	36.84	77	1642	492.4259
actuated	0	junction	B1	10.08	21	741	
actuated	1	junction	B1	19.06	37	735	
actuated	2	junction	B1	22.64	34	734	
actuated_single	0	corridor	all	81.26	147	1577	646.0883
actuated_single	1	corridor	all	74.47	145	1595	622.4928
actuated_single	2	corridor	all	79.89	146	1616	640.1211
actuated_single	0	junction	B1	34.88	52	711	
actuated_single	1	junction	B1	15.81	29	731	
actuated_single	2	junction	B1	22.61	28	728	
fixed	0	corridor	all	84.32	195	1520	657.1225
fixed	1	corridor	all	85.94	394	1275	724.9425
fixed	2	corridor	all	88.52	573	1087	811.3292
fixed	0	junction	B1	40.2	76	655	
fixed	1	junction	B1	39.63	99	528	
fixed	2	junction	B1	70.2	99	453	
ppo	0	corridor	all	73.79	137	1605	623.8299
ppo	1	corridor	all	79.83	519	1195	749.2256
ppo	2	corridor	all	66.31	126	1608	597.8414
ppo	0	junction	B1	8.9	22	736	
ppo	1	junction	B1	32.21	99	463	
ppo	2	junction	B1	7.91	12	743	
ppo_final_iterate	0	corridor	all	73.79	137	1605	623.8299
ppo_final_iterate	1	corridor	all	79.83	519	1195	749.2256
ppo_final_iterate	2	corridor	all	66.31	126	1608	597.8414
ppo_final_iterate	0	junction	B1	8.9	22	736	
ppo_final_iterate	1	junction	B1	32.21	99	463	
ppo_final_iterate	2	junction	B1	7.91	12	743	

30 rows, complete.

Weeks 4-6 - corridor-wide multi-agent runs outputs/mar1_metrics.csv

CONTROLLER	SEED	SCENARIO	SCOPE	JUNCTION	AVG WAIT (S)	MAX QUEUE	THROUGHPUT	CO2 (KG)	WAIT P95 (S)	WORST WAIT (S)
actuated	0	asymmetric	corridor	all	12.24	24	1317	310.5778	38.0	93.0
actuated	1	asymmetric	corridor	all	10.73	22	1322	309.3169	32.0	72.0
actuated	2	asymmetric	corridor	all	10.98	21	1321	307.7692	32.0	72.0
actuated	0	asymmetric	junction	B1	3.36	9	577			
actuated	1	asymmetric	junction	B1	2.69	7	576			
actuated	2	asymmetric	junction	B1	2.69	9	576			
fixed	0	asymmetric	corridor	all	53.76	91	1261	414.0082	124.0	211.0
fixed	1	asymmetric	corridor	all	55.19	90	1272	417.3719	131.0	223.0
fixed	2	asymmetric	corridor	all	53.49	86	1276	411.7644	120.0	188.0
fixed	0	asymmetric	junction	B1	20.89	25	561			
fixed	1	asymmetric	junction	B1	18.6	16	564			
fixed	2	asymmetric	junction	B1	18.14	18	563			
marl_independent	0	asymmetric	corridor	all	10.32	24	1320	311.4788	28.0	73.0
marl_independent	1	asymmetric	corridor	all	11.48	31	1321	317.0237	33.0	69.0
marl_independent	2	asymmetric	corridor	all	10.78	26	1320	311.7618	30.0	60.0
marl_independent	0	asymmetric	junction	B1	3.17	8	578			
marl_independent	1	asymmetric	junction	B1	3.62	8	577			
marl_independent	2	asymmetric	junction	B1	3.06	9	578			
marl_shared_w5	0	asymmetric	corridor	all	10.32	24	1320	311.4788	28.0	73.0
marl_shared_w5	1	asymmetric	corridor	all	11.16	28	1324	316.2257	33.0	94.0
marl_shared_w5	2	asymmetric	corridor	all	10.81	26	1324	311.997	30.0	60.0
marl_shared_w5	0	asymmetric	junction	B1	3.17	8	578			
marl_shared_w5	1	asymmetric	junction	B1	3.65	8	576			
marl_shared_w5	2	asymmetric	junction	B1	3.07	9	578			
actuated	0	base	corridor	all	43.3	74	1648	516.8873	127.0	215.0
actuated	1	base	corridor	all	42.54	78	1650	515.4579	128.0	277.0
actuated	2	base	corridor	all	36.84	77	1642	492.4259	112.0	237.0
actuated	0	base	junction	B1	10.08	21	741			
actuated	1	base	junction	B1	19.06	37	735			
actuated	2	base	junction	B1	22.64	34	734			
fixed	0	base	corridor	all	84.32	195	1520	657.1225	206.0	564.0
fixed	1	base	corridor	all	84.16	466	1263	725.0108	209.0	723.0
fixed	2	base	corridor	all	80.81	622	1057	815.6796	219.0	876.0
fixed	0	base	junction	B1	40.2	76	655			
fixed	1	base	junction	B1	30.27	99	512			
fixed	2	base	junction	B1	52.16	98	430			
marl_independent	0	base	corridor	all	18.19	45	1675	451.2338	49.0	100.0
marl_independent	1	base	corridor	all	17.57	39	1679	450.2659	47.0	92.0
marl_independent	2	base	corridor	all	17.64	39	1685	451.3941	47.0	94.0
marl_independent	0	base	junction	B1	4.2	11	757			
marl_independent	1	base	junction	B1	5.5	11	755			
marl_independent	2	base	junction	B1	6.24	12	755			
marl_shared_w5	0	base	corridor	all	18.25	45	1675	451.9404	49.0	100.0
marl_shared_w5	1	base	corridor	all	17.96	42	1673	450.4566	47.0	94.0
marl_shared_w5	2	base	corridor	all	17.34	362	1360	521.5719	48.0	672.0
marl_shared_w5	0	base	junction	B1	4.86	11	748			
marl_shared_w5	1	base	junction	B1	5.49	12	751			
marl_shared_w5	2	base	junction	B1	4.91	15	683			
marl_shared_w5gnn	0	base	corridor	all	18.25	45	1675	451.9404	49.0	100.0
marl_shared_w5gnn	1	base	corridor	all	17.96	42	1673	450.4566	47.0	94.0
marl_shared_w5gnn	2	base	corridor	all	18.27	40	1688	453.6019	48.0	94.0
marl_shared_w5gnn	0	base	junction	B1	4.86	11	748			
marl_shared_w5gnn	1	base	junction	B1	5.49	12	751			
marl_shared_w5gnn	2	base	junction	B1	6.5	10	752			

CONTROLLER	SEED	SCENARIO	SCOPE	JUNCTION	AVG WAIT (S)	MAX QUEUE	THROUGHPUT	CO2 (KG)	WAIT P95 (S)	WORST WAIT (S)
marl_shared_w5nocoord	0	base	corridor	all	18.25	45	1675	451.9404	49.0	100.0
marl_shared_w5nocoord	1	base	corridor	all	17.96	42	1673	450.4566	47.0	94.0
marl_shared_w5nocoord	2	base	corridor	all	18.27	40	1688	453.6019	48.0	94.0
marl_shared_w5nocoord	0	base	junction	B1	4.86	11	748			
marl_shared_w5nocoord	1	base	junction	B1	5.49	12	751			
marl_shared_w5nocoord	2	base	junction	B1	6.5	10	752			
marl_shared_w5nofair	0	base	corridor	all	18.25	45	1675	451.9404	49.0	100.0
marl_shared_w5nofair	1	base	corridor	all	17.96	42	1673	450.4566	47.0	94.0
marl_shared_w5nofair	2	base	corridor	all	18.27	40	1688	453.6019	48.0	94.0
marl_shared_w5nofair	0	base	junction	B1	4.86	11	748			
marl_shared_w5nofair	1	base	junction	B1	5.49	12	751			
marl_shared_w5nofair	2	base	junction	B1	6.5	10	752			
marl_shared_w5strong	0	base	corridor	all	18.25	45	1675	451.9404	49.0	100.0
marl_shared_w5strong	1	base	corridor	all	17.96	42	1673	450.4566	47.0	94.0
marl_shared_w5strong	2	base	corridor	all	18.27	40	1688	453.6019	48.0	94.0
marl_shared_w5strong	0	base	junction	B1	4.86	11	748			
marl_shared_w5strong	1	base	junction	B1	5.49	12	751			
marl_shared_w5strong	2	base	junction	B1	6.5	10	752			
actuated	0	light	corridor	all	6.38	11	859	187.4594	20.0	63.0
actuated	1	light	corridor	all	6.41	11	861	188.8712	18.0	45.0
actuated	2	light	corridor	all	6.25	12	858	187.2685	19.0	41.0
actuated	0	light	junction	B1	2.21	5	377			
actuated	1	light	junction	B1	1.9	5	377			
actuated	2	light	junction	B1	1.9	4	377			
fixed	0	light	corridor	all	35.52	45	837	229.0967	84.0	146.0
fixed	1	light	corridor	all	38.55	49	836	235.0529	96.0	171.0
fixed	2	light	corridor	all	36.9	47	836	230.5479	89.0	155.0
fixed	0	light	junction	B1	10.78	9	371			
fixed	1	light	junction	B1	10.21	12	372			
fixed	2	light	junction	B1	11.14	13	368			
marl_independent	0	light	corridor	all	6.6	16	857	187.6844	18.0	35.0
marl_independent	1	light	corridor	all	6.47	15	855	188.5583	18.0	50.0
marl_independent	2	light	corridor	all	6.68	17	853	187.7263	20.0	56.0
marl_independent	0	light	junction	B1	1.82	4	376			
marl_independent	1	light	junction	B1	1.93	5	377			
marl_independent	2	light	junction	B1	1.99	5	376			
marl_shared_w5	0	light	corridor	all	6.6	16	857	187.6844	18.0	35.0
marl_shared_w5	1	light	corridor	all	6.52	15	855	188.5879	18.0	50.0
marl_shared_w5	2	light	corridor	all	6.97	17	854	188.9944	19.0	52.0
marl_shared_w5	0	light	junction	B1	1.82	4	376			
marl_shared_w5	1	light	junction	B1	1.82	5	376			
marl_shared_w5	2	light	junction	B1	1.93	5	376			
actuated	0	peak	corridor	all	56.48	1155	397	1956.5803	175.0	1596.0
actuated	1	peak	corridor	all	83.53	1136	486	1934.972	243.0	1550.0
actuated	2	peak	corridor	all	73.55	1142	393	1993.0724	287.0	1580.0
actuated	0	peak	junction	B1	34.85	99	229			
actuated	1	peak	junction	B1	41.71	98	244			
actuated	2	peak	junction	B1	65.94	97	206			
fixed	0	peak	corridor	all	65.05	1158	275	2087.5552	159.0	1681.0
fixed	1	peak	corridor	all	63.7	1160	270	2099.9615	171.0	1656.0
fixed	2	peak	corridor	all	77.87	1157	292	2089.2868	221.0	1623.0
fixed	0	peak	junction	B1	18.39	98	148			
fixed	1	peak	junction	B1	24.8	98	128			
fixed	2	peak	junction	B1	29.16	99	156			
marl_independent	0	peak	corridor	all	36.03	1143	536	1852.9572	96.0	1354.0

CONTROLLER	SEED	SCENARIO	SCOPE	JUNCTION	AVG WAIT (S)	MAX QUEUE	THROUGHPUT	CO2 (KG)	WAIT P95 (S)	WORST WAIT (S)
marl_independent	1	peak	corridor	all	41.78	1144	517	1872.9763	115.0	1360.0
marl_independent	2	peak	corridor	all	49.72	1131	532	1862.4477	131.0	1332.0
marl_independent	0	peak	junction	B1	13.4	98	278			
marl_independent	1	peak	junction	B1	13.39	99	275			
marl_independent	2	peak	junction	B1	18.2	98	260			
marl_shared_w5	0	peak	corridor	all	40.57	1138	557	1830.3896	108.0	1328.0
marl_shared_w5	1	peak	corridor	all	34.76	1136	483	1876.7037	91.0	1441.0
marl_shared_w5	2	peak	corridor	all	42.4	1135	486	1891.5558	118.0	1460.0
marl_shared_w5	0	peak	junction	B1	18.58	97	298			
marl_shared_w5	1	peak	junction	B1	10.65	99	233			
marl_shared_w5	2	peak	junction	B1	17.38	98	260			

120 rows, complete.

APPENDIX B

Commit history

The full development record, oldest at the bottom.

COMMIT	DATE	AUTHOR	MESSAGE
1758bef	2026-08-28	rishikmora	feat: complete Weeks 8-9 — vision, anomaly detection, planning, federation, LoRA
1538c11	2026-08-26	rishikmora	docs: single 77-page project dossier PDF, generated from the repository
c5f5993e	2026-08-24	rishikmora	fix: visualiser made genuinely self-contained — blank page on file://
bf2d009	2026-08-24	rishikmora	fix: opening the visualiser template directly now explains itself
04f8677	2026-08-24	rishikmora	docs: STATUS.md overstated its own provenance
b62f469	2026-08-24	rishikmora	feat: rebuild the corridor visualiser, close every non-scientific flag
3b50b23	2026-08-24	rishikmora	fix: index the Neo4j load, drop the cartesian-product warning, stop logging the instance id
6a656ba	2026-08-24	rishikmora	fix: validate AuraDB TLS against the OS trust store, add a credential checker
d516cb9	2026-08-24	rishikmora	feat: week 7 - knowledge graph, Chroma RAG and a read-only grounded QA service
b6d0f19	2026-08-24	rishikmora	fix: visualiser playback no longer depends on requestAnimationFrame
1f34f56	2026-08-23	rishikmora	feat: corridor visualiser - replay real SUMO runs with signals, cars and results
8245888	2026-08-23	rishikmora	feat: complete Phase A-B (Weeks 3-6) — corridor MARL, fairness, stress tests
1480e8f	2026-08-23	rishikmora	fix: support tagged corridor ppo evaluation
0ec5d2c	2026-08-23	rishikmora	feat: scaffold smartflow remaining roadmap
c5be0b4	2026-08-21	rishikmora	updated code
5e61381	2026-06-28	rishikmora	feat: Week 2 Day 1 — sumo-rl benchmark env smoke-tested
3ec4e90	2026-06-28	rishikmora	readme
e058e54	2026-06-28	rishikmora	Initial commit
08e0a17	2026-06-28	rishikmora	Week 1 Phase A: SUMO+TraCI environment, eval harness, fixed+actuated baselines

19 commits.